Discovering New Exploit Techniques

Through Generalizing Known Exploits

by

Yihui (Kyle) Zeng

A Dissertation Presented in Partial Fulfillment
of the Requirements for the Degree
Doctor of Philosophy

Approved October 2025 by the
Graduate Supervisory Committee:

Tiffany Bao, Chair
Adam Doupé
David Brumley
Ruoyu Wang
Yan Shoshitaishvili

ARIZONA STATE UNIVERSITY

December 2025

©2025 Yihui (Kyle) Zeng

All Rights Reserved

ABSTRACT

Cyber attacks have been one of the major threats to the security and privacy of billions of Internet users. At the core of these attacks, software exploits provide malicious actors with initial access to the victim target systems, thus an important part of the attacks. However, despite the fact that exploits are important and there are abundant known exploits, they are understudied. This work aims to fill the gap. It aims to study known exploits and derive new knowledge from them in order to provide security researchers with more insights to develop defenses against software exploits.

This dissertation proposes *Exploit Generalization*, a procedure to analyze exploitation techniques used in known exploits, extract hidden knowledge in them, and derive new techniques. In particular, it hypothesizes the existence of *technique-dominating factors* behind exploitation techniques, and finding new approaches to temper with the dominating factors is essentially finding new exploitation techniques.

This dissertation also discusses three research studies following this line of work to showcase the feasibility of Exploit Generalization. These studies span from the kernel space to the user space, showing the generality of Exploit Generalization. In the three studies, the authors identified each research target's technique dominating factors respectively and derived new exploitation techniques by finding new approaches to temper with the dominating factors.

ACKNOWLEDGMENTS

First of all, I would like to thank my primary advisor, Professor Tiffany Bao. You taught me how to learn the research skill and provided me with the emotional support I much needed for the most difficult early research days. You provided me with tremendous help in the application for the Google PhD Fellowship, making the impossible possible. I will always remember you working on my proposal overnight in a coffee shop. I really appreciate it. I would not have my work done without you paving the way for my study.

Next, I would like to thank my other advisors: Professor Adam Doupé, Professor Yan Shoshitaishvili, and Professor Ruoyu Wang. I feel lucky to have you guiding me through the PhD career. I thank you for all the both high-level and technical advice. I also appreciate the freedom you gave me to participate in competitions like CTFs and Pwn2Own, which motivated me and prevented me from burning out from the research work.

I would like to thank Professor Giovanni Vigna and Professor Christopher Kruegel for giving me the opportunity to join UCSB SecLab in 2018. You led me into the field of cybersecurity and made my career possible.

Besides my advisors, I would like to thank Professor David Brumley for the insightful suggestions and comments not only for the dissertation but also for my academic career. You made me think more deeply about academic work and helped me become a better researcher.

I would like to thank Google for the Google PhD Fellowship and Arizona State University for the research assistant position. Thanks to your financial support, I could focus on and finish my research work.

I would like to thank my ASU friends: Jayakrishna Menon Vadayath, Wil Gibbs,

Zion Basque, Siyu Liu, Fangzhou Dong, Hulin Wang, Hongkai Chen, Jennifer Miller, Xiang Mei, and everyone in the SEFCOM lab. Thank you all for the help in research, CTFs, and life. I would also like to thank my UCSB friends: Chris Salls, Lukas Dresel, Fabio Gritti, Eric Gustafson, and everyone in SecLab. Thank you for teaching me research and technical skills. What you taught me interested and motivated me to continue my cybersecurity career. You impacted my life more than you could imagine.

Last but not the least, I would like to thank my family: my father, Mr. Jiacheng Zeng, and my mother, Ms. Jianying Chen. Thank you for the unconditional support in my life. Thank you for loving me. Thank you for being there for me when I go through the ups and downs. No words can describe my appreciation. Also, I would like to thank my partner, Miss Shuyi Huang. Thank you for showing up in my life. Thank you for having fun with me in the brightest days and accompanying me in the darkest days. Finally, I would like to thank Oreo, my cat, for the cutest meows in the world.

TABLE OF CONTENTS

Page

LIST OF TABLES viii

LIST OF FIGURES ix

CHAPTER

1 INTRODUCTION 1

1.1 Software Exploit Overview 3

1.2 Exploit Generalization 6

1.3 Limitation 8

1.4 Contribution 9

1.5 Outline 10

2 BACKGROUND 11

2.1 Linux Kernel Exploits 11

2.2 Control Flow Hijacking Exploits 11

2.3 Code Reuse Attack 12

3 KERNEL HEAP EXPLOIT RELIABILITY 13

3.1 Background 16

3.1.1 Kernel Memory Allocator 17

3.1.2 Kernel Task Scheduler 17

3.1.3 Context Switch 18

3.1.4 Linux Kernel Heap Exploits 18

3.1.5 Exploit Stabilization Techniques 20

3.2 Overview 21

3.3 Kernel Heap Exploit Reliability Interview 22

3.3.1 Interview Approach 23

CHAPTER Page

3.3.2 Stabilization Techniques 24

3.3.3 Expert-based Intuitions 25

3.4 Evaluating Stabilization Techniques 27

3.4.1 Experiment Setup 27

3.4.2 Experimental Results 29

3.5 Expert-based Intuition Incorrectness 32

3.6 Technique Dominating Factors 34

3.6.1 Unknown Heap Layout 34

3.6.2 Unwanted Task Migration 35

3.6.3 Unexpected Heap Usage 35

3.6.4 Unpredictable Corruption Timing 36

3.7 Kernel Heap Exploit Model 36

3.7.1 Critical Phases for Exploit Reliability 38

3.7.2 Exploit Stabilization Success and Failure 39

3.8 New Technique Discovery 42

3.8.1 Context Conservation 42

3.8.2 Compositing Stabilization Techniques 45

3.9 Discussion 46

3.10 Limitation 47

3.11 Related Work 48

3.12 Conclusion 51

4 KERNEL CONTROL-FLOW HIJACKING ATTACK 52

4.1 Background 55

4.2 Study of Existing CFH Attacks 57

CHAPTER Page

4.3 Threat Model 60

4.4 Exploitation Technique Design 61

4.4.1 Data Spillage 62

4.4.2 Data Spillage Sources 63

4.4.3 Weaponizing Spilled User Space Data 66

4.5 RetSpill vs Modern Defenses 69

4.6 Semi-Automatic RetSpill 73

4.6.1 Snapshotting the Triggering System Call 75

4.6.2 Identifying User-controllable Data 77

4.6.3 ROP Chain Generation 77

4.7 Evaluation 81

4.7.1 Data Spillage Feasibility 81

4.7.2 Semi-Automation Effectiveness 83

4.7.3 Protection Bypass 84

4.8 Defense 87

4.9 Discussion 90

4.10 Limitation 91

4.11 Related Work 92

4.12 Conclusion 94

4.13 Appendix 96

5 AUTOMATED CODE REUSE PAYLOAD GENERATION 97

5.1 Overview 101

5.1.1 Challenges 101

5.1.2 Limitations in Existing Work 106

CHAPTER Page

5.2 Design . . . 108
5.2.1 Definitions . . . 108
5.2.2 Workflow . . . 111
5.2.3 Gadget Collection . . . 112
5.2.4 Iterative Graph Optimization . . . 114
5.2.5 Chain Generation . . . 118
5.2.6 Scalability . . . 124
5.3 Evaluation . . . 125
5.3.1 x64 Evaluation . . . 126
5.3.2 Extensibility Evaluation . . . 129
5.3.3 Non-x64 Evaluation . . . 130
5.3.4 Multi-Arch Support . . . 131
5.3.5 Case Study - Linux Kernel . . . 132
5.4 Ablation Study . . . 134
5.4.1 Graph Search vs Bounded Generate-and-Test . . . 134
5.4.2 New Gadget Categories . . . 135
5.5 Discussion . . . 137
5.6 Limitation . . . 137
5.7 Related Works . . . 138
5.8 Conclusion . . . 140
6 CONCLUSION . . . 141
REFERENCES . . . 142
APPENDIX
A PERMISSION FROM CO-AUTHORS . . . 154

LIST OF TABLES

Table Page

1. Expert intuitions from the interview 26
2. Empirical evaluation of reliability techniques 30
3. Performance of Context Conservation 44
4. Performance of the proposed technique composition 46
5. Required primitives of all existing CFH techniques 57
6. RetSpill's applicability against modern defenses 69
7. The `panic_on_oops` configuration in major Linux-based OSes 70
8. Data spillage in system calls 79
9. Amount of controlled data in `poll` syscall 83
10. Overhead of the proposed defense 90
11. Techniques applied in existing CFH exploits 96
12. Capabilities of all open-source CR chaining tools 103
13. Evaluation results of all tools on x64 125
14. Evaluation against RiscyROP 129
15. Evaluation result on different architectures 130
16. Ablation study 135

LIST OF FIGURES

Figure Page

1. How mitigations interact with primitive escalation chains 5
2. The overview of our approach 21
3. The kernel heap exploit model 37
4. Kernel stack layout 58
5. IGNI design overview 71
6. CFH exploits timeline 74
7. IGNI generates ROP chain using graph search 78
8. Unhardened and hardened stack frames 88
9. Stack patch of a x64 gadget 109
10. Register moving graph 114
11. How ropbot normalizes a `jmp_mem` gadget. 115
12. The process of chaining two ROPBlocks 120
13. Register setting graph of setting {rax, rbx, rcx} 121

Chapter 1

INTRODUCTION

Software vulnerabilities are dangerous. Malicious actors may take advantage of software vulnerabilities and cause catastrophic results, such as financial losses and exposure of personal information. In 2017, the WannaCry ransomware exploited a vulnerability in the Windows kernel, encrypting victims' files and demanding a ransom, which was estimated to have caused 6 billion in financial losses [83]. Started in 2019, LockBit ransomware group has been a threat to the security of the whole Internet. It gains access to victims' computers by exploiting multiple vulnerabilities, causing millions of financial losses [130].

The most critical step in those campaigns is the exploitation of the vulnerabilities, or known as exploits. These exploits start from an initial capability provided by the vulnerabilities (*e.g.*, out-of-bound write), use different techniques to bypass protections in the target system, and eventually gain unauthorized access, even control the system completely.

Exploits are complicated and difficult to analyze, due to their nature of entanglement with low-level computer designs. As a result, they are understudied. There have been a few attempts to collect exploits, such as ExploitDB [31] and KernelCTF [41]. But there is still no comprehensive study of these exploits, showing what we can learn from these known exploits.

This dissertation aims to fill the gap. It investigates known software exploits, with the goal of discovering new exploitation techniques, revealing the weaknesses in computer software. More specifically, the thesis is that *understanding and generalizing*

known software exploits can lead to a better understanding of the target program (§ 3), identifying new attack surfaces (§ 4), and the discovery of new exploitation techniques (§ 3, § 4, § 5).

This dissertation makes contributions to system security by showing 1) how to distill knowledge from existing known exploits and extract technique dominating factors and 2) how to derive new exploitation techniques based on the understanding of the technique dominating factors. More specifically, by analyzing Linux kernel heap exploits, we identify four dominating factors for stabilization techniques, design a generic kernel heap exploit model, which led to the discovery of a new technique called Context Conservation (§ 3) that can stabilize exploit kernels. Our follow-up research indicates that its use is beyond stabilizing kernel heap exploits, it is also critical in stabilizing prefetch attacks. By inspecting Linux kernel control-flow hijacking exploits, we noted the user data accessibility is the dominating factor. With this realization, we identify a user data injection region, thus new attack vector for control-flow hijacking exploits, namely the kernel stack, that has not been explored before (§ 4). Further analysis revealed its critical nature to the security of the Linux kernel. By studying existing code reuse attack chain generation technique, we discover that gadget chainability is the dominating factor. We design a novel gadget grouping approach to guarantee chainability, which leads to a new code reuse attack payload generation technique that significantly outperforms all existing ones. Overall, we argue that known exploits contain hidden knowledge that even the exploit developers may not know explicitly. By understanding and generalizing exploits, we can extract and formalize the knowledge and use it for hardening software security.

1.1 Software Exploit Overview

An exploit consists of a *vulnerability*, and a list of *techniques*, working on *primitives*. A primitive is the capability an exploit has on the target system, such as heap out-of-bound-write, program counter (PC) control [5]. The vulnerability provides an initial primitive that is less powerful but allows attackers to achieve certain capabilities that are undefined in the original program state. A technique is a way to transfer an already gained primitive to another, through approaches like abusing design weaknesses in the target system. Through applying various techniques, attackers can gradually escalate and amplify the initial primitive into more powerful ones, such as write-what-where, and eventually gain unauthorized access and even full system compromise.

Clearly, there are two ways to eradicate software exploits: the first is to eliminate vulnerabilities, and the second is to mitigate exploitation techniques.

Unfortunately, eliminating all vulnerabilities is impractical. On the one hand, it is infeasible in theory to find and fix all vulnerabilities because being vulnerability-free is a non-trivial program property, thus making sure a target problem is vulnerability-free is an undecidable problem according to Rice's Theorem. On the other hand, eliminating vulnerabilities has been evidenced to be impractical by the fact that thousands of bugs are still being found each year despite the tremendous amount of effort spent on securing critical software programs. For example, syzbot [42], a state-of-the-art vulnerability discovery system, has been deployed to find vulnerabilities in the Linux kernel, a critical component of the modern network infrastructure. Despite having found and helped fix 6,546 bugs since 2017, in addition to significant human-driven vulnerability-discovery efforts [55, 98, 116], it still finds more than one thousand bugs each year [43]. The same trend is observed in Chrome, a browser that is used by

70% of the Internet users [112]. There have been more than 300 unique security bug reports each year for five consecutive years [44, 45, 46, 47, 48], with no indication that this trend will change soon. Both theory and practice draw the same conclusion: vulnerabilities cannot be completely avoided.

As such, the security community is turning to the other approach of stopping software exploits: mitigating exploitation techniques. The goal is to interrupt exploits' primitive escalation chains so that a program is not exploitable despite the presence of vulnerabilities. This approach has made optimistic progress towards eradicating software exploits. For example, the introduction of the stack canary protection makes stack-based overflow much difficult to exploit; pointer authentication prevents attackers from forging pointers [6]; kalloc_type in iOS makes sure attackers cannot confuse data with pointers in use-after-free vulnerabilities [2]. These mitigations make developing software exploits much more difficult, as we are seeing fewer stack-based overflow exploits and iOS exploits.

However, they have not completely stopped software exploits. We still observe exploits deployed in the wild despite the presence of modern mitigations. This is because attackers only need *one* chain of primitives to develop successful exploits. As shown in Figure 1, writing an exploit can be considered as a path-finding problem, where each node in the graph is a primitive and each edge is a technique that can escalate one primitive into another. Then the goal of writing a successful exploit is equivalent to finding a path from the initial primitive to full system compromise. An attacker only needs to find one path in the graph to carry out successful attacks.

Although attackers have an advantage in the cat-and-mouse game of eliminating exploits, it does not mean that developing mitigations against exploitation techniques is meaningless. Mitigations essentially remove edges in the graph, giving attackers

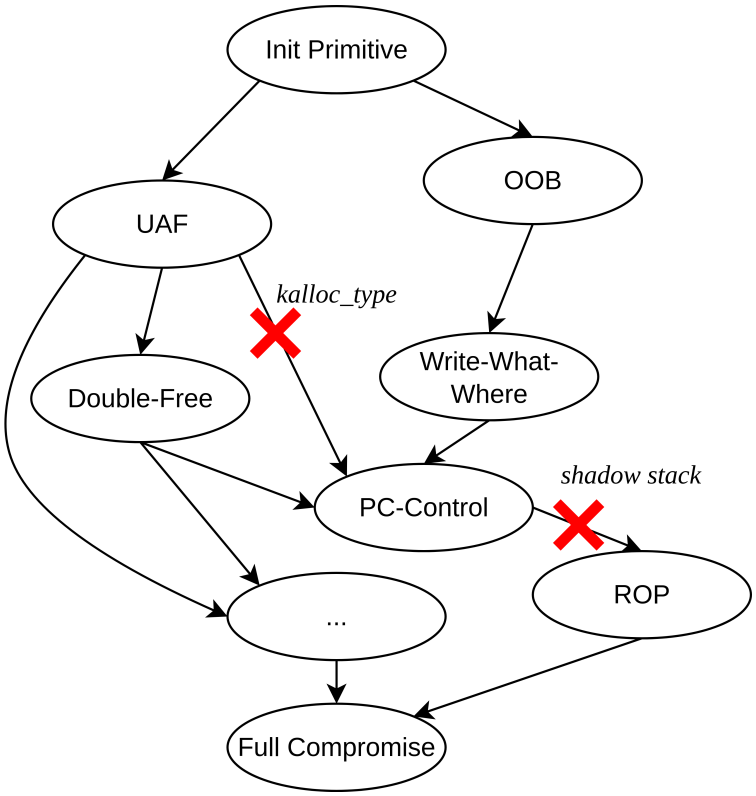

Figure 1. How mitigations interact with primitive escalation chains.

fewer choices when writing exploits. With continuous effort in developing mitigations, we may remove more and more edges in the graph so that there are fewer practical paths between the initial primitive and the full system compromise node, making exploitation much harder. Eventually, we may even stop the exploitation of some types of vulnerabilities.

As such, we are motivated to find more techniques, revealing more weaknesses in the target system, so that we can develop targeted mitigations.

1.2 Exploit Generalization

In this dissertation, we propose to understand and generalize known software exploits in order to identify more exploitation techniques, thus revealing weaknesses in the target system.

The reasoning for this study is two-fold.

First, a study of existing exploits to *extract* knowledge from them is needed. Although exploit developers can find a list of techniques to fully bypass the defenses of the target system, they may not fully understand each technique themselves. What weakness the technique reveals and what further damage this technique can lead to are often understudied. This has even happened to Google Project Zero, one of the most renowned vulnerability research groups in the world. When studying the exploitation of CVE-2022-42703 [62], they mentioned that their prefetch attack [51], an exploitation technique, cannot bypass Kernel Address Space Layout Randomization (KASLR) with the presence of Kernel Page Table Isolation (KPTI). This was proven wrong later by willsroot [132], leading to the discovery of EntryBleed [80], a prefetch attack that bypasses KASLR even with the presence of KPTI. As such, we need to study existing exploits.

Second, we need a way to *distill* existing knowledge from exploits and use it to guide the discovery of new techniques. Techniques do not stand alone, they are built upon some system features/factors and can achieve unexpected primitives. In the previously mentioned example, both the original prefetch attack and EntryBleed interact with the data caching mechanism in CPUs to infer the existence of pages at an attacker-specified address, which can be used to defeat KASLR. In other words, they both abuse the data caching mechanism as an oracle for page existence. This distilled

knowledge is important for us to understand the techniques themselves and potentially guide us to find new techniques due to the realization that the focus is on page existence. In short, there are some hidden factors within the techniques themselves that dominate their feasibility. We call these factors that dominate technique feasibility *technique dominating factors*, or dominating factors for short. In the above example, page existence is the dominating factor. By distilling the factors out of the techniques, we may be able to find new techniques by tampering with the dominating factor.

Here, we propose a three-step procedure for exploit generalization:

1. Fully understand the exploits and document the techniques and their details.
2. Derive the dominating factors by inspecting the techniques.
3. Attempts to find new approaches to temper with the dominating factors and discover new techniques.

We use the above prefetch/EntryBleed example to showcase how exploit generalization leads to the discovery of new techniques. As mentioned above, the dominating factor of prefetch/EntryBleed is page existence. EntryBleed uses system calls to load a special kernel function `entry_SYSCALL_64` that is mapped in the userspace [132] into CPU cache first and then uses the `prefetchnta` instruction to probe addresses until it hits the cache, which signals the existence of pages and the address of `entry_SYSCALL_64`. With the realization that the core is page existence in mind, we can further improve EntryBleed by getting rid of system calls because they may pollute CPU caches due to the number of instructions. Instead of using system calls, we can `prefetchnta` an address once. If the page exists, the result will be cached in the Translation Lookaside Buffer (TLB). Now we prefetch the address again. If the page exists, it will return fast, telling us the existence of the page. Although

this improvement over EntryBleed is minor, it clearly showcases the feasibility and potential of exploit generalization.

Following this line of work, we studied exploit techniques in multiple fields and discovered new techniques in the corresponding fields. Specifically, in Chapter 3, we analyzed all stabilization techniques in Linux kernel heap-based exploitation and discovered four dominating factors. Based on the discovery of the dominating factors, we discovered a new exploit stabilization technique, named Context Conservation, that can further improve exploit reliability. In Chapter 4, we analyzed existing control-flow hijacking exploitation techniques in the Linux kernel and concluded that the dominating factor is user data accessibility. By examining where user data flows to, we noticed that there is user-controllable data on the kernel stack. By accessing this on-stack user data, attackers can bypass modern protections deployed in the Linux kernel, leading to the discovery of the Retspill exploitation technique. In Chapter 5, we studied existing automated code-reuse attack generation techniques and noticed that gadget chainability is the dominating factor. We designed an algorithm to guarantee chainability with an abstraction called ROPBlock. Our research prototype, ropbot, powered by a ROPBlock-based graph-search algorithm, significantly outperforms all state-of-the-art code reuse payload generation techniques in their own respective benchmarks.

1.3 Limitation

Exploit generalization comes with limitations.

First, as we mentioned before, dominating factors are embedded in known techniques, thus in known exploits. Exploit generalization can only lead to the discovery

of new techniques that interact with known dominating factors. Thus, techniques interacting with unknown factors cannot be discovered through the process of exploit generalization.

Second, the distillation of knowledge from existing techniques, such as dominating factors, does not scale to other platforms where the original techniques are not applicable. This is because the knowledge itself may be tightly connected to the platforms themselves, despite the fact that some of the details are stripped away during technique distillation. For example, "Unwanted Task Migration" is one of the dominating factors for Linux kernel heap-based exploitation, and it does not apply to programs that do not have the task migration feature.

1.4 Contribution

In this dissertation, we learn from the abundant but understudied dataset of known software exploits to derive new exploitation techniques and assist in the hardening of software security. We show that known exploits contain hidden knowledge that can be extracted, distilled, and generalized, leading to a deeper understanding of analyzed programs and the discovery of new exploitation techniques. In summary, this dissertation makes the following high-level contributions:

- We propose the concept of exploit generalization with a concrete three-step procedure to understand exploits, extract knowledge from them, and derive new techniques based on the knowledge.
- The three-step procedure of exploit generalization can serve as a systematic study of existing techniques, leading to a deepened understanding of target programs and existing techniques.

- We followed the procedure, evaluated it on three different domains, and discovered multiple new exploitation techniques, demonstrating the feasibility and potential of this procedure.

1.5 Outline

This dissertation is structured as follows. In Chapter 2, we present the background and related work of exploitation. In Chapter 3, we discuss how understanding and generalizing known software exploits can deepen the understanding of a target program. In Chapter 4, we show that studying existing exploits can lead to the discovery of new attack surfaces. In Chapter 5, we investigate all existing code reuse attack techniques and distill their hidden knowledge, which leads to the discovery of a new exploitation technique, showcasing the importance of studying existing exploits. In the end, we conclude the dissertation in Chapter 6.

Chapter 2

BACKGROUND

In this chapter, we discuss the background in offensive software security research. Overall, security research on exploits mainly focuses on finding new exploitation techniques and automated payload generation. A systematic study is needed to provide a holistic view on found techniques and guide the direction of research.

2.1 Linux Kernel Exploits

Typically, the goal of a Linux kernel exploit is to use kernel vulnerabilities to escalate privilege from a less privileged user (*e.g.*, `nobody`) to `root`. Compared to the exploitation of userspace programs, where an attack is considered successful once a shell is achieved, Linux kernel exploitation needs to additionally consider safely returning to the user space without triggering a kernel panic or killing the escalated user space process.

2.2 Control Flow Hijacking Exploits

In the exploitation of binary executables, a Control Flow Hijacking Primitive (CFHP) is the capability where an attacker can control the target program's program counter (PC). CFHP is also commonly known as PC-control. Throughout this dissertation, we use the "CFHP" and "PC-control" terminology interchangeably. Note that we do not assume the capability of controlling registers other than the program

counter or controlling the contents pointed to by any register in this dissertation, when referring to CFHP. CFHP is commonly used in both user-space and kernel-space exploitation as a way to bootstrap more sophisticated exploitation.

2.3 Code Reuse Attack

Once attackers gain CFHP in binaries, they tend to launch code reuse attacks, such as Return-Oriented-Programming (ROP) [107] to compromise the target system. Historically, when no mitigations were deployed in programs, attackers used to inject executable code into a target program's memory and execute their code, called shellcode injection. With the introduction of Data Executable Prevention (DEP), which stopped shellcode injection, attackers resorted to chaining existing snippets in the original program executables to carry out their arbitrary payload, leading to the invention of code reuse attacks. With the increasing popularity of code reuse attacks, a few attempts [7, 28, 104, 105] were made to generate code reuse payloads in an automated fashion.

Chapter 3

KERNEL HEAP EXPLOIT RELIABILITY

Among all Linux kernel vulnerabilities, *Linux kernel heap vulnerabilities* are perhaps one of the most complicated vulnerabilities, especially that exploiting kernel heap vulnerabilities are *nondeterministic* from the attacker's viewpoint. Contrary to stack vulnerabilities where the stack layout is fixed during compilation and thus the exploits succeed deterministically, the memory layout of the heap dynamically changes *while executing exploits of heap vulnerabilities*. The kernel's heap layout can change depending on a series of factors such as the designs of dynamic memory allocation and task schedulers. As a consequence, exploits of kernel heap vulnerabilities are not guaranteed to be successful, and this unreliability can hinder the usage of exploits in an offensive context.

Exploit unreliability also negatively impacts improving system security. As demonstrated by Ohira et al. [88], software maintainers assess the security impacts of bugs manually. In fact, Arnold et al. [3] present a case study of a Debian server attack, where the maintainers did not apply the patch for eight weeks because the security impact of the bug was unclear.

This concern also affects kernel heap vulnerabilities—for instance, an initial exploit for CVE-2017-7308 has a success rate of merely 0.02% in our analysis. If a maintainer is given the exploit as a proof of a vulnerability, they may consider the bug not exploitable unless they run the exploit for up to 5000 times. The maintainer may also just deprioritize the vulnerability, although attackers may use other techniques

to improve the exploit reliability. In either case, the maintainer will underestimate the seriousness of the vulnerability and delay the defense.

Previous research studies techniques that prove the exploitability of kernel vulnerabilities—demonstrating the existence of a vulnerability by generating a testcase that works with a positive probability [16, 64, 133, 134]. However, little research has been done to address the *kernel heap exploit reliability problem*—if a kernel heap vulnerability is exploitable, how can its the success rate be improved? Although multiple kernel heap exploit stabilization techniques exist in the wild, such as Heap Grooming [87] and Defragmentation [135], little progress has been made to empirically determine whether and how to apply these kernel heap exploit stabilization techniques. Instead, human analysts leverage their personal experiences in the decision makings, and, without systematic evaluation and validation, such ad-hoc results can be not generic and even incorrect.

In this chapter, we aim to apply exploit generalization on existing kernel heap exploit stabilization techniques with the goal of finding new stabilization techniques.

As the first step, we want to systematically study existing kernel heap exploit stabilization techniques and tackle the *kernel heap exploit reliability problem*. Specifically, we aim to answer the following questions: (1) What are the common kernel heap exploit stabilization techniques in the wild? (2) What are the experts' intuitions regarding using these techniques, and what are the real-world evaluation results? Are experts' intuitions correct? (3) What are the reasons that a stabilization technique succeeds/fails? (4) How to improve kernel exploit reliability?

Answering these questions requires the access to both the expert intuitions and experimental results for kernel heap exploit reliability. To this end, we conduct a hybrid study containing both qualitative and quantitative analyses. We interviewed

11 kernel exploit experts, through which we collected 5 stabilization techniques along with the experts' opinions towards the techniques. Then, we implemented the exploits for 17 real-world kernel heap vulnerabilities and measure the reliability improvement associated with each stabilization technique. Finally, we compared the experimental results against the expert-based intuition results and investigated the factors of exploit reliability.

In our measurement, we found that many expert-based intuitions are incorrect in specific kernel heap exploit scenarios. For example, while Defragmentation was considered by experts to improve the reliability for all exploits, we discovered that Defragmentation only works for heap-based buffer overflow vulnerabilities. Even worse, Defragmentation, in fact, *harms* the reliability of other exploit types, which is the opposite of expert-based intuition gathered from the interviews.

To better understand the kernel heap exploit reliability problem, we analyze existing kernel heap exploits and discovered four factors dominating the feasibility of stabilization techniques. With the realization, we devise a generic kernel heap exploit model that incorporates the dominating factors and explains the reason that each technique succeeds or fails to improve exploit reliability. Inspired by the discovery of the dominating factors and the model, we create Context Conservation, a novel technique that is demonstrably effective in improving the exploitation reliability for double free and use-after-free vulnerabilities. Our evaluation shows that Context Conservation improves the reliability by 29% for 80% real-world use-after-free and double-free exploit scenarios. We also design a composition stabilization technique, which boosts the reliability of 97% exploit scenarios with the increase from 2% (98.06% -> 99.98%) to 67100% (0.12% -> 80.58%), and, even more, improves the reliability of 4 exploit scenarios from 0.00% rounded to 65%, 75.84%, 82.62% and 86.20%.

In summary, this application of exploit generalization makes the following contributions:

- We systematically study the existing kernel heap exploit stabilization techniques commonly used in practice, and we found that many expert-based intuitions are incorrect in deciding the use of the techniques.
- We identify four technique dominating factors and devise a general kernel heap exploit model that explains the exploit reliability problem as well as the success or failure of each stabilization technique in different exploits.
- We create a sole technique and a technique composition that both significantly improve kernel exploit reliability.

To foster future research, we release the artifacts including the technique implementation and the experiment dataset [1]. This will benefit the community by helping to stabilize future kernel exploits and evaluating future exploit stabilization techniques.

3.1 Background

In this section, we present the technical background of the kernel heap exploit reliability problem. We introduce the components related to kernel heap vulnerabilities and demonstrate the reliability issues in kernel heap exploits.

[1] https://github.com/sefcom/KHeaps

3.1.1 Kernel Memory Allocator

Kernel heap vulnerabilities are directly associated with the dynamic memory allocators in Linux kernels. Linux kernels and its kernel components use the SLAB/SLUB allocator. SLAB/SLUB organizes memory in the unit of `cache`, where kernel objects in the same cache are either of the same type or in similar sizes. Each cache consists of a set of `slabs` which are contiguous pages. When creating a new slab, the SLAB/SLUB allocator partitions the new slab into multiple `slots` and chain the slots together into a per-slab `freelist`. A CPU uses the freelist of one slab for allocation at a time. When dealloction happens, the freed slot goes back to the freelist of its original slab instead of the freelist used by the CPU.

For kernel heap exploits, the memory allocation may be corrupted when an exploit takes effect. After the corruption, the Linux kernel may turns panic if it detects the abnormality, which leads to exploit failure. Therefore, a successful exploit should be able to stop the panic before the system detects the corruption.

3.1.2 Kernel Task Scheduler

Linux kernel task scheduler decides the execution order for processes and affects the use of kernel heap shared by different processes. In Linux kernel, most tasks are scheduled by Completely Fair Scheduler (CFS) [12] which schedules processes in the granularity of threads. It treats a thread as a `task` and keeps a `runqueue` for the tasks ready to run.

In multi-core systems, each CPU maintains its own per-CPU runqueue. When the workload on CPUs are unbalanced, the kernel will migrate tasks from a heavy-workload

runqueue to a light-workload runqueue. The busier the system is, the more frequently task migration happens.

3.1.3 Context Switch

Context switch occurs when a task scheduler switches tasks for executing them based on its scheduling policy. When a scheduler starts switching contexts, the kernel decides a new task and performs a serial of operations to load the context information of the new task into a CPU, which takes a long computing time.

Context switch happens in many scenarios. One of the most common scenarios is when a timer interrupt occurs during the execution of a userspace process. Because a timer interrupt can raise at any time, context switches also can potentially occur at any time during a userspace process is executing, introducing a long-time sleep to the running process.

3.1.4 Linux Kernel Heap Exploits

In general, there are four kernel heap exploit types as follows:

Out-of-bounds Object (OOB-Object) Exploits. Given a vulnerability that can overflow kernel heap, the adversary can tamper a function pointer in the adjacent object (i.e., target object) and dereference that to hijack control flow. Besides, he/she can choose to overwrite a data pointer in the adjacent object and direct it to a memory region under his/her control. In the memory region, the adversary could construct a fake object with a function pointer referencing a malicious address.

Out-of-bounds Freelist (OOB-Freelist) Exploits. Another way to exploit OOB

write vulnerabilities is to corrupt the freelist by overwriting the address of the next freed slot stored in the metatdata header of the slot (i.e., target slot) right after the vulnerable object. This drives the kernel to chain the memory region under the adveraries' control to the freelist.

Use-after-free (UAF) Exploits. Slight different from the OOB write exploitation, the exploitation of use-after-free vulnerabilities works as follows. First, the adversary deallocates a vulnerable object and leaves the freed slot on the kernel heap. Then, he/she selects a kernel object the content of which is under control. By allocating this object to occupy the freed slot, the adversary manages to tamper the critical field (*e.g.*, function or data pointer, the address of the next freed slot). Using the same approach in the exploitation of OOB writes, the adversary can obtain the control over kernel execution.

Double-free (DF) Exploits. When a double-free vulnerability is triggered, the kernel heap is corrupted with a freelist that refers to itself, which provides an opportunity to the adversary for hijacking the control flow. First, the adversary chooses a victim object which contains critical field based on his/her expertise. After this, he/she continues to select an object of which control he/she can control. By allocating the two objects sequentially, the adversary achieves the goal of tampering the critical fields in the victim object. As a result, the adversary obtains the control of the program counter easily in the same approach as described above. Alternatively, the adversary can first allocate the object the control of which is under control to the kernel heap, and thus, direct the freelist to a memory region. Then, he/she constructs a fake object in that region for the control flow hijacking.

3.1.5 Exploit Stabilization Techniques

Kernel heap exploits is difficult to be reliable because there are many runtime factors that can potentially affect the reliability. For example, the SLAB/SLUB allocator is shared among kernel components and thus the status of it is influenced by all kernel activities including user space processes, kernel threads, and interrupts. Considering that a kernel executes in an asynchronous mechanism, the status of kernel heap is very dynamic. For a UAF exploit, for instance, right after the slot holding the vulnerable object is freed, another user space process can happen to be selected by CFS for execution. This user space process may invoke a system call requesting allocation in the kernel, which takes over the target slot. Later on, when attempting to modify the sensitive data in the vulnerable object by reoccupying the target slot, the exploit fails because the slot is no longer in free. Even worse, modern multi-core systems allows tasks to be migrated between CPUs, making the interaction between processes and SLAB/SLUB even more complicated. Since the exploit has no full control over the whole system, it is challenging to rule out all such unexpected influences and succeed reliably.

There are several techniques commonly used to improve kernel heap exploit reliability in practice. For example, Defragmentation [135] improves the exploit reliability by allocating a number of objects to take over freed slots and enforces the kernel to create a new slab. If the freelist randomization is disabled, the layout of newly created slab is predictable. Therefore, the exploit will locate the slot more precisely and the attack is more likely to succeed. In our paper, we call such techniques *kernel heap exploit stabilization techniques*, or *exploit stabilization techniques* for short.

Although the exploit stabilization techniques are used prevalently in practice, little

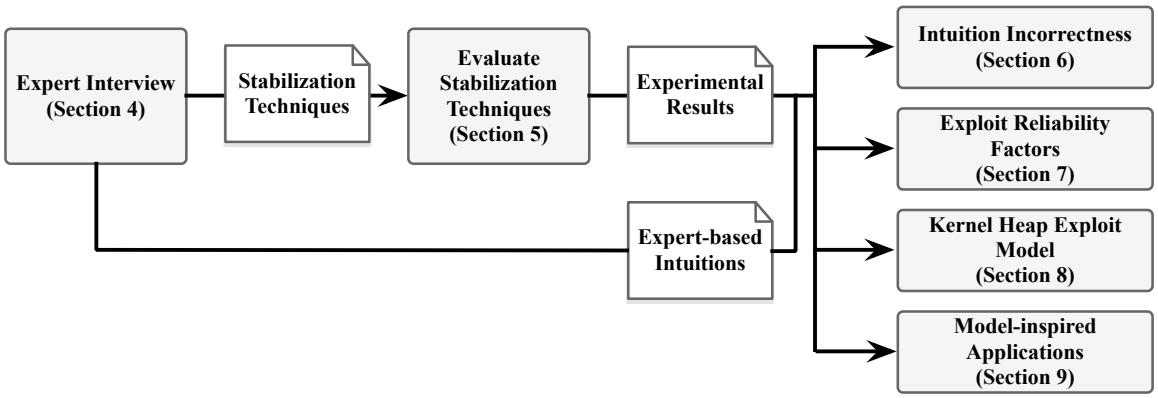

Figure 2. The overview of our approach.

research has been done to study these techniques. This lack of study inspired us to conduct a quantitative survey to collect experts' opinions on each technique and use actual experiments to check the validity of their opinions. We carried out the study aiming to deepen the understanding of these techniques for the security community. As a consequence, security analysts use these techniques in an ad-hoc way, which results in no improvement or even negative impact on kernel heap exploit reliability.

3.2 Overview

Problem Statement. In this chapter, we aim to study the kernel heap exploit reliability problem as part the exploit generalization procedure. We define the *kernel heap exploit reliability problem* as the issue that a kernel heap exploit cannot work reliably. This is based on the condition that a vulnerability is known to be present and exploitable, with a given exploit that is able to take advantage of the vulnerability *and* execute malicious payload. While previous work [16, 64, 133, 134] seeks for solutions to generate exploits that work with a positive success rate (*e.g.*, an exploit works for once within 100 times of trials), our work focuses on the problem of how to help these

generated exploits gain a higher success rate. In particular, our goal is to understand the reliability issue as well as the use of the kernel heap exploit stabilization techniques commonly used in practice.

Approach. We design a hybrid approach to study the kernel heap exploit reliability problem, as shown in Figure 2. Our approach includes both qualitative human subject study and quantitative real-world experiments. We first conduct qualitative study on security experts, through which we identify the commonly-used kernel heap exploit stabilization techniques and the expert-based intuitions for each technique. Next, we evaluate the stabilization techniques on real-world kernel heap exploit scenarios and we measure the reliability of the exploit with and without each technique. Based on the expert-based intuitions and the experimental results, we identify the intuition incorrectness, investigate the dominating factors for exploit reliability, and build a generic model to explain the kernel heap exploit reliability problem. Finally, we use the model to explore novel techniques and technique compositions that boost exploit reliability.

3.3 Kernel Heap Exploit Reliability Interview

In this section, we describe our interview for the kernel heap exploit reliability problem. We will present the interview approach, introduce the identified kernel heap exploit stabilization techniques, and summarize the expert-based intuitions.

3.3.1 Interview Approach

The goal of the interview is to 1) identify techniques commonly used by experts to improve the exploit reliability and 2) collect the empirical results regarding these techniques. As the interview involves human subjects, we obtained the IRB permission from the institute before the participants recruitment.

Given the nature of the interview that we need to exhaustively collect candidate techniques and empirical opinions, we design it in a semi-structure fashion [98]. We invited security experts through emails and introduced our study as a task aiming to study the approaches to improving the reliability of kernel exploit. In the email, we required the participants to have at least two years recent experience in exploiting Linux kernel vulnerabilities. We kept recruiting participants until the results saturate. By the time the interview results saturated, we recruited 11 participants in total.

For the semi-structured interview, we prepared a list of open questions for the participants. During the interview, we asked follow-up questions based on what the participants say in the response to the prepared questions. During the interview, we took notes on the shared document so that the participants can see what we are recording. We explicitly confirmed with the participants that the note content faithfully interprets their response.

In the interview, we first collected the techniques that the participants have ever used to make the exploit more reliable. The participants are required to explain why they deem the technique effective and describe in what condition the technique is expected to work. We also ask the participants whether they have encountered with the situation where the technique does not improve but even harm the reliability. This part of interview helps us get familiar with the techniques provided by the participants.

Next, we provided the technical details of 18 vulnerabilities in our dataset to the participants. For each vulnerability, we iterated the kernel heap stabilization technique mentioned by the participant and asked them whether they would apply the technique to the exploit of the vulnerability. More specifically, we asked them to describe how they plan to implement the technique, including what system calls they would use, whether to create a separate threads or fork multiple processes for executing the technique code, and etc. In the measurement, we followed their implementation approaches to craft the exploit variants. Last but not least, the participants were required to predict the outcome of applying the technique they described to the vulnerabilities in our dataset (Refer to Section 3.4.1 for the construction of dataset). The outcome consists of two parts. One is the effectiveness when the target for machine is in a light workload and another is that in a heavy workload. Each interview takes from one to two hours.

3.3.2 Stabilization Techniques

Based on our interview, we identified 5 stabilization techniques as described below.

1. **Defragmentation.** This technique allocates a large number of objects in the same cache as the vulnerable object with the goal of filling up all the half-full slabs and forcing the allocator to create a new slab.
2. **Heap Grooming.** To place the victim object that contains sensitive field to be adjacent to the vulnerable object, this technique initializes the kernel heap by allocating many victim objects. Then, it quickly frees one of the victim objects and allocates a vulnerable object to occupy the freed slot.
3. **Single-Thread Heap Spray.** The goal of heap spraying is to occupy a desired

slot instead of forcing the kernel to create a new slab. Single-Thread Heap Spray is performed in the same thread or in a separate thread for better modulariation, right after the triggering of use-after-free/double free vulnerability or before the triggering of out-of-bound access vulnerability.

4. **Multi-Process Heap Spray.** The purpose of Multi-Process Heap Spray is the same as Single-Thread Heap Spray. The difference lies in the implementation approach that Multi-Process Heap Spray forks multiple processes for the allocation of spray objects while Single-Thread Heap Spray employs only one thread.
5. **CPU Pinning.** By invoking This technique pins the execution of exploit threads/processes to an idle CPU by `sched_setaffinity` system call.

3.3.3 Expert-based Intuitions

We summarize the expert-based intuitions in Table 1 and present the following findings:

F1. Some opinions are shared among multiple interview participants. From the empirical results, we discover that the interview participants can have the same opinion about one technique. Taking the result 1-1 of Defragmentation as an example, seven participants think that Defragmentation can improve the reliability of all types of slab-based vulnerabilities, including use-after-free, double free, and out-of-bound write, no matter whether the triggering needs race condition or not. Although this opinion is widely acknowledged, the correctness of it has never been evaluated.

F2. Some opinions conflict with each other. It is not uncommon to observe that opinions can conflict with each other. Still taking the result 1-1 as an example,

Technique	Empirical Results
Defragmentation	1-1. Works for UAF, DF, and OOB no matter whether the triggering needs race condition [7] 1-2. Only works for OOB [3] 1-3. Only works on light workload [4] 1-4. Most effective among all techniques [1] 1-5. Even if this technique does not improve reliability, at least, it doesn't harm reliability [1]
Heap Grooming	2-1. Only works for OOB (2) 2-2. Only works on light workload [3] 2-3. More useful than Defragmentation [1] 2-4. This technique is as good as Defragmentation [1]
Single-Thread Heap Spray	3-1. Works for UAF, DF, and OOB no matter whether the triggering needs race condition [5] 3-2. Most effective among all techniques [1]
Multi-Process Heap Spray	4-1. Works for UAF, DF, and OOB no matter whether the triggering needs race condition [5] 4-2. The single-thread implementation is as good as the multiple-process implementation [1] 4-3. The single-thread implementation is better than the multiple-process implementation [2] 4-4. Most effective among all techniques [1]
CPU Pinning	5-1. Effective when the threads/processes for exploit are pinned to idle CPUs [3] 5-2. Not sure if heavy workload will influence the effectiveness [1]

Table 1. Summary of expert-based intuitions from the interview. The number in the brackets indicates the amount of participants holding the corresponding opinion.

the result 1-2 of the same technique states that Defragmentation only works for out-of-bound write vulnerability. Three participants support this claim. Obviously, the two results are inconsistent and cannot hold at the same time. This indicates that there exist misunderstandings of these commonly used techniques among the kernel exploit experts.

F3. Knowledge gap exists in the use of some techniques. For some techniques, although the participants have used them for many times to exploit different vulnerabilities, they have difficulty in predicting the outcome for specific situations.

For example, the result 4-2 of heap spraying showcases that four participants have no clue whether using multiple processes for implementing spraying can improve the reliability.

3.4 Evaluating Stabilization Techniques

We evaluate the effectiveness of techniques collected from the interview. First, we construct a dataset of real-world vulnerabilities with public exploits. Following the implementation approaches suggested in the interview, we craft the exploit variants for each technique. Then, we develop a platform and run the public exploits and the corresponding variants for 5000 times (10 rounds and 500 times in each round) to obtain the success rate. Finally, we compare the success rates and analyze the experiment results.

3.4.1 Experiment Setup

Dataset. To obtain the test cases for our experiment, we exhaustively search for public exploits online (*e.g.*, [66]) and from the released repositories of previous research works [16, 17, 133, 134] focusing on Linux kernel exploitation. We extracted exploits against kernel heap to form the dataset. For these vulnerabilities and exploits, we require them to meet the following criteria. ❶ The public exploits can demonstrate the capability of obtaining control flow hijacking primitive which indicates successful exploitation. ❷ They must be able to backported to to v4.15 kernel and still showcase success after migration. This helps us avoid noise from version update. We introduce the vulnerability back to the base kernel by applying reverse patches. ❸

The exploitation of the vulnerabilities does not rely on special hardware devices or emulators. In total, we gathered 17 vulnerabilities and corresponding public exploits in our dataset. We argue that this dataset is representative not only because it includes common vulnerability types on kernel heap but also it is the largest corpus of working Linux kernel exploits at the time of the experimentation.

From the public exploits, we construct exploit variants in the following two steps. First, we strip reliability techniques already implemented away from the public exploits and form the baseline. Second, we implement each technique collected in the interview following the suggestions from participants and thus craft the exploit variants. Each variant encloses only one technique. During the variants construction, we keep the modifications on baseline exploits to the minimal to avoid unexpected noises due to exploit structure change. Note that we skip the construction of exploit variants if the technique cannot be implemented in the corresponding vulnerability. Finally, we attain 80 exploits for the evaluation.

Platform Design. We built a platform to measure the success rate of exploitation. This platform automatically runs the exploits of all vulnerabilities and collects the panic log to determine whether the exploit succeeds or not. To evaluate the influence of workload, the platform boots Apache from the Phoronix benchmark [91] to simulate the busy status of victim system.

Our platform has 4 servers, each of which has Intel(R) Xeon(R) CPU E5-2670 v2 @ 2.50GHz (20 cores in total) and 252G memory, running Ubuntu 18.04 LTS. For each exploit, we compiled the corresponding kernel and disk image and run it in `QEMU 5.0.0` virtual machine (VM). We configure the VM with 2 CPU cores and 2GB RAM. Admitted that the success rate of each exploit we obtain from VMs may be different from that on bare-metal machines. However, we argue that the change of

reliability (*e.g.*, improve or degrade) is consistent. Therefore, by observing the results from VM, we can reasonably draw conclusions.

For each exploit, the platform runs it for 5000 times (10 rounds and 500 times per round) to ensure accuracy. We use the success rate of exploits as the indicator of their reliability. We calculate the average value of 5000 times and present them in Table 2.

3.4.2 Experimental Results

Table 2 shows the experiment results of exploits in both idle and busy system. As we can see in the table that, the technique with the best performance is Multi-Process Heap Spray in busy system, which improves the reliability of all vulnerabilities that are exploited with Multi-Process Heap Spray implemented. Even for the worst case - Defragmentation in idle system, it improves the reliability for 6 out of 12 vulnerabilities. Second, we can observe that the outcomes of idle and busy system are generally consistent. Without scrutiny, the most effective technique in idle system is Multi-Process Heap Spray which improves the reliability of 14 vulnerabilities. In busy system, Multi-Process Heap Spray also outperforms all others. This indicates that the techniques we collected from the interview are all effective no matter whether they are used in idle or busy system.

For OOB vulnerabilities, the reliability of exploits can be largely improved by Defragmentation, Heap Grooming, Single-Thread Heap Spray, and Multi-Process Heap Spray. This is because these four techniques are able to reduce entropy of heap status and create a clear heap for the exploitation. Among the four techniques, the most effective one is Heap Grooming which is able to improve reliability to the largest

Type	CVE	Base	Defrag	Heap Groom	Single-Thr Heap Spray	Multi-Proc Heap Spray	CPU Pinning
				In Idle State			
OOB	2017-7533	22.32%	48.44% [+]	N/A	97.96% [+]	99.34% [+]	21.22% [−]
OOB	2017-7184	31.46%	99.46% [+]	99.96% [+]	45.00% [+]	96.48% [+]	53.74% [+]
OOB	2016-6187	34.28%	99.56% [+]	N/A	N/A	N/A	37.46% [+]
OOB	2010-2959	21.44%	99.84% [+]	98.98% [+]	67.60% [+]	53.52% [+]	25.16% [+]
OOB	2017-7308	0.02%	0.28% [+]	N/A	0.00% [−]	3.90% [+]	0.10% [+]
UAF	2018-6555	0.12%	0.08% [−]	N/A	61.46% [+]	68.92% [+]	0.30% [+]
UAF	2016-8655	96.78%	94.38% [−]	N/A	99.46% [+]	99.74% [+]	99.76% [+]
UAF	2017-15649	13.00%	12.48% [−]	N/A	64.20% [+]	99.24% [+]	99.94% [+]
UAF	2016-4557	99.86%	N/A	N/A	100.00% [+]	99.66% [−]	99.48% [−]
UAF	2017-8824	96.86%	N/A	N/A	97.70% [+]	97.46% [+]	97.80% [+]
UAF	2016-0728	2.26%	5.68% [+]	N/A	77.46% [+]	99.98% [+]	7.20% [+]
UAF	2017-10661	37.56%	N/A	N/A	45.98% [+]	83.32% [+]	79.34% [+]
UAF	2016-10150	0.00%	N/A	N/A	0.00% [=]	78.56% [+]	0.00% [=]
UAF	2017-11176	98.06%	N/A	N/A	99.80% [+]	100.00% [+]	95.32% [−]
DF	2017-2636	96.34%	26.54% [−]	N/A	99.70% [+]	99.56% [+]	90.56% [−]
DF	2017-6074	88.50%	84.32% [−]	N/A	88.84% [+]	88.20% [−]	86.94% [−]
DF	2017-8890	0.00%	0.00% [=]	N/A	0.00% [=]	69.22% [+]	0.00% [=]
		Total	6-5-1	2-0-0	13-1-2	14-2-0	10-5-2
				(Improvement–Degradation–No change)			
				In Busy State			
OOB	2017-7533	7.48%	21.32% [+]	N/A	85.98% [+]	92.60% [+]	8.62% [+]
OOB	2017-7184	40.90%	98.98% [+]	99.70% [+]	51.20% [+]	97.08% [+]	42.66% [+]
OOB	2016-6187	39.62%	95.08% [+]	N/A	N/A	N/A	35.78% [−]
OOB	2010-2959	7.18%	93.56% [+]	57.92% [+]	29.82% [+]	69.04% [+]	10.60% [+]
OOB	2017-7308	0.00%	2.08% [+]	N/A	0.12% [+]	3.58% [+]	0.02% [+]
UAF	2018-6555	0.20%	0.20% [=]	N/A	29.04% [+]	69.36% [+]	0.78% [+]
UAF	2016-8655	16.02%	14.60% [−]	N/A	32.86% [+]	97.62% [+]	63.46% [+]
UAF	2017-15649	14.30%	14.78% [+]	N/A	50.96% [+]	86.58% [+]	97.96% [+]
UAF	2016-4557	36.04%	N/A	N/A	96.40% [+]	99.00% [+]	47.12% [+]
UAF	2017-8824	87.66%	N/A	N/A	89.62% [+]	98.92% [+]	99.74% [+]
UAF	2016-0728	0.10%	0.86% [+]	N/A	62.88% [+]	99.92% [+]	0.20% [+]
UAF	2017-10661	1.18%	N/A	N/A	39.28% [+]	77.92% [+]	2.34% [+]
UAF	2016-10150	0.00%	N/A	N/A	0.00% [=]	78.88% [+]	0.32% [+]
UAF	2017-11176	27.20%	N/A	N/A	99.56% [+]	99.90% [+]	77.50% [+]
DF	2017-2636	92.96%	27.26% [−]	N/A	98.98% [+]	99.16% [+]	92.36% [+]
DF	2017-6074	86.18%	83.18% [−]	N/A	84.84% [−]	84.64% [+]	87.08% [+]
DF	2017-8890	0.00%	0.00% [=]	N/A	0.00% [=]	43.92% [+]	0.00% [=]
		Total	7-3-2	2-0-0	13-1-2	16-0-0	15-1-1
				(Improvement–Degradation–No change)			

Table 2. Performance of reliability techniques in idle and busy system respectively. N/A indicates that the technique cannot be applied to the corresponding vulnerability. + in the parenthesis means observing improvement over baseline, − is for degradation, and = denotes no change. For each technique, the Total row represent the amount of improvement, degradation, and no change.

extent in idle system. In comparison, in busy system, Defragmentation is the best, increase the success rate of three out of five vulnerabilities to over 90%.

For UAF and double free vulnerabilities, the most effective techniques are Single-Thread Heap Spray and Multi-Process Heap Spray. Taking CVE-2016-0728 as an example, the success rate of baseline exploit is 2.26% in idle system. After applying Single-Thread Heap Spray and Multi-Process Heap Spray, the success rate is raised to 77.46% and 99.98% respectively. Recall that the exploitation of the UAF and double free vulnerabilities require the adversaries to take over the freed slot on heap, with heap spraying implemented, the possibility for successful occupation is increased as the number of spray objects is far larger than other irrelevant objects. Moreover, we can observe that in most cases of our experiment results, Multi-Process Heap Spray outperforms Single-Thread Heap Spray by a huge margin. This is because Multi-Process Heap Spray not only increases the number of spray objects but also consumes enough CPU time that other processes has fewer chance to destroy the desired heap layout in kernel. It strongly suggests security researchers to implement heap spray in a multi-process fashion.

Comparing the success rate of exploits in idle and busy systems, we observe that the workload of the target system does influence the effectiveness of reliability techniques. Taking the technique CPU Pinning as an example, in idle system, the success rates of 6 out of 17 vulnerabilities are more than 90% while in busy system, this number decreases to three.

3.5 Expert-based Intuition Incorrectness

Comparing the results in Table 1 with those in Table 2, we observe that experts have incorrect intuitions towards kernel exploit stabilization techniques. In the following, we describe and discuss the four major types of incorrectness.

Defragmentation for UAF/DF Exploits. For Defragmentation, seven participants hold the view that this technique works for UAF, DF, and OOB while three participants believe that *Defragmentation only works for OOB vulnerabilities*. The experiment results in both Table 2 supports the latter statement. For 12 vulnerabilities the exploits of which are integrated with Defragmentation, five are OOB vulnerabilities and the reliability of them are all improved significantly. In comparison, Defragmentation works for only one out of nine UAF vulnerability (*e.g.*, `CVE-2016-0728`) and none of the DF vulnerability demonstrates improvement. We investigate this inconsistency, and we discovered that Defragmentation should not work for UAF or DF in most of the conditions due to the nature of the technique and the exploits (see Section 3.7.2 for more details). Therefore, we conclude that the expert-based intuition on Defragmentation is incorrect.

Workload. For almost all technique, the participants believe that workload will largely harm the effectiveness (i.e., 1-3, 2-2, 5-2 in Table 1). Intuitively, the increased workload occupies the CPU time and compete with the exploit threads/processes for the usage of kernel heap. As a result, it is difficult to reach the desired heap layout. Our experiment results generally confirms this hypothesis. However, the stabilization techniques are still able to improve success rate to a high value. Even for some techniques, for example, heap spray implemented using multiple processes, the success rate of the exploits are near 100%. Compared with the experiment results obtained in

idle workload, the improvement is less significant for most cases. This result strongly indicates that *increasing workload does have influence in the effectiveness of techniques but is not killers.*

Comparison. Regarding the question which technique is most effective, participants have totally different opinions (i.e., 1-4, 2-3, 2-4, 4-4 in Table 1). In fact, there is *no universal answer* to this question and the most effective technique varies case by case. Taking `CVE-2017-7533` as an example, the most effective techniques for the reliability improvement of this technique is heap spray no matter whether the workload is ligth or heavy. For `CVE-2017-7184`, the optimal choice becomes Heap Grooming.

Recall that in Section 3.3, the participants mentioned two approaches to implementing heap spraying. One is to place the spraying code in the same thread as the vulnerability triggering and another is to fork multiple processes dedicated to allocation. Participants have divergence about which implementation is more effective (i.e., 4-2, 4-3 in Table 1). To resolve this difference, we craft exploit variants in both implementation approaches and draw comparison in Table 2. The result suggests that *Multi-Process Heap Spray outperforms Single-Thread Heap Spray* for almost all cases except `CVE-2010-2959` with light workload.

Others. The empirical result 1-5 is refuted by the experimental results that the reliability of five vulnerabilities are hurt after the Defragmentation is applied. This hints the exploit developers that the outcome is not always as they expect.

The participants agree that the target CPU should be in idle for CPU Pinning to be effective (5-1 in Table 1). This statement is correct but missing one important implementation detail. In our measurement, we observe that except pining exploit threads/processes to the idle CPU, it is necessary to ensure that they are all executed on the same CPU. Recall that in Section 3.1, the allocator has per-CPU featue which

allocates/frees the `slot` from/to the local freeslit. Pining all thread/processes to the same CPU ensures that the manipulation focuses on the local heap.

3.6 Technique Dominating Factors

Based on the experiment, we investigate the limitations of reliability and conclude four dominating factors for exploit unreliability described as follows.

3.6.1 Unknown Heap Layout

After the kernel starts running, the kernel will start using the heap. The heap usage creates randomness to the heap layout. On the one hand, the order of slots on any `freelist` is randomized. On the other hand, many slabs are created, it is difficult to ensure two objects comes from the same slab to ensure they can operate on the same freelist.

OOB exploits aims to allocate a vulnerable object and a victim object next to each other. Without the information of the initial heap layout, it is difficult to create the needed heap layout. Usually, security researchers assumes an initial heap layout and run the exploits. If the initial heap layout is not the same as the assumption, the exploit is likely to fail. This assumption on unknown initial heap layout introduces randomness to exploit reliability.

3.6.2 Unwanted Task Migration

When the workload on CPUs are unbalanced, the kernel will try to perform task migration between CPUs to balance workload. Note that freelist is maintained by each CPU, task migration effectively forces a migrated process to operate on another freelist. This inconsistency on freelists can fail exploits. For example, a UAF exploit process allocates the vulnerable object on CPU-A. After that, it gets migrated to CPU-B. When it frees the vulnerable object, the slot goes back to the freelist associated with CPU-A. But its payload object will be allocated from the freelist associated with CPU-B, leading to unsuccessful exploitation. If the slot is acquired by a process running on CPU-A, a kernel panic happens.

3.6.3 Unexpected Heap Usage

Due to the nature of Linux kernel, the heap is used by many execution units at the same time. When an exploit process is running, other processes on the system or interrupts are also using the heap. The exploit process may be doing critical operations when unexpected heap operations are performed by them.

Specifically, one of the common ways that unexpected heap usage happens is context switch in exploit process. Context switch can happen at any moment in exploit process, when it happens, other processes acquire the CPU and may perform some heap operations. When the exploit process gets rescheduled, the heap operations have already happen and the exploit process is not aware of it.

At this moment, the unexpected heap operations shatters the heap layout assumptions the exploit process has and will lead to exploitation failures.

3.6.4 Unpredictable Corruption Timing

In Linux kernel, many non-essential operations are delayed for the sake of performance. These operations will be scheduled by different components of Linux kernel: softirq, CFS, RCU, and etc. Without access to the internal runtime information of these components, there is no way for exploit processes to predict when an specific operation will be executed. When a vulnerability involves these delayed operations, and more specifically, heap corruption happens inside a delayed operation, there is a time delay between vulnerability triggering and the heap operation takes effect. This poses difficulty on when to perform exploitation after the vulnerability is triggered. The common solution is to sleep for some time to make sure the operation happens. However, this leaves more time for unexpected heap usage to happen and introduces unreliability.

3.7 Kernel Heap Exploit Model

Based on the experimental results as well as the factors observed from the real-world exploit reliability experiments, we generalize the kernel heap exploit process as a model, which we call the *Kernel Heap Exploit Model*. This model explains the process of kernel heap exploit, spanning from the moment that an exploit starts to interact with a vulnerable system, to the moment that the exploit successfully triggers a payload. It also includes all the possible failures occurring in the middle of the process. This model will help to understand the cause of failure of a kernel heap exploit, as well as the reasons why a kernel heap stabilization technique succeeds or fails to improve kernel heap exploit reliability. It can also help to instruct the development of the new

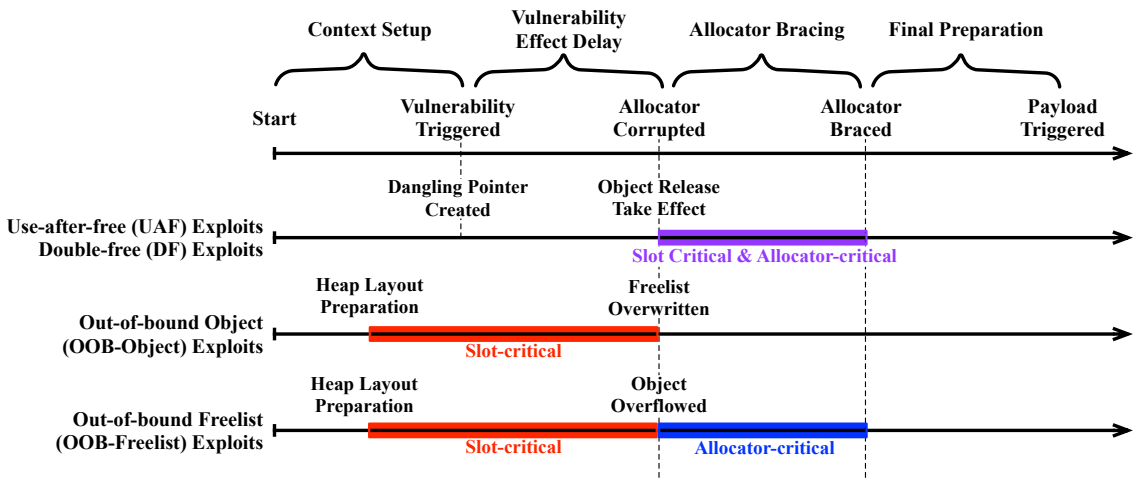

Figure 3. The kernel heap exploit model with different types of exploits and the critical phases.

techniques or technique combinations that mitigate the reliability issues for different types of kernel heap exploits.

The Kernel Heap Exploit Model is depicted in Figure 3. We divide the kernel heap exploit process into 4 stages:

1. **Context Setup.** An exploit prepares necessary context to trigger the vulnerability, *e.g.*, allocating a vulnerable object. Some kernel heap exploit stabilization techniques can be involved in this stage, mitigating the unreliability factors (*e.g.*, unknown heap layout, see § 3.6.1) appropriately and increasing exploit reliability.

2. **Vulnerability Effect Delay.** By the end of context setup, the exploit will trigger the vulnerability. However, such operations may not take effect immediately. For example, for UAF and DF exploits, when the release operation is invoked, the Linux kernel may delay the actual object free operation due to the design of Read-copy update (RCU) [131]. We define this stage as the period from the moment the vulnerability is triggered, to the moment the corresponding

operations take effects. Specifically, at this stage, the vulnerable object will be freed for UAF and DF exploits, the vulnerable object will be overflowed for OOB-Object exploits, and the freelist will be overwritten for OOB-Freelist exploits.

3. **Allocator Bracing.** The memory allocator may transit to a corrupted status after the operations take effect, especially for UAF, DF, and OOB-Freelist exploits. In this case, the exploit must restore the allocator from corrupted state before the kernel turns panic and thus fails the exploit.
4. **Final Preparation.** The exploit may perform some final preparations, such as checking the payload object is at the correct location and modifying the payload object. At this stage, the heap manipulation is finished and the payload can be triggered to obtain control flow hijacking primitive.

3.7.1 Critical Phases for Exploit Reliability

Based on the experimental results, we identify two types of critical phases that could lead to the failure of a kernel heap exploit. For each types of kernel heap exploits, we indicate the critical phases in Figure 3.

Slot-Critical Phase. This phase starts when a target slot is open and ends when the slot is filled by the target object. During this phase, the target slot is under the risk of being occupied by other object from other tasks. This phase is critical to the exploit, because a kernel heap exploit will succeed *only if* the exploit manages to get the target slot.

The slot-critical phase takes place at different stages for different types of exploits. For UAF and DF exploits, slot-critical phase happens after the vulnerability-triggering

operations take effects, because the vulnerable object will be freed at this stage. On the other hand, for OOB-Object exploits and OOB-Freelist exploits, this phase happens before the vulnerability-triggering operations take effects, because the nature of overflow exploits is to allocate vulnerable object and victim object adjacent to each other, which already occurs when the vulnerability-triggering operations take effects.

Allocator-Critical Phase. In this phase, the allocator is in an corrupted status due to the operations of an exploit. This is critical to the success of an exploit because the exploit must smooth the allocator before kernel realizes the issue and turns panic.

Similar to the slot-critical phase, the allocator-critical phase also occurs at different stages for different types of exploits. For UAF and DF exploits, it happens at the same stage of slot-critical phase, because releasing a vulnerable object will cause the memory allocator corrupted. For OOB-Freelist, the allocator-critical phase happens after the slot-critical phase, because overwriting the freelist will cause the allocator corruption issue. For OOB-Object, allocator-critical phase does not exist, because overflowing object to the adjacent object does not trigger the memory allocator.

3.7.2 Exploit Stabilization Success and Failure

Based on the kernel heap exploit model and the reliability-critical phases, we revisit the experimental results for the kernel heap exploit stabilization techniques and investigate the reasons why each technique succeeds or fails to improve exploit reliability. We present for each exploit stabilization technique as follows:

3.7.2.0.1 Defragmentation.

This technique is usually used in the stage of context setup, at which the number of half-full slabs and the kernel heap layout are indeterministic and unknown to the exploit. It helps to achieve a more deterministic heap layout by draining the half-full slabs and current freelist so that the kernel will create a fresh linear freelist whose layout is easier to predict. As mentioned in Section 3.6, heap layout is a factor impacting the stability of OOB exploits. Therefore, Defragmentation works for OOB exploits. Also, it does not help for UAF and DF exploits because UAF and DF exploits do not necessarily need specific kernel heap layout.

3.7.2.0.2 Heap Grooming.

Similar to Defragmentation, Heap Grooming is also able to mitigate the unreliability issue caused by unknown heap layout. This technique improves the reliability of OOB exploits.

3.7.2.0.3 Single-Thread Heap Spray.

This technique improves exploit reliability by mitigating unknown heap layout (§ 3.6.1), unexpected heap usage (§ 3.6.3), and unpredictable corruption timing (§ 3.6.4). It handles unknown heap layout by spraying a large number of objects, draining half-full slabs and increase the chance that a payload object will land in the correct slot in the correct freelist. It also helps remediate unexpected heap usage, since it can fill up unexpected free slots and still land payload objects at the target slot.

Besides, its constantly allocating payload objects makes it more likely to land a payload object right after the critical transaction starts. For exploiting UAF or DF exploits, Single-Thread Heap Spray is resistant to unpredictable corruption timing. Given the above reasons, Single-Thread Heap Spray is able to improve exploit reliability significantly in most cases.

3.7.2.0.4 Multi-Process Heap Spray.

Single-Thread Heap Spray is effective only when context switch does not happen, because context switch will stop the spray. If context switch happens during a critical phase, then the critical phase will keep open until the spray process is switched back. Multi-Process Heap Spray resolves the context switch issue by spraying in multiple processes. In this case, even though context switch happens, it is likely that the switching task is still a spray process, so that the spray will continue.

Furthermore, Multi-Process Heap Spray helps for a great proportion of exploits which requires no heap spray in the exploit process. As implied by task migration (§ 3.6.2), processes will not affect freelists and vulnerable objects across CPUs. For these exploits, Multi-Process Heap Spray still improves the reliability because the exploit process is likely to have a heap spray process in the same CPU when there are a great number of heap spray processes existing in the system.

However, Multi-Process Heap Spray works with a cost. Launching many processes shortens the time slice each process can have, which increases the chance of occurrence for context switch. As a result, Multi-Process Heap Spray is a double-edged sword for exploit reliability: When the baseline exploit is unreliable, this technique is able to

boost the reliability by a huge margin. On the other hand, it may also introduce a slight reliability reduction to the baseline exploits that are already very reliable.

3.7.2.0.5 CPU Pinning.

A heap spray process may be scheduled to a different CPU from the exploit process due to task migration (§ 3.6.2). In this case, the expected overwrite will never happen, rendering the exploit attempt a failure. To mitigate the exploit failure cased by task mitigation, CPU Pinning will pin processes to specific CPUs, forcing the tasks to operate on the same per-CPU freelist.

3.8 New Technique Discovery

In this section, we present two new techniques discovered by tempering with the technique dominating factors and inspired by the kernel heap exploit model. The first application is Context Conservation, a new technique that improves exploit reliability for UAF and DF exploits by reducing the impact of Unexpected Heap Usage. The second application is stabilization technique composition, which are sets of exploit stabilization techniques selected for different types of exploits. We will introduce both applications and their evaluation results.

3.8.1 Context Conservation

By analyzing the kernel heap attack model we design, we observe that context switch impacts both slot-critical and allocator-critical phases. Once context switch

happens in the exploit process within the critical transaction, the transaction will keep open until the process gets rescheduled and finish the exploitation. This introduces long-time sleep into critical phases, giving the competitors more chances to occupy the target slot or crash the allocator (Unexpected Heap Usage).

Guided by the insight derived from the model, we designed a novel stabilization technique to improve kernel exploit reliability. This technique aims at ruling out the impact of context switch thus the impact of Unexpected Heap Usage, by avoiding context switch during critical phases. Firstly, it suggests security researchers to remove unnecessary code from the critical transaction. It is common to see code for context setup, debugging and reporting code within the critical transaction. As guided by the model, it effectively enlarges the critical transaction. These code should be removed or moved outside of the critical transaction to improve exploit reliability. Secondly, it injects a stub into the process to ensure the critical transaction starts in a fresh time slice. This prevents context switch from happening during the critical transaction.

The stub we inject runs in a loop that times each iteration in terms of CPU's time stamp counter. Without context switch happening within the loop, each iteration takes little cycles to finish. However, when context switch happens in the loop, that specific iteration will observe a large elapsed cycles. This is because context switch makes the CPU switch to execute other processes. This stub gives the exploit process the ability to detect context switch with minimal latency, ensuring the critical transaction starts in a fresh time slice. Hence, it ensures no context switch within the critical transaction.

On the other hand, we also realize that Context Conservation will be unlikely to work effectively if it is solely used for OOB exploits. In particular, without heap layout preparation, there is no guarantee that the next slot the allocator is going to

give will land at the correct slot after the vulnerable object is allocated. Avoiding context switch between these two allocation does not increase the chance that they will be adjacent to each other. Therefore, we conclude that *Context Conservation will only work for UAF and DF exploits.*

To test our hypothesis, we evaluate Context Conservation in the same way as public techniques and the result confirms its effectiveness as shown in Table 3. As shown in the result, it improves reliability for 8 out 10 UAF and DF exploit scenarios. The improvement is more significant when the system is busy since context switch happens more often in busy systems. Also, results indicate that Context Conservation does not improve reliability for OOB exploits. Instead, it hurts the reliability in many cases. These observations are aligned with our analysis based on the kernel heap exploit model.

Type	**CVE**	**Idle**		**Busy**	
		Baseline	**C.S.**	**Baseline**	**C.S.**
OOB	2017-7184	31.46%	28.20% [−]	40.90%	37.34% [−]
OOB	2016-6187	34.28%	26.54% [−]	39.62%	35.78% [−]
OOB	2010-2959	21.44%	19.36% [−]	7.18%	19.40% [+]
OOB	2017-7308	0.02%	0.04% [+]	0.00%	0.00% [=]
UAF	2018-6555	0.12%	0.02% [−]	0.02%	0.42% [+]
UAF	2017-8824	96.86%	99.40% [+]	87.66%	98.64% [+]
UAF	2017-11176	98.06%	99.98% [+]	27.20%	57.58% [+]
DF	2017-2636	96.34%	96.30% [−]	92.96%	96.00% [+]
DF	2017-6074	88.50%	96.70% [+]	86.18%	95.64% [+]

Table 3. Performance of Context Conservation (i.e., C.S.) for improving exploit reliability in idle and busy system respectively. The meaning of symbols in the brackets are the same as Table 2. As our hypothesis suggested, Context Conservation should *not* work effectively for OOB exploits, and it should work effectively for UAF and DF exploits.

3.8.2 Compositing Stabilization Techniques

Based on the kernel heap exploit model, we identify techniques that could potentially co-work and improve exploit reliability. We recommend security practitioners follows the instruction described below.

First, we recommend to always target a `cache` that is suitable and the least busy in the system. Since Multi-Process Heap Spray outperforms Single-Thread Heap Spray consistently in our evaluation and improve reliability by a huge margin generally, it is recommended to deploy this technique for reliability improvement. We also always recommend CPU Pinning since it improves reliablity greatly in many cases and in the worst case, it reduces reliability only slightly. We recommend applying Defragmentation to OOB exploits, when it is not applicable, Heap Grooming then can be applied. It is recommended to apply Context Conservation to UAF and DF vulnerabilities whenever it is applicable.

To evaluate the effectiveness of our recommendations, we combine the techniques as instructed above and deploy them to the baseline exploits. As an outcome, we generate a set of new exploits, which we label them with "Combo". The evaluation result is shown in Table 4. The result shows that our recommended composition manage to improve the exploit reliability for 33 out of 34 (97%) exploit scenarios. While the average success rate of baseline exploits is 35.17%, our composition achieves 95.37%, which improves by 171.12%. The increase of reliability ranges 2% (from 98.06% to 99.98%) to 67100% (from 0.12% to 80.58%), depending on the value associated with the baseline exploit. Moreover, the proposed composition improves the reliability of 4 exploit scenarios from 0.00% (rounded by 2) to 65%, 75.84%, 82.62% and 86.20%.

Type	CVE	Idle		Busy	
		Baseline	Combo	Baseline	Combo
OOB	2017-7533	22.32%	95.28% [+]	7.48%	90.36% [+]
OOB	2017-7184	31.46%	98.90% [+]	40.90%	98.80% [+]
OOB	2016-6187	34.28%	99.56% [+]	39.62%	99.66% [+]
OOB	2010-2959	21.44%	99.64% [+]	7.18%	97.44% [+]
OOB	2017-7308	0.02%	75.14% [+]	0.00%	67.28% [+]
UAF	2018-6555	0.12%	80.58% [+]	0.02%	72.06% [+]
UAF	2016-8655	96.78%	99.92% [+]	16.02%	98.30% [+]
UAF	2017-15649	13.00%	99.06% [+]	14.30%	97.86% [+]
UAF	2016-4557	99.86%	100.00% [+]	36.04%	99.94% [+]
UAF	2017-8824	96.86%	99.64% [+]	87.66%	99.74% [+]
UAF	2016-0728	2.26%	100.00% [+]	0.10%	99.96% [+]
UAF	2017-10661	37.56%	99.38% [+]	1.18%	89.28% [+]
UAF	2016-10150	0.00%	75.84% [+]	0.00%	80.72% [+]
UAF	2017-11176	98.06%	99.98% [+]	27.20%	99.82% [+]
DF	2017-2636	96.34%	99.00% [+]	92.96%	98.58% [+]
DF	2017-6074	88.50%	85.56% [−]	86.18%	87.58% [+]
DF	2017-8890	0.00%	86.20% [+]	0.00%	82.62% [+]

Table 4. Performance of our proposed technique composition for improving exploit reliability in idle and busy systems. The meaning of symbols in the brackets are the same as Table 2.

Based on the result, we see that our recommended technique composition is able to improve the stability of real-world exploits significantly.

3.9 Discussion

Experiment repeated times. In the current experiment, for each tested exploit (with or without stabilization), we run it for 500 times, calculate the success rate, and we repeat the test for 10 times and report the average success rate over the 10 runs. We set the times 500 for each run under the consideration that 500 is a relative large number comparing to previous work (*e.g.*, SLAKE [17] tested each exploit for 100 times), and that we need balance between preciseness and cost. During

the experiment, we observed that some experiments do not need 500 times and some others may require more times in order to decide for statistical significance. One future work could be to understand the correlation between the experiment repeated times and the exploit type and tested stabilization techniques. This will help to instruct the evaluation on future kernel exploit reliability research.

Kernel heap exploit reliability in other operating systems. In this paper, we focus on Linux operating systems, aiming to understand the kernel heap exploit reliability issues rooted in Linux kernels. Kernel heap exploit reliability also exists in other operating systems such as Windows. For example, the exploit developers propose to use heap data repairing and memory alignment to impprove the reliability of exploit against CVE-2015-0057 in Windows [127]. We will study this research problem for other operating systems in the future work.

3.10 Limitation

Although this works formulates the kernel heap exploit model which can be used to derive a new stabilization techniques and a technique combination, it comes with limitations.

Unknown dominating factors We identified four dominating factors for exploit reliability in this work which forms the foundation of the kernel heap exploit model. However, these factors are obtained by analyzing 17 existing exploits. Although it is the largest dataset at the time of the experimentation, it is still a small dataset. There are potentially more factors that impact exploit reliability that are unknown to us. But we cannot find them due to the limitation of exploit generalization.

Non-optimal combination of exploit stabilization techniques. As pointed

out in Section 3.8.2, using multiple exploit stabilization techniques help to improve exploit reliability. And our experiment shows that applying all applicable techniques (our combination) indeed significantly improves exploit reliability. However, this combination may not be the *best* combination. It is possible that exploit reliability saturates when using a list of techniques, adding other heavy techniques (*e.g.*, Multi-Process Heap Spray) may negatively impact exploit reliability. But neither our model or exploit generalization offers advice on how to derive the best technique combination. Given the complexity of this problem, we leave the study of the optimal technique combination as separate research for future work and a limiation of this study.

3.11 Related Work

In this work, we study the reliability of kernel exploit with the goal of facilitating exploitability assessment and providing guidance for exploit generation. In the following, we discuss the exiting works in these two directions and clarify their difference from ours.

Kernel Exploitabilty Assessment. To facilitate the exploitability assessment in kernel, there are a rich collection of research works along the exploitation chain.

The first research area is to explore the capability of vulnerabilities. FUZE [134] searches for new use sites of use-after-free vulnerabilities using under-context fuzzing and further employs symbolic execution to identify exploitable primitives implied by the new use sites. KOOBE [15] retrieves the corruption capabilities of a slab-out-of-bound access vulnerability manifested in the PoC programs and disclose hidden capabilities using capability-guided fuzzing. Yonchan et al. [73] increases the success

rate of triggering race condition vulnerabilities by invoking inter-process interrupts to enlarge the time window.

The second research area is to obtain exploitable primitives. For use-after-free vulnerabilities, Xu et al. [135] uses two memory collision mechanisms to occupy the freed memory region in the kernel. Lu et al [82] exploits use-before initialization vulnerabilities using deterministic stack spraying and reliable exhaustive memory spraying. To take one step forward, Cho et al. [18] further proposes to make use of BPF functionality in the kernel for stack spraying. SLAKE [17] assists the exploitation slab-based vulnerabilities by first building a database of kernel objects and then systematically manipulating the layout of kernel heap by using the kernel objects in the database.

The third research area is to bypass mitigations in the kernel. ret2dir [64] takes advantage of physical memory mapped to kernel space for payloay injection. KEPLER [133] uses communication channels between kernel space and user space to leak stack canary and inject payload to kernel stack for ROP programming. ELOISE [16] leverages a special but prevailing type of structure to bypass KASLR and heap cookie protector. Relying on hardware side channels, [51, 57, 60, 79] circumevent KASLR without triggering SMEP/SMAP protection.

Our work is the first to study the reliability problems in the kernel exploit. In addition to the measurement of existing techniques, we summarize the factors that influence reliability and build a theory model to boost the exploration of more novel techniques that can improve reliability and facilitate kernel exploitability assessment in the future.

Exploit Generation Techniques. In the past decade, researchers have been exploring the approach to automatically generating exploits.

Brumley et al. [4, 10] proposed preconditioned symbolic execution to develop an automated approach that generates exploits for stack overflow and format string vulnerabilities. Bao et al. [8] developed shellcode layout remediation and path kneading approaches that automatically transplant existing shellcode to the target vulnerability. The Shellphish team designed PovFuzzer and Rex to turn a crash to an exploit [108, 109, 113]. For PovFuzzer, it keeps mutating the input and at the same time observing how the input influences the crash. Following this work, Rex is proposed to symbolically execute the input with the purpose of jumping to a shellcode prepared in advance. Heelan et al. focus on heap buffer overflow vulnerabilities in user space programs. In [53], they use regression test to learn how to automate heap layout manipulation so that one could corrupt the sensitive pointers. They further improve their proposed approach by using a genetic algorithm to replace the random search algorithm in [54] for exploiting heap overflow vulnerabilities in language interpreters. With the similar goal, Revery [126] also explores exploitable memory layouts for vulnerabilities in userspace programs. It utilizes fuzz testing along with a program synthesis method to guide the construction of a working exploit. Insu et al. [138] discover new exploitation primitives in the heap allocator. They provide heap operations and attack capabilities as actions, driving the heap allocator to execute until primitives such as arbitrary write or overlapped chunks are identified.

Unlike above works that focus on userland program, our work targets at kernel which is much more complicated. We believe the findings of our work can also provide guidance for the similar research in userland. Besides, our work is not manipulating exploitable memory layout or synthesizing exploits. Rather, we focus on the reliability of exploit which has never been studied before.

3.12 Conclusion

In this chapter, we apply exploit generation to the kernel heap exploits. We first systematically study the kernel heap exploit reliability problem. We collected kernel heap exploit stabilization techniques commonly used by security experts as well as their intuitions on the use of these techniques, and we test these techniques on 17 real-world exploits and 34 different exploit scenarios. The results indicate that many of the expert intuitions are incorrect. To understand the nature of kernel heap exploit reliability, we investigate the real-world evaluation results, based on which we identified four technique dominating factors. We then devise a generic kernel heap exploit model that explains the reliability issue. Inspired by the model, we create a novel technique that improves the reliability of use-after-free and double-free exploits by mitigating Unexpected Heap Usage dominating factor, as well as a technique composition that boosts 97% exploit scenarios significantly, from 2% to 67100%.

Chapter 4

KERNEL CONTROL-FLOW HIJACKING ATTACK

Among Linux kernel vulnerabilities, those giving attackers the ability to carry out Control Flow Hijack (CFH) is a prevalent and serious type of flaws. Attackers exploit such vulnerabilities to achieve *control flow hijacking primitives* (CFHP) (*e.g.*, by overwriting a function pointer stored in the kernel heap), then use creative techniques to bypass modern defenses (*e.g.*, SMAP) before finally escalating the attacker's privilege level from normal users to root. CFH is popularly used for Linux kernel exploitation *despite modern defenses*. For example, 15 out of 16 public real-world Linux kernel exploits [40] reported to 2022 Google's kCTF kernel bug bounty program are control-flow hijacking attacks.

However, there is a significant gap from CFHP to a full kernel privilege escalation exploit. In most modern cases, CFHP stems from flaws on the heap (*e.g.*, a function pointer overwrite stemming from a use-after-free), instead of the stack. This is problematic for exploitation, as building and triggering an ROP payload on the heap is hard. Current solutions to this problem attempt to pivot the stack pointer to a fake stack on the heap [34, 61, 86] or in the flat physical mapping [133]. Such techniques are brittle because they rely on either specific kernel memory layouts, the existence of precise bespoke gadgets, or additional primitives (such as register control) that are not always granted alongside the CFHP. Unfortunately, for attackers, the decline of straightforward stack overflow vulnerabilities (and the adoption of effective mitigations, such as stack canaries), leaves no other option than to accept this complexity, brittleness, and additional primitive requirements.

In this work, we systematically analyzed all existing CFHP exploitation techniques and noticed that user data accessibility is the dominating factor. By clearly listing all regions that can be controlled by users and the kernel can access during runtime, we noticed that kernel stack is an understudied attack surface. By analyzing the impact of placing user data on the kernel stack during the exploitation of CFH vulnerabilities, we discovered a new exploitation technique. We call it Retspill.

Retspill exploits the *intended design* of the kernel stack: Linux's kernel stack is designed to save various forms of data, *including user-controlled data from user space*. Attackers can leverage this intended storage to store carefully-crafted malicious data in kernel space, that they can access after triggering vulnerabilities through stack-pivoting.

In studying the implications of these capabilities, we realized that the Linux kernel stack design can be utilized as an attack surface for malicious attackers. We built on our discovery to propose a new mitigation to defend against it. Our technique, RetSpill, enables exploits that bypass most existing kernel security mitigations (and emerging mitigations such as Function Granular Kernel Address Space Layout Randomization [68]) with no other requirements (such as register control or specific memory configuration requirements that limit other techniques). Importantly, RetSpill can be semi-automated, demonstrating that the bar for successful Linux kernel attacks is significantly lower than where it is currently assumed to be by the security community.

To comprehensively understand RetSpill's potential impact, we identify the scenarios where user-controlled data appears on the kernel stack (§4.4.2), and we quantify the size of user-controlled data to check if they are sufficient for a privilege escalation exploit. We discover four different scenarios (one of them is discovered by another researcher in parallel to this research [99]), three of which are Linux kernel's intended

operations which are inevitable by design. We also find that, Linux system calls typically leave enough user-controlled data for attackers to launch a privilege escalation exploit. Specifically, our measurement shows that, given Linux kernel's intended operations, the user-controlled data in a Linux system call can hold 11 ROP gadgets on average. Even worse, we found that given Linux kernel's current design, 11 ROP gadgets are sufficient for privilege escalation (§4.4.3). Through clever downstream applications, one can take advantage of the 11 gadgets to obtain unlimited Arbitrary Read/Write/Exec primitives in kernel space and can even call and retrieve return values from kernel functions.

We show the power of RetSpill by applying it to 22 real-world vulnerabilities, demonstrating its ability to achieve successful exploitation on 21 of them, with fewer primitive requirements and more reliability than existing public exploits for these vulnerabilities. Critically, because the technique is so applicable, and very few mitigations are effective against it to any extent, we were able to implement RetSpill as a semi-automated pipeline that directly synthesized 20 of these exploits. The only manual effort needed is providing the CFHP and integrating the output of the pipeline into the exploit.

Finally, we investigate RetSpill from a defensive perspective, checking it against all existing commonly-adopted protections (which cannot defend against it) and proposing new protection to mitigate the impact of this dangerous exploitation technique and re-raise the bar of Linux kernel exploitation.

In summary, this work applies the exploit generalization procedure and made the following contributions:

- We identify user data accessibility as the dominating factor for control flow

hijacking exploitation techniques and identify the kernel stack as an attack surface.

- By accessing user-controlled data on the kernel stack, we devise a novel and powerful exploitation technique, RetSpill, that can bypass commonly-adopted protections in the Linux kernel and enable powerful exploitation of Control Flow Hijacking vulnerabilities.
- We evaluate RetSpill on 22 real-world vulnerabilities and demonstrate its ability to break the security boundary between user space and kernel space.
- We propose a new protection to harden Linux kernel security. Combined with existing protections, the new defense can mitigate the impact of the powerful RetSpill exploitation technique.

To foster future research, we released the artifacts, including RetSpill and all the experiment data[2]. This will benefit the community by letting researchers understand the severity of RetSpill and inspiring new protections to harden the Linux kernel. We have contacted the Linux kernel security team and responsibly disclosed the exploitation technique.

4.1 Background

Kernel ROP. Kernel ROP, similar to that in user space, grants attackers the ability to perform arbitrary execution by chaining ROP gadgets together. Compared to user space ROP, kernel ROP has two additional requirements. First, leaked information in the ROP chain needs to be transferred back to user space. This can be done by using

[2]https://github.com/sefcom/RetSpill

the fact that registers are not cleared during CPU privilege transition [72]. Attackers can load the leaked information into a register, perform a privilege transition back to user space, and obtain the leaked value. Second, ROP chains in kernel space need to end gracefully to avoid panic. Currently, there are three existing ways to achieve this: 1. return back to user space, such as using KPTI trampoline [72]; 2. sleep indefinitely [139], which freezes the current task; 3. kill the current task by invoking `do_task_dead` function.

Kernel Stack and System Calls. In the Linux kernel, each task has its own kernel space stack to facilitate the execution of system calls invoked from user space. The kernel stack is allocated as a fixed-size buffer at the creation of each kernel task [116]. The kernel stack layout is shown in Figure 4. When the user space invokes a system call, the program counter will switch to kernel space functions and the stack pointer will be set to the kernel stack. Then, the kernel will push the user space context, including all user space registers, onto the bottom of the stack, and invoke the corresponding system call handler. The saved user space context is named `pt_regs`. When returning back to user space, the kernel will restore the user space context and resume the execution of the user space program.

Due to the fixed-sized nature of the kernel stack, developers usually store large chunks of data in dynamically allocated memory regions (*e.g.*, heap) to avoid stack exhaustion. However, for performance considerations, some performance-critical system calls [120, 122] still store large amounts of data on the kernel stack.

Table 5. With the development of modern protections in the Linux kernel, exploiting CFH vulnerabilities requires many primitives and becomes brittle. Our work, RetSpill, is a powerful exploitation technique that requires fewer primitives and yet is still reliable.

Technique	**Required Primitives**	**Data Region**	**Reliable***	**Payload Rewrite**	**Still Work**
ret2usr [71]	CFHP	user space	✓	✓	✗
ret2dir [64]	CFHP+PL	physmap	✗‡	✓	✗
pivot to user [32]	CFHP+CL	user space	✓	✓	✗
change CR4 [93]	CFHP+CL+rdi	user space	✗§	✓	✗
run_cmd [49]	CFHP+CL+HL+rdi	kernel heap	✓	✓	✗△
pivot to heap [34]	CFHP+CL+HL+RC	kernel heap	✓	✗	✓
KEPLER [133]	CFHP+CL+PL+RC	physmap	✗	✓	✓
set_memory_x [27]	CFHP+CL+HL+rdi&rsi	kernel heap	✗§	✗	✓
RetSpill	CFHP+CL	kernel stack	✓	✓	✓

PL: physmap leak, CL: codebase leak, HL: heap info leak, RC: register control

* Given all the required primitives, whether the technique can derive the target primitive without failures.

‡ ret2dir is reliable only if given a page frame info leak primitive.

§ need to obtain CFHP twice.

△ run_cmd function can get inlined, thus not exist, in some kernel builds.

4.2 Study of Existing CFH Attacks

In this section, we present a brief study of existing CFH attacks including how they work and their applicability in modern Linux, and how we derived user data accessibility as the dominating factors.

Existing Protections Against CFH Attacks. With increasingly powerful defenses developed by the Linux kernel, exploiting CFH vulnerabilities has become difficult. In a modern Linux kernel, obtaining CFHP does *not* imply the ability to obtain root or even carry out Return-Oriented-Programming (ROP) or arbitrary code execution because of modern kernel hardening techniques.

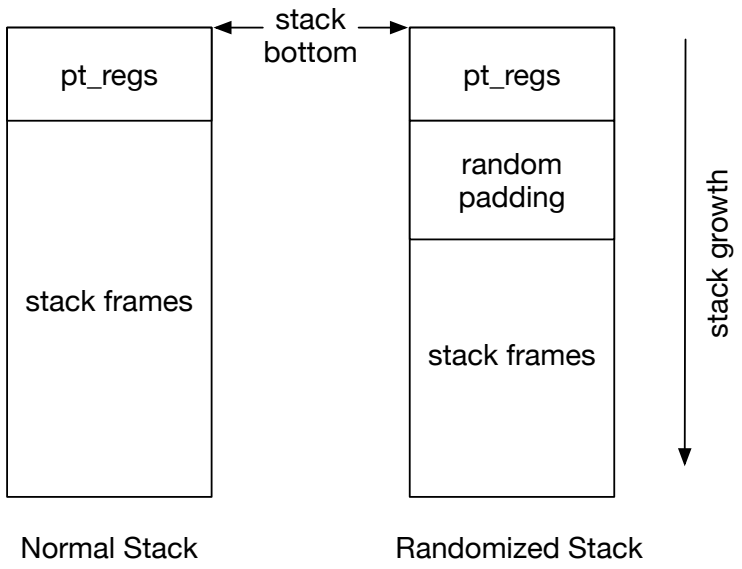

Figure 4. Kernel Stack Layout With and Without RANDKSTACK Protection

Most protections aim to prevent attackers from accessing user space data improperly. `SMEP` [89] and `KPTI` [119] prohibit executing user space code from kernel space. `SMAP` [22] disallows the kernel from reading/writing user space data directly. `NX-physmap` [64] marks the `physmap` region as not executable. These protections limit attackers' capability to turn CFHP into arbitrary code execution.

Other protections were designed to limit attackers' capabilities despite limited code execution: `CR Pinning` [118] prevents unauthorized modification to the CR4 register (which contains bits to enable certain mitigations), thus preventing attackers from disabling mitigations. `STATIC_USERMODE_HELPER` [69] prohibit abusing "user mode helpers" to perform privilege escalation. `RKP` [102] disallows directly modifying process credentials. `pt-rand` [26] invalidates the technique of directly modifying kernel page tables.

Notably, `RANDKSTACK` [97] is a protection that aims to randomize kernel stack layout, preventing attackers from exploiting use-of-uninitialized-variable vulnerabilities. Its

change to kernel stack layout is shown in Figure 4. Similar to their use in user space applications, `STACK CANARY` mitigates stack-overflow vulnerabilities, removing the possibility of overwriting the return address (and higher memory addresses) to achieve ROP.

Bypassing exploitation protections. Because of strong stack protections, most modern CFHPs happen as a result of kernel heap misuse. Typically, an attacker overwrites (*e.g.*, via heap overflow) a function pointer in a victim object in the heap. The attacker then invokes a *trigger system call* that calls the overwritten function pointer and obtains CFHP. As direct shellcode execution is no longer viable (due to NX-physmap and SMEP), and stack control is elusive (due to the use of stack canaries and death of stack overflow vulnerabilities), a typical CFH exploit needs to combine the CFHP with other primitives, such as a heap address disclosure, to obtain stack control and start an ROP chain [24, 34, 61, 86] as shown in Table 5.

Imperfect bypass: "pivot to heap". Anecdotally, we observed that most modern kernel exploits use heap-related vulnerabilities and must prepare and trigger attack payloads on the heap. For example, all 15 public kCTF CFHP exploits pivot to a fake stack in the heap to achieve ROP. This has drawbacks: it does not support rewriting payloads directly because the payload is in the kernel heap, a region users do not have direct access. In other words, to execute a different payload, the attacker needs to swap the heap object containing the payload or even trigger the vulnerability again, which significantly reduces reliability for exploits that need to trigger different payloads to achieve privilege escalation [36, 67]. Additionally, these exploits rely on either specific kernel memory layouts, the existence of precise bespoke gadgets, or additional primitives (such as register control). As shown in Table 5, "pivot to heap"

requires more primitives than RetSpill, which significantly increases the difficulty of developing exploits.

User Data Accessibility When we were studying the existing CFH techniques, we wrote down how each technique work and noticed that all of them involve accessing data in regions that attackers control. This is intuitive because the initial PC-Control only provides 8-byte control, which is not enough to carry out any computation attackers desire. As a result, accessing other user data using the initial 8-byte control is necessary. This observation leads us to conclude user data accessibility is the dominating factor for kernel CFH attacks. As shown in Table 5, existing exploits have tried placing their controlled data in the user space/physmap/kernel heap, but one important region, the kernel stack, has been under the radar. By analyzing the impact of placing user data on the kernel stack when exploiting CFH vulnerabilities, we found out that it can lead to the bypass of all commonly deployed modern mitigations, leading to the discovery of the Retspill exploitation technique.

4.3 Threat Model

In our threat model, the target Linux kernel has all the protections as the model in Kepler [133], in addition to Function-Granular Kernel Address Space Layout Randomization (`FG-KASLR`) [68], which is a cutting-edge kernel hardening technique implemented recently.

Specifically, we allow the target Linux kernel enabling `SMEP`, `SMAP`, `KPTI`, `NX-physmap`, `CR Pinning`, `STATIC_USERMODE_HELPER`, `RKP`, `pt-rand`, `RANDKSTACK`, `STACK CANARY`, and `FG-KASLR`.

In terms of exploitation primitives, our model requires two primitives: control-flow

hijacking and kernel image base address leakage. Both primitives are reasonable and can be achieved through a reasonable number of Linux vulnerabilities (Table 8). Our required primitives are fewer than all existing work (Table 5), as well as many latest real-world kernel exploitation techniques such as those published through Google kCTF [40].

4.4 Exploitation Technique Design

A modern CFH exploit has two steps: 1) injecting user-controlled data into kernel memory, and 2) using this controlled data for ROP. Modern protections prevent attackers from directly accessing user space data, leaving physmap and kernel heap the only known choices for storing user-controlled ROP chains. The difficulty of accessing these two regions induces the requirement of additional exploitation primitives (such as register control or additional address disclosures) onto modern CFH exploits.

Our proposed technique, RetSpill, is based on the insight that potentially attacker-controlled user data is loaded, *by design*, onto the kernel stack during system calls. When PC-control is obtained, which happens inside the triggering system call after the attacker sets up the conditions for the Control Flow Hijacking Primitive (CFHP), the attacker can use this controlled user data readily on the kernel stack to launch code reuse attacks (*e.g.*, ROP).

However, this is not enough for full exploits: the individual resulting ROP attacks are limited, and our study of 22 kernel vulnerabilities (§4.7.1) shows that some triggering system calls have as few as seven controllable gadgets for ROP payload. This leads to RetSpill's other contribution: because stack layouts of functions are

fixed during compilation, attackers can repeatedly invoke the triggering system call unlimited times to trigger different ROP payloads.

With this, RetSpill can obtain *unlimited* arbitrary read/write/exec in kernel space, thus completely breaking the security boundary between user and kernel space despite the presence of modern kernel protections. This is more severe than existing "single-shot" exploitation techniques [133]: as shown in §4.7.3, RetSpill can bypass more protections than existing techniques.

4.4.1 Data Spillage

Attacker-controlled user space data can spill onto the kernel stack either directly or indirectly.

Direct Data Spillage refers to the situation where user data is loaded directly onto the kernel stack in one system call. The `poll` system call shown in Listing 1 performs direct data spillage when it uses `copy_from_user` to copy user space data onto the kernel stack directly.

Indirect Data Spillage happens when data is loaded onto the kernel stack through multiple system calls. For example, with carefully crafted arguments, a call to the `open` system call stores data from user space to the kernel heap, and a subsequent call to `readlink` loads this controlled data to the kernel stack.

In this work, we mainly focus on systematically analyzing direct data spillage. We first investigate how user space data can flow into kernel space during system calls. We identify two methods: 1) user space registers and 2) user space memory. More specifically, when a user space program invokes a system call, its general-purpose registers contain data that can potentially flow into kernel space. During the lifetime

of a system call, the kernel may need additional information from user space memory to complete the system call, which leads to userspace data flowing into kernel space.

To comprehensively discover direct user data spillage causes, we perform taint analysis on user space data (both user space registers and memory) and observe the traces on kernel stack in triggering system calls to identify data spillage causes as detailed in §4.6.2. By manually analyzing the taint analysis results, we identify three causes of direct user data spillage.

Depending on where user data is temporarily stored, there are potentially many variants of indirect data spillage methods. In this work, we empirically identify one general case of indirect data spillage where user data is temporarily stored on the kernel stack and shared across system calls because of uninitialized memory. We leave the systematic study of indirect data spillage as future work.

4.4.2 Data Spillage Sources

We identified four data spillage sources as follows:

Valid Data. In a Linux kernel, user space data is sometimes directly copied onto the kernel stack for performance reasons. Take the simplified `poll` system call handler in Listing 1 as an example. It copies a considerable amount (0x1e0 bytes in our build) of user space data into `stack_pps`, which is on the kernel stack. If an attacker controls a `file` object in user space, when CFHP is obtained through the `file->f_op->poll` call, they will have control of 0x1e0 bytes on the kernel stack. The attacker can then use an `add rsp, X; ret` gadget to pivot into the controlled region and launch code reuse attacks. During the lifetime of the `poll` system call, the on-stack data is valid

```
1 int do_sys_poll(...)
2 {
3     long stack_pps[POLL_STACK_ALLOC/sizeof(long)];
4     struct poll_list *walk = (struct poll_list *)stack_pps;
5
6     // copy user space data onto kernel stack
7     copy_from_user(walk->entries, ufds, sizeof(walk->entries);
8
9     // invoke poll handler for each fd
10    pfd = walk->entries;
11    for(; ... ; pfd++) {
12        file = fdget(pfd).file;
13        file->f_op->poll(...);
14    }
15    ...
16 }
```

Listing 1. The simplified code snippet of poll system call handler in the Linux kernel.

(*i.e.*, its presence on the kernel stack is *not* a security violation), but it is used for malicious purposes.

Preserved Registers. Each user space thread has its own kernel stack. When the user space thread invokes a system call, the kernel will switch to using the associated kernel stack by setting the `rsp` register. Immediately following the stack pointer change, the kernel pushes the user space context onto the kernel stack to preserve the context as shown in Figure 4. Here, the "user space context" is a data structure called `pt_regs` [121] that includes all of the user space registers. These values can be carefully set by malicious users before invoking the system call.

In other words, a fully user-controllable region is at the bottom of the kernel stack. When the attacker triggers a CFHP, they can use the controlled `pt_regs` region as a ROP payload.

Calling Convention. In the Linux kernel's calling convention [117], callee and caller functions both need to preserve sets of registers, termed "callee-saved" and "caller-saved" registers. The way compilers implement the calling convention is by

emitting code to push the registers that will be potentially clobbered in the function to the kernel stack at the function prologue and pop them at the epilogue. This mechanism gives attackers two ways to place controlled data onto the stack.

First, registers can be pushed onto the kernel stack when the kernel invokes the actual system call handler function to serve the system call. System call handlers are invoked right after userspace registers are pushed onto the kernel stack as `pt_regs`. At the time of invoking system call handlers, the registers *still* contain userspace data. Since system call handlers are calling convention compliant, they will save registers that will be clobbered within the handler by pushing them onto the stack as well.

Second, the system call handlers might call other helper functions, which will also be calling convention compliant, and registers still containing user space data (or reloaded with user space data in the course of system call servicing) might be saved as a result.

Uninitialized Memory. Since there is only one kernel stack for each thread, data used in previous system calls within the same thread can be left on the kernel stack if not overwritten by the triggering system call. Due to the fact that the kernel's control flow is usually hijacked in the middle of a function, variables used *after* the hijacking point will not be initialized by design. In other words, hijacking control flow can create unexpected uninitialized memory situations on the kernel stack that would not be a security problem in normal operation and could result in a considerable number of usable data from the attackers' perspective.

An attacker can then carry out a targeted stack spray attack [82] to place controlled data on specified stack locations.

As shown in Listing 2, the `address` variable will be initialized inside the target socket's `recvmsg` handler, which means it will always be initialized and copied to user

```
1 int __sys_recvfrom(...)
2 {
3     struct sockaddr_storage address;
4     ...
5     // initialize `address`
6     sock->ops->recvmsg(...);
7     ...
8     // copy address to user space
9     copy_to_user(uaddr, &address, ulen);
10 }
```

Listing 2. The simplified code snippet of recvfrom system call handler in the Linux kernel.

space in the normal control flow. But if the `sock` data structure is fully overwritten by an attacker through a vulnerability, when CFHP is obtained at `sock->ops->recvmsg`, the `address` variable is not initialized, thus susceptible to the targeted stack spray attack.

4.4.3 Weaponizing Spilled User Space Data

After obtaining enough data spillage on the kernel stack, the moment an attacker invokes the triggering system call and acquires CFHP, they have stack control as well. In other words, the attacker can immediately launch code reuse attacks. In this work, we focus on executing a ROP payload.

Starting ROP. In the optimal case, the stack pointer might be pointing directly to user-controlled data when the CFHP is triggered. However, even if it is not, the typical proximity of user-controlled data to the stack pointer means that the attacker can "reach" it by using the CFHP to redirect execution to a stack-shifting gadget such as `add rsp, X; ret`. After this redirection, the attacker's ROP chain, stored in user-controlled portions of the kernel stack, will be triggered.

Small Chains. However, the initial ROP payloads enabled by RetSpill are too small for full exploits. This is due to three factors.

First, different triggering system calls exhibit different data-spilling behaviors depending on how they process userspace data. While the amount of Preserved Registers is stable across different system calls, the amount of Valid Data, data spilled due to Calling Conventions, and Uninitialized Memory varies widely.

Second, the CFHP itself might require specific inputs in some registers and memory to trigger successfully. For example, triggering the CFHP for CVE-2010-2959 requires a specific value in the `fd` argument (passed via the `rdi` register) to the triggering `ioctl` system call. This means that spilled values from `rdi`, regardless of the spillage method, cannot be used in the ROP chain (as they cannot have arbitrary values). Other vulnerabilities require more arguments to have precise values, reducing the amount of controllable spilled data further.

Third, spilled user space data is not contiguous, and thus some of the resulting gadgets must be dedicated to shifting the stack to the next controllable spilled data.

With resulting ROP chains as short as 7 gadgets (as measured in §4.7.1), some cleverness is required to achieve full exploitation.

Independent ROP. Recall that the CFHP that initiates the ROP attack is provided by an overwritten victim object representing a system resource in kernel heap. Thus, all threads in the exploit process have access to the system resource, and also the victim object. An attacker can create a separate thread to execute the ROP chain so that ROP chain termination does not affect the main thread. This approach makes each ROP chain execution isolated in each thread, which means each ROP chain execution is performed under a different task context.

Notice that this step does not involve re-triggering vulnerabilities for additional

ROP chain iterations. Hence, in practice, it does not introduce extra unreliability for exploits (for example, from having to otherwise repeat heap massaging efforts).

There are two advantages to this independent ROP chain execution. First, as described in §4.2, it enables the use of the `do_task_dead` function as an epilogue gadget in the ROP chain to end the chain gracefully. Second, since the ROP payload is executed in a new thread, even if the ROP payload fails, the kernel will only terminate the executing thread instead of the main thread. Therefore, as we will describe in §4.7.3, one could repetitively try the execution of ROP payloads to bypass certain probabilistic kernel protections without exploit reliability degradation.

Unlimited Arbitrary Read/Write/Exec. The victim object that gives the attacker CFHP is typically on the kernel heap and the attacker can put data on the kernel stack. Each time the attacker invokes the triggering system call, which grants the attacker CFHP, they can put different controlled data on the kernel stack and then invoke a different ROP chain. Combined with the Independent ROP Chain primitive, this feature of RetSpill effectively gives the attacker the ability to trigger different ROP chains as many times as desired without swapping payload in the kernel heap. Then the attacker can have unlimited Arbitrary Read/Write primitive, as well as the ability to invoke arbitrary functions in kernel space and get back the return values unlimited times.

In other words, RetSpill turns one single CFHP into unlimited Arbitrary Read-/Write/Exec without sacrificing exploit reliability.

Putting it all together. At this point, the controlled victim object in the kernel heap becomes something analogous to a *backdoor*. By using the triggering system call on the victim object, the attacker can do Arbitrary Read/Write in kernel space and invoke Arbitrary Kernel Function Calls and obtain the return values. Moreover, they

Defense	CFHP Achievable?	RetSpill Works?	Deployed?†
SMEP/SMAP/KPTI	✓	✓	✓
RANDKSTACK	✓	✓	✓
STACKLEAK	✓	✓	✗
FG-KASLR	✓	✓	✗
KCFI/IBT	✓	✓	✗
Shadow Stack*	✓	✓‡	✗
CFI+Shadow Stack*	✗	✗	✗

Table 6. RetSpill's applicability against all modern defenses. * Shadow Stack is unavailable in Linux on x86_64 architecture. † Whether the defense is deployed in at least one of the major Linux distros mentioned in Table 7. ‡ RetSpill works with restrictions.

can perform all these actions in kernel space cleanly for *unlimited times* from separate task contexts. In other words, the boundary between the user space and the kernel space is completely broken.

4.5 RetSpill vs Modern Defenses

In this section, we state how RetSpill manages to bypass a series of modern defenses in the Linux kernel. These defenses cover all existing protections deployed in kernels shipped in major Linux-based operating systems (listed in Table 7). We also evaluate the effectiveness of emerging protections that are not implemented, not production-ready, or not deployed yet. A brief summary can be found in Table 6.

SMEP/SMAP/KPTI. These protections aim to prevent attackers from directly accessing user space data when obtaining CFHP in kernel space. But RetSpill does not rely on data in the user space. Instead, it solely relies on user-controlled data on the

OS	Version	Status
Ubuntu	22.04	✗
Debian	11	✗
CentOS	9	✓
Fedora	36	✗
Arch Linux	2022.10.01	✗
Mageia	8	✗
Mint	21	✗
Open SUSE	Leap 15.4	✗

Table 7. The `panic_on_oops` configuration in major Linux-based operating systems.

kernel stack. In other words, RetSpill shows that user-controlled data on the kernel stack alone is enough to compromise the security of the full system, thus bypassing these three protections.

RANDKSTACK. RANDKSTACK [97] introduces a randomized offset between function stack frames and the `pt_regs` region as shown in Figure 4. This protection aims to randomize the kernel stack layout at each system call entry to prevent deterministic data control on the kernel stack when CFHP is obtained. However, this protection only mitigates attacks that access data spillage from Preserved Registers and Uninitialized Memory. Attackers can still deterministically access controlled data spilled by Valid Data and Calling Convention because they are part of the stack frames.

Moreover, we discover that Preserved Registers data spillage can still be used for exploitation despite the presence of this protection. Due to the low entropy (5-bit [97]) in the randomized offset of the current design, attackers can hardcode an offset and prepend a "ret-sled" (a series of `ret` gadgets) to the actual ROP chain to increase the chance of executing the actual ROP payload correctly.

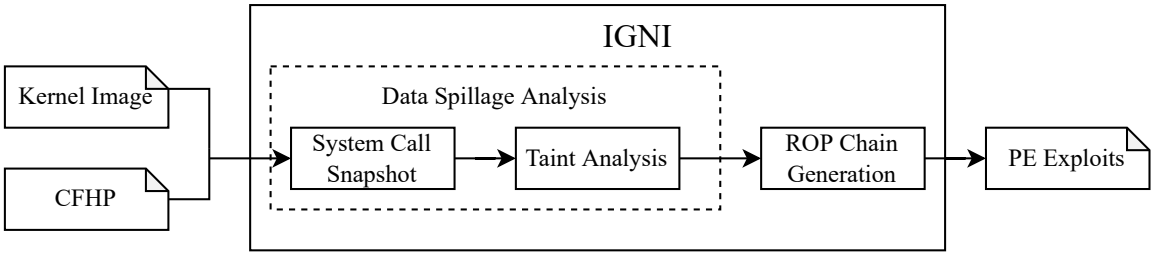

Figure 5. IGNI Design Overview

More importantly, if the target system disables `panic_on_oops`, which is true by default for most major Linux-based operating systems, as shown in Table 7, RANDKSTACK can be fully bypassed. In other words, RetSpill can bypass RANDKSTACK on most of the Linux distros. The reason is that with `panic_on_oops` disabled, an oops in task contexts will not crash the kernel. By using threads to trigger the payload, if an attempt fails, only the offending thread will be terminated. The exploit process will be kept intact and can continue attempts until the exploit succeeds. Such a create-thread-and-retry attack works only if failed attempts only generate oops and do not crash the kernel. We demonstrate that this is realistic—by calling a system call that is known to place "safe" data, such as NULL pointers, on the kernel stack multiple times to ensure that failed attempts can only trigger invalid-memory access and generate oops. To the best of our knowledge, we are the first to demonstrate that RANDKSTACK can be fully bypassed through recovery from oops.

STACKLEAK/STRUCTLEAK/INITSTACK. These protections prevent attackers from accessing uninitialized memory. More specifically, STACKLEAK [94] clears the kernel stack after each system call and INITSTACK [95]/STRUCTLEAK [78] ensure stack variables are initialized before use. In other words, they only prevent user data spillage from Uninitialized Memory. RetSpill is still effective by using other user data spillage methods to obtain stack control.

FG-KASLR. Function Granular Kernel Address Space Layout Randomization (FG-KASLR) [68] shuffles all kernel function addresses during boot time to protect the kernel from CFH attacks. However, function-level randomization is not enough to mitigate RetSpill. Other code snippets, such as assembly stubs, can also be compiled into executable sections with fixed offsets in the kernel. With RetSpill, attackers can look for ROP gadgets in the position-invariant code and use the arbitrary-read primitive to dynamically resolve function addresses, as demonstrated by Dang Khac Minh Le [72]. Then they can trigger the payload again and use the resolved function addresses to achieve the exploitation using RetSpill.

KCFI/IBT. KCFI [115] and Indirect Branch Tracking (IBT) [143] are two control-flow integrity (CFI) protection schemes merged into the Linux kernel recently. They both aim to protect the Linux kernel from forward-edge control-flow hijacking attacks. While they make it harder, they do not eliminate CFHP attacks.

Due to compatibility issues, even with the presence of these strong protections, some call targets in the Linux kernel are still not protected, susceptible to CFHP attacks. For example, in a kernel compiled with KCFI, `__efi_call` function does not verify its call targets. An attacker can overwrite the call target and obtain PC-control. We confirmed with the Linux kernel team that some call targets are not verified due to incompatibility between some subsystems and the CFI schemes.

Even with perfect implementation, attackers can still obtain PC-control by hijacking backward-edge control flow because KCFI/IBT does not protect them. For example, attackers have demonstrated the capability of turning a heap-based vulnerability to the kernel stack (dubbed kernel stack overflow primitive) [63], which can tamper the backward-edge control flow to obtain PC-control.

Once PC-control is obtained, attackers can carry out RetSpill despite the presence

of KCFI and IBT because RetSpill itself does not rely on forward-edge control-flow hijacking.

Shadow Stack. Shadows Stack is proposed to protect backward-edge control in the Linux kernel. However, it is not implemented yet in the Linux kernel for the x86_64 architecture. We only analyze its security guarantee against RetSpill in theory. In kernels protected by Shadow Stack, attackers can obtain PC-Control from forward-edge control flow hijacking, but they cannot ROP. To carry out RetSpill, attackers can use other code reuse attacks that do not rely on backward-edge control flow hijacking, such as JOP[9]/PCOP[100] to utilize data spillage from user space,

CFI+Shadow Stack. In theory, on a kernel deployed with perfectly implemented CFI schemes and Shadow Stack, attackers cannot obtain CFHP anymore, which prevents RetSpill.

4.6 Semi-Automatic RetSpill

RetSpill has a critical impact on the Linux kernel because it can not only escalate privilege but also has the potential to achieve the escalation in a semi-automated pipeline. To show the feasibility of semi-automation, we represent IGNI, a framework that can semi-automatically generate RetSpill attacks for many real-world CFH cases. As the goal of this work is to show the feasibility rather than creating a tool to facilitate malicious attacks, we only take one attack payload for privilege escalation, namely executing `commit_creds(init_cred)` and returning back to user space gracefully. Creating attacking tools that can comprehensively leverage different attack vectors is out of the scope of this paper.

The design of IGNI is shown in Figure 5. IGNI takes a proof-of-concept (PoC)

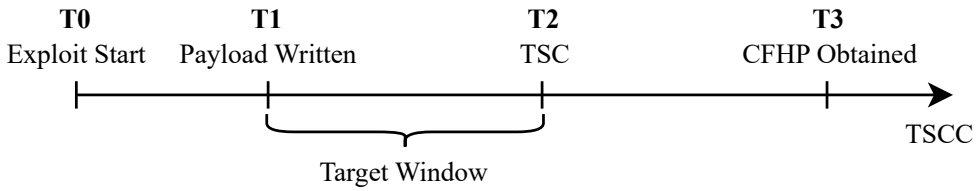

Figure 6. CFH exploits timeline

binary that demonstrates CFHP and a target kernel image as input, and outputs all the information needed for carrying out RetSpill. More specifically, it outputs 1) the address of a stack-shifting gadget that can transfer the initial CHFP into kernel stack and 2) an `ignite` function that performs data spillage, invokes the triggering system call, and leads to PE automatically. By setting the initial CFHP to the stack-shifting gadget and changing the original triggering system call to `ignite` function, the initial crashing PoC is turned into a full-fledged PE exploit.

Our approach relies on virtual machine (VM) snapshots. We take a VM snapshot when the triggering system call just enters kernel space, which ensures deterministic CFHP. Then, we analyze all user data that flows into the kernel space and identify user data that can be used for data spillage. Finally, we use symbolic execution to generate a ROP chain using the spilled data and link it with the initial CFHP to perform a full exploitation.

The design of IGNI involves the three following challenges:

- How to identify the triggering system call when thousands of irrelevant system calls will be invoked during the lifespan of an exploit process?
- Once we locate the triggering system call, how to distinguish user-controllable data from others?

- After finding the kernel data provenance, how to generate a PE payload on the discrete controlled region?

We detail how to conquer these challenges in this section.

4.6.1 Snapshotting the Triggering System Call

In Linux kernel exploits, CFHP is usually obtained through system calls. We call the system call that obtains CFHP the triggering system call (TSC). During the life span of the exploit process, thousands of irrelevant system calls will be invoked, it is challenging to identify the correct system call that will lead to CFHP and take a VM snapshot at its entrance. For example, in the exploit for CVE-2017-7533 [140], attackers use `write` system call to hijack the kernel control flow. The one `write` system call leading to CFHP is TSC, while another `write` system call displaying some debug messages is irrelevant.

We define triggering system call candidates (TSCC) as system calls invoked by the exploit process and having the same syscall number as TSC. The challenge becomes: how to identify the TSC among all TSCCs while the number and order of TSCCs may differ across runs (because of threading).

Intuitively, among all TSCCs, the last one invoked before obtaining CFHP is TSC. However, this assumption is wrong. In practice, the exploit process can issue other system calls, including other TSCCs, after issuing the TSC. The reasons are twofold. First, after the TSC is invoked, before it reaches the code that grants CFHP, other threads in the exploit process may race with the execution and invoke more TSCCs. Second, even when the triggering system call is executing, the kernel may stop its execution halfway and switch to execute other threads, which can issue more system

calls, because of the voluntary preemption mechanism in the Linux kernel. Depending on how the exploit is implemented, the number of extra TSCCs after invoking the TSC can be drastically different.

One naive solution will be taking a VM snapshot at the entrance of each TSCC and verifying whether it is the TSC. This solution does not work because constantly taking VM snapshots alters the timing difference between kernel threads, thus failing time-sensitive exploits (*i.e.*, race condition exploits). Notice that binary search does not apply either because the number of TSCCs varies across runs.

We show the timeline of CFH exploits with respect to TSCC in Figure 6. Our heuristic-based search algorithm is built upon an insight that TSC can be deduced from a snapshot taken between T1-T2. This is because after the exploit payload is placed correctly in the heap, CFHP can be obtained deterministically [141]. To deduce the TSC, we run the snapshot to T3, obtain information about the TSC thread (`task_struct`), rewind the snapshot, and take a snapshot at TSC using the thread-specific information.

To take a snapshot between T1-T2, we use a search algorithm with increasing granularity. More specifically, at the start of the search, we take a snapshot of every `N` TSCCs until we obtain CFHP. Then we revert the execution to the last snapshot and start taking a snapshot of every `N/2` TSCCs. We repeat this process until we obtain CFHP by executing one single TSCC. This algorithm ensures at least one snapshot is taken between T1-T3. Since T2-T3 is a time frame less than the time it takes to finish one system call (TSC), it is likely to obtain snapshots between T1-T2. As demonstrated in Section 4.7.2, the algorithm succeeds for all 22 exploit samples.

4.6.2 Identifying User-controllable Data

In this step, we mutate all potential userspace inputs (*i.e.*, registers and user memory) that 1) flow into the kernel space and 2) are not relevant in obtaining CFHP. In other words, we want to identify all userspace inputs that can be used for data spillage.

Notice that the VM snapshot taken from the previous step ensures the deterministic execution of TSC: it will always lead to CFHP without external intervention. We mutate all user space inputs word by word (because they are used for placing gadgets) directly in a fresh VM state (by rewinding VM states using the snapshot) and continue the execution. If the execution still leads to CFHP, the word can be used for data spillage since it does not affect CFHP. The mutation is performed by directly modifying user space registers and hooking kernel functions that consume user space data, namely, `copy_from_user` and `get_user`.

Then we analyze the provenance of all data spillage for payload generation in the next step. Specifically, we taint the whole kernel stack and CFHP-irrelevant userspace input with different special values in the VM state and then continue its execution to CFHP. By analyzing the tainted data on the kernel stack when CFHP is obtained, we can learn the amount of data spillage we control and their provenance.

4.6.3 ROP Chain Generation

In this step, we generate a ROP chain based on the user-controlled data on the kernel stack. There are two challenges in this task. First, the user-controlled data scatter over the kernel stack because they are spilled on the stack through various

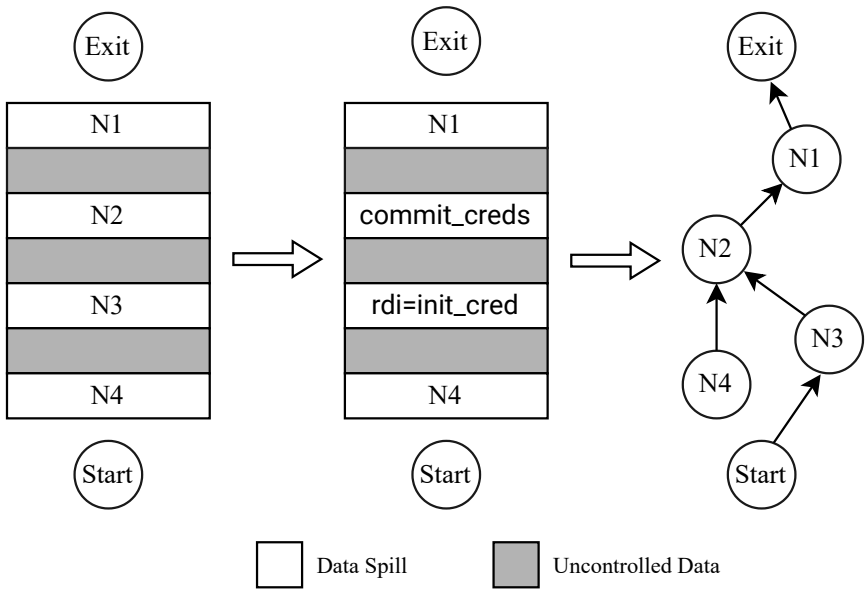

Figure 7. IGNI turns the discrete ROP chain generation problem into a graph search problem. Each node in the graph is a controlled data spill and each edge is a bridging gadget that can perform the control flow transfer.

methods (Section 4.4.2), as shown in Figure 7. Second, there are implicit constraints on the user-controlled data because several data spillages may come from the same source (*e.g.* `rbx` can lead to both Saved Registers and Calling Convention spillages), and they will be of the same value on the kernel stack.

ROP chain generation on discrete controlled regions is an open challenge: all existing automatic ROP chain generation solutions assume consecutive data control on the stack [19, 52, 105, 106, 114, 129]. The difficulty lies in how to connect multiple controlled areas while maintaining CFH.

To tackle this problem, we propose to use graph search to solve the controlled area connection problem and find potential ROP chain candidates, then use symbolic execution to verify ROP chains, similar to BOPC [59]. Notice that the longer the needed ROP chain payload is, the more implicit data constraints will be added to

Table 8. Data spillage is prevalent in system calls. The abundance of data spillage allows 20 out of 22 proof-of-concept programs that manifest CFHP to be semi-automatically turned into full privilege escalation exploits. Among the 20 success samples, five do not have public end-to-end CFH exploits. For ten samples, the techniques used in the original exploits no longer work with the presence of modern protections, but IGNI successfully make them work again automatically using RetSpill. Detailed information on the original techniques is shown in Table 11 in Appendix. Preserved Registers (PR), Calling Convention (CC), Valid Data (VD), and Uninitialized Memory (UM), the original exploit does not provide CFHP or there is no public end-to-end exploit (-). * RetSpill works on CVE-2021-27365 with manual modification.

CVE	Version	Trigger System Call	PR	CC	VD	UM	Total	Total w/o UM	S.A.	RetSpill Automated
2010-2959	4.15	ioctl	9	7	0	10	22	12	✗	✓
2016-0728	4.15	keyctl	4	3	0	0	7	7	✗	✓
2016-4557	4.15	close	5	3	0	0	8	8	-	✓
2016-6187	4.15	read	3	6	0	2	11	9	✓	✓
2017-2636	4.15	recvfrom	4	8	0	14	26	12	✗	✓
2017-6074	4.15	recvfrom	4	8	0	14	26	12	✗	✓
2017-7184	4.15	read	3	6	0	2	11	9	✓	✓
2017-7308	4.15	lseek	4	5	0	0	9	9	✗	✓
2017-7533	4.15	write	3	6	0	3	12	9	-	✓
2017-8824	4.15	getsockopt	2	5	0	2	9	7	✗	✓
2017-10661	4.15	clock_adjtime	4	7	24	1	36	35	✗	✓
2017-11176	4.15	setsockopt	3	7	0	0	10	10	✗	✓
2018-6555	4.15	getsockopt	2	6	0	1	9	8	✗	✓
2021-3490	5.11.0	prctl	12	4	0	3	19	16	-	✓
2021-3492	5.4.94	read	9	0	0	0	9	9	✓	✓
2021-4154	5.4.120	read	9	0	0	6	15	9	✓	✓
2021-27365	5.11.0	sendmsg	10	0	1	32	43	11	✗	✗*
2021-43267	5.11.0	ioctl	11	1	0	15	27	12	-	✓
2022-0185	5.4.120	read	9	0	0	6	15	9	✓	✓
2022-1786	5.10.90	execve	9	0	0	0	9	9	-	✓
2022-25636	5.11.0	read	9	0	0	0	9	9	✓	✗
2022-29581	5.4.170	ioctl	10	3	0	9	22	13	✓	✓
AVERAGE			6.1	3.9	1.1	5.5	16.5	11.1		

S.A.: original technique Still Applicable.

the graph. Thus, our approach may not scale to long ROP chains. In this work, we only focus on executing `commit_creds(init_cred)` and returning back to user space gracefully to demonstrate RetSpill.

We first fit independently-generated (using `angrop`[114]) ROP chain pieces into controlled memory and stitch them together using stack-shifting gadgets (*e.g.*, `add rsp, 0x10; ret`). For example, in Figure 7, we fit `| pop rdi; ret | init_cred |` into `N3` first. Then the stitching algorithm turns each ROP chain piece and other controlled gadgets into nodes in a Directed Acyclic Graph. Two nodes are connected if there is a stack-shifting gadget that transfers the stack control from one node to the other. The start node is a special node that does not represent any controlled memory. Instead, it represents the initial CFHP. An exit node is a Preserved Register data spillage. We explain its significance later. At this stage, each path in the graph represents an execution flow that starts from the initial CFHP provided by the given PoC program (start node), traverses all ROP chain pieces, and ends with a Preserved Register spillage node. So far, the algorithm does not guarantee a successful ROP execution because of implicit data constraints. For example, if a `pop rdi; add rsp, 0x10; ret` stack-shifting gadget is chosen after setting `rdi` to `init_cred`, the ROP chain will fail. But the algorithm reduces the amount of potential ROP chain candidates because it solves the controlled area connection problem. We then use symbolic execution (*i.e.*, `angr`) to verify the execution results of the chains to select a working chain that can escalate privilege and reach a controlled Preserved Register spillage.

Every Preserved Register data spillage can end ROP chains in kernel space gracefully. This is because preserved registers are stored in `pt_regs` and KPTI trampoline [72] is a code snippet that pops all the `pt_regs` and returns back to user space. By

carefully calculating offsets in KPTI trampoline, we can use one Preserved Register gadget to resume the normal execution of the KPTI trampoline and return back to user space gracefully. For example, suppose the data in `pt_regs` is `r10|r9|r8` and the trampoline needs to restore the registers by `pop r10; pop r9; pop r8`. If we set `r10` (part of our ROP chain) to the address of `pop r9; pop r8;`, when `r10` is consumed by our ROP chain, the stack content will become `r9|r8` and the PC points to `pop r9; pop r8`, which resumes the normal execution flow of the KPTI trampoline and can return back to user space gracefully.

As a result, our algorithm ensures a payload that starts from the initial CFHP, runs the privilege escalation payload, and returns back to user space gracefully.

4.7 Evaluation

In this section, we evaluate the RetSpill attack by answering three questions as follows:

- How feasible is the RetSpill attack when applied to real-world vulnerabilities?
- How effective is the semi-automatic RetSpill attack applied to real-world vulnerabilities?
- How effective is RetSpill in bypassing existing Linux kernel protections?

All the experiment artifacts are open-source.

4.7.1 Data Spillage Feasibility

We evaluate RetSpill's applicability by measuring the amount of data on the kernel stack that is fully controllable by attackers when a triggering system call is

invoked. We only include fully controllable gadgets, because partial control on stack data (*e.g.*, 32-bit control on a 64-bit machine) does not necessarily enable attackers to execute gadgets, thus it does not reflect the applicability of RetSpill. The numbers are obtained by using the taint analysis described in §4.6.2.

Dataset. We create a dataset consisting of 22 proof-of-concept exploits that provide CFHP for evaluating the effectiveness of RetSpill. It includes 13 exploits from previous work [141] and nine publicly accessible exploits. In total, we exclude 4 exploits from the dataset of previous work. CFHPs in 2 of 4 excluded samples are not triggerable in system calls, applying RetSpill to those 2 exploits requires significant engineering effort, thus we exclude them. The other 2 excluded samples only work when the SMAP protection is disabled, which requires a weaker threat model than our paper's. As a result, we exclude them from our dataset.

In the dataset, we enable all the protections compatible with the kernels, namely, `SMEP`, `SMAP`, `KPTI`, `NX-physmap`, `CR pinning`, and `STACK CANARY`. We do not enable `STATIC_USERMODE_HELPER` because it is not compatible with kernels used in the dataset. FG-KASLR, RKP, and PT-Rand are not enabled because they are not in the Linux kernel main tree, thus not in our kernel source code.

As shown in Table 8, there is a considerable amount of user-controllable data on the kernel stack when CFHP is obtained in the triggering system calls. On average, when attackers obtain CFHP, there are 16.5 gadgets readily on kernel stack, which is more than enough for carrying out privilege escalation.

Notice that in the same exploit with the same payload, using different triggering system calls to obtain CFHP will lead to a different exploitation scenario because of the difference in stack layouts. Thus, by carefully choosing triggering system calls, the amount of fully controlled data on the kernel stack can be greatly increased. We

CVE	P.R.	C.C.	V.D.	U.M.	Total
2010-2959	3	6	29	75	113
2016-4557	3	6	29	75	113
2017-7308	3	6	29	75	113

Table 9. The number of fully controllable gadgets by trigging payloads using the `poll` system call. Preserved Registers (P.R.), Calling Convention (C.C.), Valid Data (V.D.), and Uninitialized Memory (U.M.).

demonstrate this by using `poll` system call to trigger payloads for a subset of the exploits, the result is shown in Table 9.

4.7.2 Semi-Automation Effectiveness

We evaluate RetSpill's ability to semi-automatically turn proof-of-concept programs into privilege escalation exploits. We contacted the authors of targeted stack spray [82] and leak-kptr [18], but failed to port their prototypes to work against the modern Linux kernel. Thus, without a method to automatically perform targeted stack spray, we only use Valid Data, Preserved Registers, and Calling Conventions for data spillage in this evaluation. We tested the refined IGNI on the same set of real-world vulnerabilities built for the data spillage feasibility experiment (§4.7.1). As shown in Table 8, our prototype can generate end-to-end exploits for 20 out of the 22 samples even without using Uninitialized Memory for data spillage. This result demonstrates that RetSpill can be automated, and is thus that much more of a dangerous exploitation technique.

We further investigated and determined that CVE-2021-27364 *is* susceptible to RetSpill attack. Our prototype fails to automatically generate an exploit for it because of the lack of methods to perform targeted stack spray automatically. We successfully

```
1 long vuln_ioctl(...)
2 {
3     switch(cmd) {
4     case CMD_VULN:
5       obj = kzalloc(sizeof(vuln_obj_t), GFP_KERNEL);
6       copy_from_user(obj, arg, sizeof(obj));
7     case CMD_TRIGGER:
8       obj->func();
9     }
10 }
```

Listing 3. The simplified code snippet of the contrived vulnerable kernel module.

create an end-to-end exploit for CVE-2021-27364 using RetSpill by performing the targeted stack spray, which takes advantage of spilled data from uninitialized memory, manually. Our system fails on CVE-2022-25636 because of the lack of a specific bridging gadget to transfer control flow into a controlled region. In other words, RetSpill fails on this vulnerability for this specific kernel build. In our experiment, this kernel is compiled with the default configuration plus the aforementioned protections, and thus does not have much reusable code. However, in real-world kernel builds compiled with many additional functionalities enabled, we may be able to find the missing bridging gadget in the larger reusable code region and make RetSpill possible.

4.7.3 Protection Bypass

Due to the fact that RetSpill provides a probabilistic success rate in bypassing the most strict randomized-based protections, to accurately measure its success rate in bypassing protections, we use a synthetic vulnerability to guarantee CFHP. More specifically, we write a vulnerable kernel module that contains a contrived vulnerability that provides CFHP to simulate real-world exploit scenarios. We do not use real-world

exploits for this evaluation because the unreliability of real-world kernel exploits in providing CFHP [141] will affect our measurement of RetSpill's effectiveness. The gist of the vulnerable kernel module is shown in Listing 3.

RANDKSTACK. We compile a Linux kernel on `x86_64` architecture with the default configuration and enable RANDSTACK. Since RANDKSTACK adds a random offset on kernel stack, which can only mitigate Preserved Registers and Uninitialized Memory data spillage methods, in our RetSpill exploit, we only use Preserved Registers to put user data on the kernel stack to evaluate this mitigation objectively. Our RetSpill exploit aims to obtain privilege escalation using the `commit_creds(init_cred)` payload.

We perform 5000 trials each on the kernel with `panic_on_oops` on and off. In each trial, we insert the vulnerable kernel module and then repetitively run the exploit until it succeeds or crashes the kernel, following the previous work's method [141]. The result shows that RetSpill can achieve 100% success rate on systems with `panic_on_oops` off. Even with `panic_on_oops` on, the RetSpill exploit can still achieve 25.44% success rate, which is non-negligible. Since `panic_on_oops` is off by default on most Linux distributions as shown in Table 7, this suggests that even with RANDKSTACK on, RetSpill can achieve 100% success rate on most Linux distributions.

Notice that the 25.44% is achieved when the attacker only relies on Preserved Registers or Uninitialized Memory for data spillage. If they use Valid Data or Calling Convention for data spillage, this mitigation cannot provide any protection. In other words, if attackers can use Valid Data and Calling Convention to perform data spillage (which is common according to Table 8), they can still achieve 100% success rate despite the presence of RANDKSTACK.

KCFI/IBT. We simulate a vulnerability that grants PC-control in kernels com-

piled with CFI schemes by modifying the vulnerable kernel module used in the previous setup. More specifically, the function pointer is directly invoked using `__x86_indirect_thunk_rdi` that transforms forward-edge control-flow transition to backward-edge control-flow transition, which is not protected by the CFI schemes. This setting is realistic, as discussed in §4.5.

We successfully performed CFH attacks and obtained privilege escalation on kernels compiled with KCFI and IBT using RetSpill. The success is because KCFI and IBT only protect forward-edge control flow. Once PC-control is obtained, RetSpill does not rely on the ability to hijack forward-edge control flow again because one initial CFHP is enough for using data spilled on the kernel stack to carry out malicious payload through ROP. We demonstrate RetSpill's ability to bypass KCFI/IBT with experiments [3].

FG-KASLR. We contacted the authors of FG-KASLR and obtained its latest source code. Similar to the setup in RANDKSTACK, we compile the kernel with default configuration with FG-KASLR enabled. We write a tool to parse the kernel's FG-KASLR-related metadata and extract a position-invariant region of 4MB from the kernel image we build. As mentioned in §4.5, this region consists of executable code that is not within the boundary of any functions (*e.g.*, manually written assembly stubs). In total, we obtained 42631 unique gadgets from the position-invariant region, which is more than enough for dynamically resolving kernel function address. We succeed in using RetSpill to perform privileged escalation against the kernel compiled with FG-KASLR using the vulnerability in the contrived kernel module.

Case study: Breaking Security Boundary. We demonstrate the ability of RetSpill to break the security boundary between user space and kernel space using

[3]https://github.com/sefcom/RetSpill/tree/main/experiments/kcfi_eval

a real-world example. We implement an end-to-end proof-of-concept exploit for CVE-2022-1786 that performs Arbitrary Read/Write in kernel memory and Arbitrary Function Call in one single exploit by only changing the payload on the stack. Besides the individual primitives, this experiment also confirms RetSpill's ability to provide unlimited Arbitrary Read/Write/Exec primitive. In our analysis, Independent ROP Chain proves to be vital in obtaining CFHP multiple times. This is because, in this specific exploit, CFHP is obtained after the exploit thread acquires a lock. Since the control flow returns back to user space without releasing the lock, if we try to obtain CFHP again in the same thread, the exploit will stop indefinitely waiting for the lock to be released. Independent ROP Chain provided by RetSpill avoids the locking issue because each CFHP is obtained in a different thread.

4.8 Defense

In this section, we explore plausible protections against RetSpill besides CFI+Shadow Stack, which is not ready yet and requires special hardware. To mitigate this exploitation technique, the key is to eliminate *all* data spillage on the kernel stack.

Preserved Register. RANDKSTACK is the only existing protection that mitigates Preserved Register data spillage. It comes with near-zero performance overhead [30] but offers probabilistic protection. In our experiment, exploits can still have a 25.44% chance of succeeding against kernels with this protection. Moreover, this protection should be coupled with `panic_on_oops`, or it can be fully bypassed (§4.7.3). To completely mitigate this data spillage, we suggest preserving user space registers elsewhere instead of on the kernel stack. For example, the registers can be saved in the

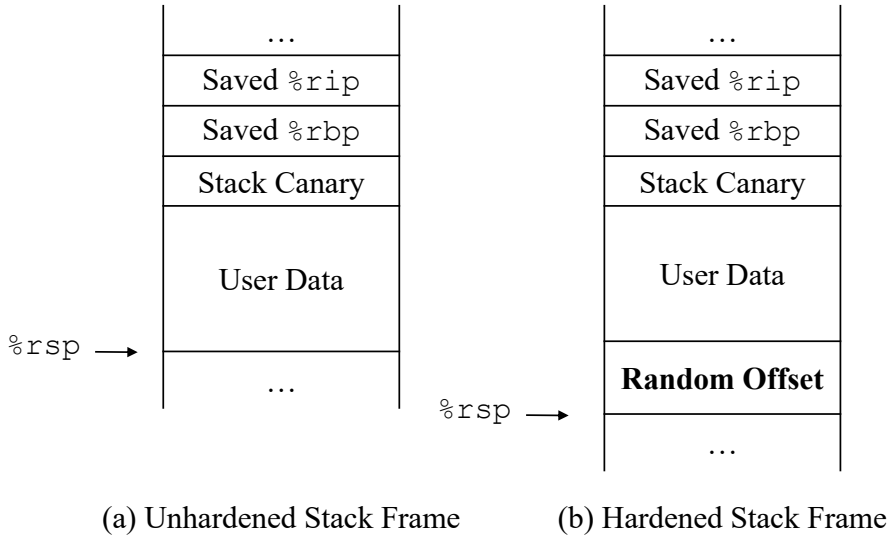

Figure 8. Unhardened and Hardened Stack Frames.

associated task's `task_struct` data structure. This way, even if CFHP is obtained by attackers, user space registers will not be directly accessible on kernel stacks to facilitate further exploitation.

Uninitialized Memory. For Uninitialized Memory data spillage, all STACKLEAK, STRUCTLEAK, INITSTACK, and RANDKSTACK can provide different levels of protection. STACKLEAK clears the kernel stack after each system call, which ensures zero usable Uninitialized Memory data spillage when CFHP is obtained. However, STACKLEAK has an average overhead of more than 40% [81]. INITSTACK and STRUCTLEAK use compilers' pattern initialization feature to initialize stack variables, which also ensures no Uninitialized Memory data spillage while having only a performance overhead of 2.7%-4.5% [136]. As mentioned above, RANDKSTACK has near-zero performance overhead but only provides probabilistic protection. Considering protection and performance, we suggest enabling INITSTACK or STRUCTLEAK to avoid Uninitialized Memory data spillage on kernel stack.

Calling Convention and Valid Data. The latest version of the Linux kernel has

deployed a protection that clears user space registers that are not part of the ABI in the system call entry stub. As shown in Table 8, new kernel builds have significantly less data spillage caused by Calling Convention compared with old kernel builds because of this protection. However, data spillage caused by Calling Convention is still not eliminated.

Data spillage caused by Valid Data and Calling Convention is part of the design, and thus cannot be mitigated by existing protections in the Linux kernel. However, they are enough for successfully launching RetSpill to achieve full system compromise. We demonstrate it by successfully exploiting CVE-2016-4557 with RetSpill only using Valid Data and Calling Convention spillage. To fill the gap, we developed a protection to mitigate this data spillage. As shown in Figure 8, our protection inserts a random offset at the bottom of each stack frame. To prevent the random offset region from becoming the source of Uninitialized Memory data spillage, we clear the region right after the insertion. As a consequence, when CFHP is obtained by attackers, they will not be able to reuse user data directly without guessing the random offset, which drastically reduces exploit reliability.

Denote the average call-stack depth as D, the entropy of the proposed protection is $5 + log(D)$ bits, which is slightly higher than RANDKSTACK(5 bits). More importantly, the protection prevents attackers from accessing *all* data spillage, whereas RANDKSTACK only prevents Preserved Registers and Uninitialized Memory.

Due to its lightweight nature, the proposed protection only introduces an average performance overhead of 0.61% as evaluated on benchmarks from LMBench and Phoronix. The detailed evaluation result can be found in Table 10.

Benchmark	Vanilla	Hardened	Overhead
Phoronix			
Apache (Reqs/s)	149470.35	150295.67	0.55%
PHPBench (Score)	830189	840718	1.27%
PyBench (ms)	889	885	0.45%
OpenSSL-SHA256 (byte/s)	20984220257	20797618197	-0.89%
OpenSSL-SHA512 (byte/s)	6859393693	6812764710	-0.68%
OpenSSL-RSA4096 (sign/s)	3119.7	3105.9	-0.44%
OpenSSL-RSA4096 (verify/s)	204263.3	203177.1	-0.53%
LMBench			
Context Switch (ms)	2.036428571	2.047	-0.52%
UDP (ms)	7.3116	7.4854	-2.32%
TCP (ms)	9.2525	9.3546	-1.09%
10k File Create (ms)	10.99	10.89	0.92%
10k File Delete (ms)	5.89229	5.71896	3.03%
pipe (MB/s)	3648.6	3587.8	-1.67%
AF Unix (MB/s)	8968.9	8712.5	-2.86%
TCP (MB/s)	8081.6	7964.8	-1.45%
mmap Reread (MB/s)	18.61	17.95	-3.55%

Table 10. Performance overhead evaluation of the proposed defense on Phoronix and LMBench.

4.9 Discussion

Automatic Exploit Context Analysis. In this work, we only analyze RetSpill under limited exploit contexts. As mentioned in §4.7.1, different system calls can be used to trigger the payload on the heap, which create different exploit contexts. To fully evaluate the severity of RetSpill, an automatic solution is needed to explore all possible exploit contexts for each vulnerability. One such work is FUZE [134], which combines fuzzing and symbolic execution to facilitate exploit generation automatically. We can potentially use its fuzzing and symbolic execution engines to explore different exploit contexts automatically. We leave the integration with FUZE for automatic exploit context analysis as future work.

Fine-Grained Randomization Protections. Existing such protections in the Linux kernel (*e.g.*, FG-KASLR [68] and kR^X [92]) cannot randomize all kernel executable code, which leaves the kernel vulnerable to RetSpill. Even with perfect fine-grained randomization protection, attackers can still use speculative probing [36] or an arbitrary read primitive to disclose usable ROP gadgets and launch RetSpill in JIT-ROP [110] style. Thus, we believe fine-grained randomization may not be a good solution to mitigate RetSpill.

Stealthy Rootkit. Primitives provided by RetSpill allow attackers to reliably program the kernel from user space. This opens up the possibility of rootkits that manipulate the kernel without escalating its privilege. As a consequence, this hypothetical rootkit can bypass intrusion detection techniques based on detecting suspicious privilege escalation behaviors [58, 102]. Worse still, its every action involves only the triggering system call, which means malicious logic can be executed in kernel space while only leaving a trace of seemingly benign system calls such as `close`.

Applicability to Other Systems. RetSpill is not specific to Linux. Any system that involves exchanging data between a trusted entity and an untrusted one needs to avoid untrusted data spillage on the trusted stacks. For example, other operating systems or hypervisors are potentially susceptible to RetSpill attack. We leave the verification and severity evaluation of RetSpill on other systems as future work.

4.10 Limitation

Uninitialized Memory Spillage Evaluation. In our current evaluation for IGNI, we do not include data spillage caused by uninitialized memory. This is because we lack a way to automatically and deterministically control uninitialized memory

on the kernel stack. The most relevant research lately on this topic are targeted stack spray [82] and leak-kptr [18]. We contacted the authors but failed to port their prototypes to the modern Linux kernel because of its rapid development in the past few years. Since the evaluation requires a new automatic approach to perform stack spray, which is out of the scope of this work, we leave it as future work.

Severity On Other Architectures. Data spillage is not an architecture-specific behavior, it exists on architectures other than x64_64 as well, such as AArch64. This work focuses on showing and utilizing user data spillage on x64_64 and demonstrate that it is so severe that exploits can be semi-automatically generated. However, the exploitability of user data spillage may vary due to different architecture designs. The lack of evaluation on other architectures is a limitation of this work.

4.11 Related Work

Exploring Memory Corruption Capability. Memory corruption capability is obtained after a vulnerability is triggered. In order to reliably trigger race condition bugs, Lee *et al.* [73] use interrupts to extend race windows for winning races and eventually achieve memory corruption capability. FUZE [134] explores different use sites of UAF vulnerabilities with under-context fuzzing. GREBE [76] proposes an object-driven fuzzing technique to trigger vulnerabilities in different contexts and explore different memory corruption capabilities. Syzscope [144] symbolizes kernel memory and extracts memory corruption capabilities through symbolic execution.

Obtaining Exploitation Primitive. With the memory corruption capability in hand, researchers propose various techniques to obtain exploitation primitives. Cho *et al.* [18] utilizes eBPF to achieve pointer leak from uninitialized stack variables. Lu *et*

al. [82] introduce targeted stack spray techniques to exploit use-before-initialization vulnerabilities. There are also many research works on kernel heap exploitation. SLAKE [17] introduces different techniques to manipulate heap memory layout. With the help of elastic objects, ELOISE [16] proposes a general method to obtain information leak primitive from an overwrite capability. KOOBE [15] explores different exploit primitives from heap out-of-bound write capability. KHeaps [141] systematically studies the reliability problem in kernel exploitation and proposes a new stabilization technique. In order to prevent heap exploitation, many defenses are proposed. AUTOSLAB [74] proposes to isolate different types of objects into different slab caches and thus mitigates memory layout manipulation. Hardened Usercopy [20] is introduced to the Linux kernel to prevent out-of-bound access exploitation.

Escalating Privilege. Privilege escalation could be done through control flow hijacking (CFH) attacks. Originally, attackers could directly hijack the control flow to userspace [65]. The introduction of kernel space and user space isolation (*e.g.*, KPTI [50, 119], SMEP [22]/SMAP [89]) mitigates such attacks. With the presence of isolation, attackers have to inject payload into kernel space. ret2dir [64] breaks this isolation with `physmap`, which is mapped in a predictable region in kernel space. To mitigate CFH attacks, KASLR [21] and KCFI [29, 35, 70, 137] are proposed. Specifically, KASLR randomizes kernel address space layout, making addresses of existing code unpredictable and thus downgrading the reliability of code reuse attacks. Recently, KASLR has been further extended as FG-KASLR [68], which randomizes address space layout at function granularity. CFI preserves the control-flow integrity to prevent CFH attacks. There are different implementations of CFI in the Linux kernel, including software-based [35, 39] and hardware-assisted [29, 70, 137]. Other than CFH attacks, data-only attacks are also capable of escalating privilege.

Through an overwriting primitive, attackers could tamper the credentials of the exploit process to escalate privilege [14]. DirtyCred [75] demonstrates privileged escalation by replacing unprivileged credentials with privileged credentials. To prevent data-only attacks, xMP [96] protects sensitive data utilizing hardware virtualization techniques. PrivGuard [125] creates duplicate memory to ensure the modifications to sensitive data are legitimate.

Our work systematically studies the data spillage problem on kernel stack and discovers a novel and powerful attack: RetSpill. We thoroughly evaluate RetSpill and show that it can be automated with our prototype, IGNI.

RetSpill could bypass FG-KASLR [68] which previous research – KEPLER [133] is not capable of. In addition, compared to KEPLER and conventional ROP attacks [52, 129], RetSpill has no requirement of register control and heap control, which significantly simplifies the process of launching attacks. We also do not assume the information of the base of `physmap` as KEPLER. Furthermore, RetSpill enables unlimited CFHP without changing the payload on the heap, which introduces no degradation to exploit reliability for further exploitation. This feature of RetSpill makes it a stable exploitation technique, which other "single-shot" exploitation techniques, such as KEPLER, are unable to achieve.

4.12 Conclusion

In this work, we applied exploit generalization on kernel CFH exploits. We first discovered that user data accessibility is the dominating factor for CFH exploittion techniques and identified kernel stack as a new attack surface. We show that untrusted user data on kernel stack are threats to Linux kernel security. In this research,

we discover RetSpill, a dangerous Linux kernel exploitation technique that requires minimal exploit primitives by placing user controlled data on the kernel stack. Provided with CFHP, RetSpill can utilize seemingly harmless user space data on kernel stack to break the boundary between user space and kernel space despite the presence of all commonly deployed protections in Linux. We thoroughly study the technique and show that it is applicable in 21 out of 22 public exploits. Worse still, we demonstrate RetSpill can be automated, which makes it even more dangerous. Our proof-of-concept prototype, IGNI, semi-automatically turns 20 out of 22 public exploits that manifest CFHP into full privilege escalation exploits. Finally, we explore potential protections to minimize the impact of RetSpill.

4.13 Appendix

CVE	Trigger System Call	Original Technique	A.P.	S.A.
2010-2959	ioctl	ret2usr	-	✗
2016-0728	keyctl	ret2usr	-	✗
2016-4557	close	N/A	-	-
2016-6187	read	KEPLER	②③	✓
2017-2636	recvfrom	change CR4	①	✗
2017-6074	recvfrom	change CR4	①	✗
2017-7184	read	pivot to heap	②④	✓
2017-7308	lseek	change CR4	①	✗
2017-7533	write	N/A	-	-
2017-8824	getsockopt	change CR4	①	✗
2017-10661	clock_adjtime	change CR4	①	✗
2017-11176	setsockopt	pivot to user	-	✗
2018-6555	getsockopt	ret2usr	-	✗
2021-3490	prctl	N/A	-	-
2021-3492	read	set_memory_x	①④	✓
2021-4154	read	pivot to heap	②④	✓
2021-27365	sendmsg	run_cmd	①④	✗
2021-43267	ioctl	N/A	-	-
2022-0185	read	pivot to heap	②④	✓
2022-1786	execve	N/A	-	-
2022-25636	read	pivot to heap	②④	✓
2022-29581	ioctl	pivot to heap	②④	✓

Table 11. All of the old techniques that do not require additional primitives are no longer applicable in modern systems. And all of the working techniques require additional primitives besides CFHP. Additional Primitives (A.P.) needed by the original technique compared with RetSpill, original technique Still Applicable (S.A.) against modern Linux kernel deployed with protections. ① rdi control, ② any register control, ③ physmap leak, ④ heap info leak, N/A: original exploit does not provide CFHP or there is no public end-to-end exploit.

Chapter 5

AUTOMATED CODE REUSE PAYLOAD GENERATION

Code Reuse (CR) attacks, such as Return-Oriented Programming (ROP) [107] or Jump-Oriented Programming (JOP) [9], are exploitation techniques that can turn a control-flow hijacking primitive into arbitrary code execution. Despite the presence of more advanced protections, such as Address Space Layout Randomization (ASLR), Position-Independent Executable (PIE), Control-Flow Integrity (CFI), and others, CR attacks remain highly relevant and are becoming increasingly more powerful with fewer requirements on the target programs. For example, researchers have demonstrated ways to perform code reuse attacks without the need for `ret` instructions [9, 13, 100], and attackers can inject executable code into the target process and utilize gadgets in it on the fly [110].

However, synthesizing CR attacks has long outgrown its malicious origins. Nowadays, it is a vital approach in the defender's arsenal to confirm exploitability, which is critical in prioritizing fixing severe bugs. At the time of writing, 52 out of 70 exploits accepted by Google's KernelCTF Vulnerability Rewards Program [41] use CR attacks to perform privilege escalation in the Linux kernel to demonstrate the exploitability of the vulnerabilities. Despite widespread use by both malicious and benign actors and contrasting its important role in modern exploitation, constructing a CR payload (called *chain*) is non-trivial, manual, and notoriously time-consuming. This unnecessarily delays the process of exploitability verification and requires a domain expert. Hence, research has proposed to automate the task of generating CR chains.

Attempts to solve this hard CR chain generation problem started with Q [105] and have seen much follow-up traction, resulting in JIT-ROP [110], LIMBO [106], SGC [104], RiscyROP [19], Crackers [28], and Arcanist [7] among others. Surprisingly, despite the multitude of techniques being proposed, *none* achieved widespread adoption or traction, and the problem of CR payload synthesis remains a time-consuming, manual effort.

This work aims to apply exploit generalization on existing CR chain generation techniques with the goal of finding a new one.

We first carefully studied the underlying techniques of these tools. During this process, we find that all existing works rely on a *generate-and-test* (GAT) [33] scheme: Given a target goal (*e.g.*, set `rax` and `rbx` to attacker-specified values), they will first generate a list of gadgets that can potentially achieve the goal and then use a heavyweight approach, *e.g.*, symbolic execution or SMT solvers, to verify whether the list of gadgets can be chained and achieve the desired goal. This algorithm results in a time complexity of $\mathcal{O}(2^n)$, where n is the number of gadgets. Thus, it does not scale well with large binaries. As a balance between capability and speed, most existing works [7, 19, 28, 104] decide to solve a *sub-problem* instead: they aim to find a chain within a maximum number of gadgets (N). We call it *bounded generate-and-test* (BGAT). This algorithm has a time complexity of $\mathcal{O}(n^N)$, which leads to the dilemma of choosing N. If N is too large, the algorithm will face state explosion; if N is too small, the tool cannot find useful chains.

This realization led us to dive deeper into why all existing works use the slow GAT scheme. Our study shows that the reasons are two-fold: a technique needs to 1) ensure all selected gadgets are chainable, and 2) their effects do not conflict with each other, but can actually satisfy the attacker's goal. This analysis suggests

that gadget chainability is the bottleneck of CR chain generation techniques, thus a dominating factor. The discovery of the dominating factor further inspires us to revisit the CR chain generation problem from a different perspective: what if we have a way of grouping gadgets that guarantees chainability? This way, we bypass the issues previous works face and only need to focus on finding groups that can satisfy the attack goal. This is a much easier task, promising improvements in terms of efficiency and effectiveness.

In this work, we present such a novel abstraction of gadgets that we call ROPBlock. It groups gadgets in a way that guarantees chainability between ROPBlocks. This layer of abstraction reduces the classic CR chain generation task, namely register setting, into an $\mathcal{O}(n)$ graph search problem. In practice, it proves to be 2-3 orders of magnitude faster than the GAT approach widely used by *all* existing works [7, 19, 28, 104, 106], as demonstrated in our large-scale evaluation. Besides its impressive performance, working on this intermediate layer built on top of gadgets allows us to model complex gadget types, such as gadgets with conditional branches and so-called ret2csu [84] gadgets. This significantly enhances the capability and real-world relevance of our CR chain generation engine. In addition, ROPBlock is architecture-agnostic, which allows our approach to seamlessly scale across architectures.

Based on our ROPBlock abstraction, we implemented our approach in a research prototype named ropbot that can generate CR chains in an architecture-agnostic manner. It is the first approach to support x86/x64/MIPS/ARM/AArch64/RISC-V for both little-endian and big-endian. Our evaluation shows that even on x64, an architecture that has been extensively tested in the past, ropbot outperforms the state of the art significantly for every single CR chain generation task on the datasets proposed by prior work. For the task of generating `execve` chains, ropbot achieves a

success rate of 89.7%, compared to 33.1% of the second-best technique, exrop. When trying to invoke `mmap`, which requires setting six arguments, ropbot finds CR chains for 5x targets compared to the second-best technique. Beyond these, ropbot is ready for real-world use and can generate complex CR chains. For example, it can generate chains that invoke three function calls (`dup-dup-execve`) in 3.7s on average—a feat no other tool can achieve. We also clearly define what a "Gadget" is, which leads to a new type of gadget called GOT gadgets. In fact, ropbot is used internally by Google, multiple vulnerability research companies, and serves as the foundation of some research prototypes.

Contributions. We make the following key contributions through exploit generalization:

- We analyze existing works in CR chain generation and point out that chainability is the dominating factor.
- We propose an abstraction layer of gadgets called ROPBlocks that guarantees chainability and significantly reduces the complexity of CR chain generation.
- Building on all datasets of prior work, we release a comprehensive dataset and evaluation framework that can serve as a benchmark for future CR techniques.
- We clearly define what a "Gadget" is, which leads to the discovery of GOT gadgets, a new category of gadgets that are pervasive in binary executables but have not been used so far.

To foster future research, we open source ropbot, our evaluation framework, and the datasets used in the evaluation at https://github.com/sefcom/ropbot.

5.1 Overview

In this work, we apply exploit generalization in the field of CR chain generation. We study existing techniques and aim to find a new technique that can solve the problem of *automatically synthesizing a code reuse chain*: Given a vulnerable binary program with a control flow hijacking primitive and a target payload computation, our goal is to automatically generate a CR payload that carries out the payload computation in the vulnerable binary. In this section, we state the challenges of tackling this problem in general and review shortcomings of state-of-the-art techniques in addressing them.

5.1.1 Challenges

There are many open challenges in the domain of automatically synthesizing CR chains. We categorize them into three categories: diverse gadget type support, cross-architecture gadget chaining, and state exploration.

5.1.1.1 Diverse Gadget Types Support

Traditionally, CR attacks have been limited to the model of return-oriented programming, where a gadget is considered to be a series of instructions that ends with `ret`. As CR techniques evolved, more gadget types have been recognized to be useful for CR chains. One example is the gadgets used in the return-to-csu (ret2csu) attack [84]. Gadgets ending with `call [mem]` or `jmp [mem]` exist extensively in programs; for example, gadget `mov rdx,r13; mov rsi,r14; mov edi,r15d; call [r12+rbx*8]` is

inserted into *all* dynamically linked ELF executables by compilers. Combined with other inserted gadgets, it can be used to invoke a pointer in memory with attacker-controlled arguments. Surprisingly, state-of-the-art does not account for these gadgets. Some approaches, like exrop [23] and RiscyROP [19], simply ignore these gadgets during gadget selection. Crackers [28] considers these gadgets, but it concretizes the memory content into *any* value they need, simply assuming an attacker has full control over the whole memory—an assumption that usually does not hold in practice. SGC [104] and Arcanist [7] can utilize these ret2csu gadgets if one extra constraint is satisfied: the address of the payload needs to be known, which implies that an information leak vulnerability must exist. In summary, how to automatically utilize these gadgets in realistic settings without requiring additional primitives is still an open research problem.

One intuitive solution to automatically leverage such gadget types would be to dynamically generate them on-the-fly, i.e., to write the needed gadgets into memory first, and then invoke them. Although the change is minor, this poses a problem that cannot be handled by the classic generate-and-test (GAT) algorithm used by all state-of-the-art works [7, 19, 23, 28, 104, 106]. Fundamentally, this is because all existing works only generate a list of available gadgets at the generation stage, which does not contain enough information for verification. Currently, all existing works only output chains consisting of gadgets that are *unconditionally chainable*. For example, for `pop rax; ret | pop rbx; ret`, no matter what the values of `rax` and `rbx` are, these two gadgets are chainable. But first writing data to memory and then invoking it as a gadget later creates a *conditionally chainable* situation. In other words, the list of gadgets may be chainable depending on the values in the chain. For example, considering the following list of the gadgets: `pop rax; ret|pop rbx; ret|`

Table 12. Capabilities of all open-source CR chaining tools. *BGAT refers to bounded generate-and-test.*

	SGC	**exrop**	**RiscyROP**	**Crackers**	**Arcanist**	**ropbot**
Architectures	x86/x64	x64	AArch64/RISC-V	x64*	x86/x64/ARM	Multi
Algorithm	BGAT	GAT	BGAT	BGAT	BGAT	Graph
Byte Generation	✓	✓	✓†	✗	✗	✓
Conditional Branch	✗	✗	✓	✗	✓	✓
Multi-Block Gadget	✗	✗	✓	✗	✗	✓
Return-to-csu	✗‡	✗	✗	✗§	✗‡	✓
Payload Chaining	✗	✓	✗	✗	✗	✓
Fully Utilize Gadgets	✗	✓	✓	✗	✗	✓

* Crackers claims to support multiple architectures, but only evaluated on x64.
† RiscyROP can output payload in bytes only with our modification.
‡ SGC and Arcanist can utilize ret2csu gadgets if the address of the payload on stack is known.
§ Crackers can utilize ret2csu gadgets only if all memory is controlled by attackers.

`mov [rax], rbx; ret| pop rcx; ret| pop rdi; call [rcx]`, they are only chainable if `rax == rcx` and `rcx` is a gadget that cleans up the return address pushed to stack by the `call` instruction. Without the additional constraints, neither symbolic execution nor SMT solvers can verify the feasibility of this chain, thus rejecting it. As a result, existing works fundamentally cannot model ret2csu style gadgets without additional primitives.

5.1.1.2 Gadget Identification across Architecture

Many existing CR chaining tools [101, 103, 104] search for gadgets using the Galileo Algorithm [107]: They first identify "anchor instructions", such as `ret` or `jmp` on x86, and then search backwards to find potential gadgets. However, there are a few shortcomings associated with this approach:

- **Overlooking gadgets:** The intuition behind using such "anchor instructions"

```
label1:                 label2:
    pop rax                 ret
    jmp label2
```

Listing 4. Gadget consisting of more than one basic block.

is that gadgets *usually* end with such instructions. However, not all potential gadgets necessarily do. For example, although most gadgets in ARM end with `bx` and `pop pc`, there is a significant number of instruction sequences that contain `ldr pc, [sp], #<X>`. While ignored by existing works [101, 103], they should be considered as gadgets.

- **Requiring domain knowledge and manual effort:**
  It is potentially possible to exhaustively collect all instructions that may serve as the end of a gadget. However, this approach requires domain knowledge and manual effort to support each architecture. Also, "anchor instructions" may differ for different compilers or calling conventions.
- **Overlooking multi-block gadgets:** Consider the example in Listing 4, where block A ends in a direct jump to block B, which ends in a `ret`. The reverse search algorithm will only mark block B as a gadget. In reality, any execution starting from block A will also end with `ret`, extending the pool of available gadgets. The underlying issue here is the reverse search starting from an "anchor instruction".

How to find all potential gadgets with minimal or no domain knowledge becomes even more challenging when attempting to find gadgets for more than one architecture.

5.1.1.3 Payload Chaining across Architectures

Payload chaining is an essential functionality to achieve arbitrary exploitation goals. For example, we may want to chain two payloads, where the first writes "/bin/sh" into memory and the second then invokes `execve` with "/bin/sh". Ideally, we have an architecture-agnostic solution to do so. However, previous work either does not support payload chaining [7, 19, 28, 104] or supports only x86/x64 [23]. Here, chains can be just concatenated as strings due to its architectural design: for example, to combine two chains, `pop rax; ret | 0x41414141` and `pop rbx; ret | 0x42424242`, we can just concatenate them into `pop rax; ret | 0x41414141 | pop rbx; ret | 0x42424242`. This is no longer a trivial task on other architectures, such as AArch64. There, functions end with `jmp x30` (still rewritten as `ret`). In other words, invoking a function and continuing execution now involves two steps: set `x30` to a gadget we want to execute after the function call and then invoke the function.

Furthermore, consider the following ARM-based chain: `ldr r0, [sp,#4]; ldr lr,[sp],#4; add sp,#8; bx lr | 0x0 | 0x41414141`. This chain loads `r0` from `[sp+4]`, `lr` from `[sp]`, and then increases `sp` by 12. If we want to chain it with `pop.w {r1, lr}; bx lr | 0x42424242` to set both `r0` and `r1`, the final chain needs to be like this: `gadget1 | gadget2 | 0x41414141 | 0 | 0x42424242`, which is completely different from the linear layout that we usually see on x86/x64. In summary, different architectures with different calling conventions make generic payload chaining across architectures challenging.

5.1.2 Limitations in Existing Work

With these challenges in mind, we now review the capabilities and features of state-of-the-art CR chaining tools. We list the results of our analysis in Table 12, which shows one striking similarity and many potential shortcomings. All tools share their reliance on the use of a *generate-and-test* algorithm to find a suitable CR chain to achieve the desired effect, which does not scale to large binaries. All but exrop adopt the *bounded generate-and-test* algorithm, which comes with a dilemma: how to choose a proper bound N (maximum length of chains). If N is too large, the engine will suffer from state explosion [104]; if it is too small, it may fail to generate a chain. Beyond this similarity, they exhibit various differences, of which we subsequently review six.

First, all tools except Crackers [28] and Arcanist [7] support generating CR chains in byte forms, allowing security researchers to easily pass the payload to an exploit. In contrast, Crackers and Arcanist output a list of gadgets, relying on the user to fill in the correct data in a payload, which is non-trivial when complicated computation is involved.

Second, gadgets containing conditional branches are known to lead to nondeterministic executions. As a consequence, all but RiscyROP and Arcanist filter out gadgets containing conditional branches. Unfortunately, our empirical evaluation finds that RiscyROP's support leads to 33.3% of the chains being unable to guarantee deterministic execution results.

Third, multi-block gadgets: most works have been influenced by Shacham's seminal work [107] and the proposed Galileo Algorithm: they attempt to find reusable code sequences only within a single basic block (§5.1.1.2). This is unnecessarily restrictive,

as code sequences such as in Listing 4 can also be used in a chain. Currently, only RiscyROP implements support for gadgets spanning multiple basic blocks.

Fourth, at the time of writing, none of the existing gadget chaining tools supports utilizing return-to-csu style gadgets (*e.g.*, gadgets that end with `call [mem]`) without additional primitives. As we detailed before, this is because lists of gadgets alone are not enough to verify conditional chainable chains in the GAT algorithm. How to generate lists of gadgets together with feasible constraints remains unstudied.

Fifth, in real-world scenarios, working exploits often need to achieve more than just setting registers. A full CR chain usually involves invoking multiple function calls and system calls, *e.g.*, `dup(3, 0)` + `dup(3, 1)` + `execve("/bin/sh", 0, 0)` to get a shell on a web server. This requires the chain generation engine to be aware of the calling convention of both function calls and system calls, which is difficult to design for specific architectures and even more challenging across multiple architectures. Of all existing tools, only `exrop` supports chaining payloads and, thus, realistic workloads; albeit, its support is limited to x64. All other tools can only generate a payload for one function call and do not support chaining the payload of multiple calls together. As a consequence, a human analyst still has to process the chain manually before being able to use it.

Sixth, SGC, Crackers, and Arcanist use an SMT solver to search for chains, which is extremely slow when the search space is large (*i.e.*, too many gadgets). As a compromise, they **sample** a few hundred gadgets at a time and see whether the solver can construct a chain using these gadgets. As a result, this approach is nondeterministic in nature and may miss the chance of building a chain if its gadgets are not sampled in the same batch. In other words, they may miss some gadget combinations, thus not fully utilizing all gadgets by design.

5.2 Design

We now detail the design of ropbot and explain how it overcomes the aforementioned challenges and limitations.

At a high level, we first clearly define what a "Gadget" is, and we introduce the special groups of gadgets, called ROPBlock, that guarantee chainability. This ROPBlock will then be the actual building block of CR chains. We emphasize that this deviates from the common wisdom embodies that "ROP chains consist of gadgets".

5.2.1 Definitions

We first define the terms we use throughout this work. To the best of our knowledge, this makes our work the first to clearly define what a "gadget" is in the context of CR chain generation. This clear definition enables us to identify gadgets *without* heuristics and manually crafted "anchor instructions". Ultimately, this is also what led us to discover a new class of pervasive code reuse snippets.

Gadget. A gadget is a sequence of instructions that exists in the target binary, executes in order, and can potentially set the next PC to a fully attacker-controlled value.

It is noteworthy that we relax the constraint of setting the PC to take conditional branches into account. Relating to the terminology of symbolic execution, a gadget is defined as a code sequence that can move a fully symbolic value (*i.e.*, register, stack, and memory) into the PC.

There are a few advantages to this flexible definition:

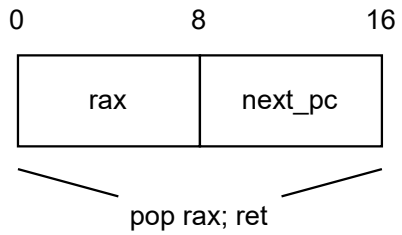

Figure 9. Stack Patch of a x64 Gadget.

- It is an architecture-*agnostic* definition that does not require architecture-specific domain knowledge.
- It allows us to find *all* instruction sequences that can be potentially used to an attacker's advantage to execute payload logic while maintaining the control flow.
- A gadget now can span multiple basic blocks instead of one block, as used in the Galileo Algorithm, leading to more usable gadgets.

One caveat is that under this definition, one address can potentially result in multiple gadgets due to conditional jumps.

Note that under this definition, any code snippet containing an indirect jump is potentially a gadget. This is interesting as dynamic linking itself is built on top of indirect jumps using the Procedure Linkage Table (PLT) and the Global Offset Table (GOT). Invoking any function in the PLT (*e.g.*, `call memset@plt`) will eventually lead to indirect jumps (*e.g.*, `jmp qword ptr [memset@got]`). As a result, any code snippet ending with a PLT function call is a gadget similar to ret2csu gadgets; we call them *GOT gadgets*. They are proven to be useful in practice for CR chain generation: they account for 9.6% of ropbot's success (§5.4). To the best of our knowledge, the current state of the art does not find, utilize, or acknowledge this class of gadgets.

Stack Patch. Studying code reuse attacks, we observe that they also consist of

"frames", akin to function stack frames. To distinguish them from function frames, we call them *stack patches* in the context of CR chains.

A stack patch is defined as the data on the stack, bounded by the stack pointer before and after executing the gadget. The change of the stack pointer before and after the gadget is referred to as *stack change*. As we can see in Figure 9, the gadget `pop rax; ret` is associated with a stack patch of 16 bytes (and the stack change is 16). During execution, it will pop the first 8 bytes into `rax` and then pop the second 8 bytes into PC, which enables gadget chaining.

This abstraction is architecture-agnostic and generic, making gadget chaining an easy task even across architectures, as we will show in Section 5.2.5.1.

Notice that due to our flexible definition of a "gadget", a gadget does not guarantee to have a positive stack change, contradicting the common understanding of gadgets. We address this by introducing a new concept called "self-contained".

Self-Contained. We say a gadget or a sequence of gadgets is self-contained if and only if it

- has a positive stack change (thus having a stack patch)
- takes the PC from the stack patch
- contains no conditional branches

In this definition, `pop rax; ret` is self-contained while `pop rbx; jmp rax` is not (it takes the PC from `rax`). `cmp rax, 1; jnz <label>; pop rbx; ret` is not self-contained either due to conditional branches.

In plain words, self-contained gadgets can be used in a CR chain by themselves, while non-self-contained gadgets need to be combined with other gadgets to be used in chains. For example, the gadget `cmp rax, 1; jnz <label>; pop rbx; ret` can be put into use if combined with a `pop rax; ret | 1` chain, so it will have the expected

and deterministic outcome. By itself, the gadget will lead to nondeterministic outcomes depending on the value of `rax` at runtime.

We emphasize that part of this property is an implicit requirement in many past works. For example, SGC, exrop, and Crackers filter out gadgets containing conditional branches. RiscyROP and exrop do not utilize gadgets ending with `call [mem]` or `jmp [mem]` because the PC does not come from the stack.

Rop Block. We define a new abstraction called ROPBlock as a sequence of gadgets (or just one gadget) that are self-contained. This ROPBlock is what we will use as the building block of CR chains. By clearly defining it, we are able to explicitly turn non-self-contained gadgets into ROPBlocks and utilize more gadgets.

Note that, by definition, a ROPBlock takes PC from the stack patch, a region controlled by attackers, thus guaranteed to be chainable with another ROPBlock. We will show that chainability constraints will be embedded into ROPBlocks themselves (§5.2.4.2), so the ROPBlocks are unconditionally chainable.

Chain. A CR chain is a sequence of ROPBlocks that work together to accomplish a semantically meaningful task (*e.g.*, invoke a function).

With these definitions in mind, we can focus on the design.

5.2.2 Workflow

At a high level, ropbot works by first collecting gadgets and analyzing gadget effects using `angr` [109], building a minimally functional chain builder and iteratively optimizing it, and generating CR chains as specified.

5.2.3 Gadget Collection

We collect gadgets in the first step.

5.2.3.1 Gadget Identification

ropbot exhaustively analyzes all executable addresses to find all potential gadgets. Given an executable address, ropbot will symbolically execute a fully symbolized state (symbolized register, stack, and memory) starting from that address and check whether the code moves any symbolic value into PC. If so, it will mark the starting address with that execution history as a gadget (note that one starting address may lead to different execution histories). This allows us to find *all* gadgets without any domain knowledge or exhaustively searching for "anchor instructions", which may vary after changes in compilers.

We also use a cache mechanism to speed up the process. Essentially, we only analyze position-independent gadgets once and cache them. To determine whether a gadget is position-independent, we move the bytes into another address and see whether constants in the gadget change. For example, `pop rax; ret` will be considered a position-independent gadget because nothing changes if we move the bytes to another address. However, `lea rax, [rip+0x100]; jmp rax` will be considered a position-*dependent* gadget because `[rip+0x100]` will be resolved into another constant when the gadget is placed at a different address. Thus, `lea rax, [rip+0x100]; jmp rax` will not be cached, allowing us to reach different code regions.

5.2.3.2 Gadget Effect Analysis

For each found gadget, we create a fully symbolized state (symbolized registers, stack, and memory) and execute the gadget. By tracking the symbolic execution history, we can track the memory accesses and conditional branches information about the gadgets. By comparing the symbolic states before and after executing the gadget, we can also obtain the gadget's effects on registers.

Specifically, `stack_change` is calculated by subtracting the stack pointers, which is important for building stack patches later. We also track the register popping, moving, and changing behaviors. Notice that we also track it when registers are set to concrete values, such as in `xor rdx, rdx; ret`, `rdx` is set to 0. This is important for chain generation for tasks that want to set `rdx` to 0, such as `execve("/bin/sh", NULL, NULL)`.

Notice that all these "popping", "moving", "changing" actions are gadget effects, not instructions, thus architecture-agnostic. Although `push rax; pop rbx; ret` involves only `push` and `pop`, it will be considered a register move from `rax` to `rbx`. Also, "register pop" here means that the register values come from the stack (*e.g.*, `mov rax, [rsp+8]; add rsp, 0x10; ret`), the gadget does not need to have a "pop" instruction.

5.2.3.3 Gadget Categorization

Based on the control flow transition types, we categorize gadgets into three types:

pop_pc. The next PC value comes from the stack. *e.g.*, `ret` or `ldr lr,[sp],#4;add sp,#8;bx lr`. Notice that all self-contained gadgets are `pop_pc` gadgets by definition (take PC from stack patch).

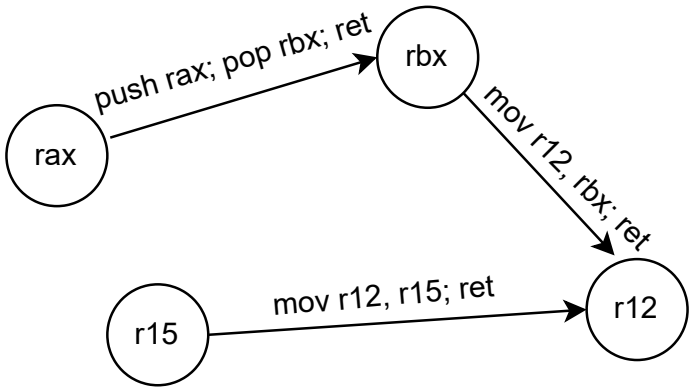

Figure 10. Register Moving Graph.

jmp_reg. The next PC value comes from another register. *e.g.*, `jmp rax`.

jmp_mem. The next PC value comes from a specific memory location (*e.g.*, `call [rax+8*r12]` in ret2csu). By definition, ret2csu and GOT gadgets are all `jmp_mem` gadgets.

Notice that we categorize gadgets by effects, not by instructions. Even though a gadget such as `pop rax; jmp rax` ends with a jump to registers, it is considered as pop_pc in ropbot, because at the end of the execution, the PC comes from a stack value.

5.2.4 Iterative Graph Optimization

In this step, we iteratively improve our chain builder's capability in register setting by utilizing a Register Moving Graph and the Gadget Normalization procedure.

Our chain builder only works with ROPBlock, so we first bootstrap and create a minimally functional chain builder with only self-contained gadgets (they are

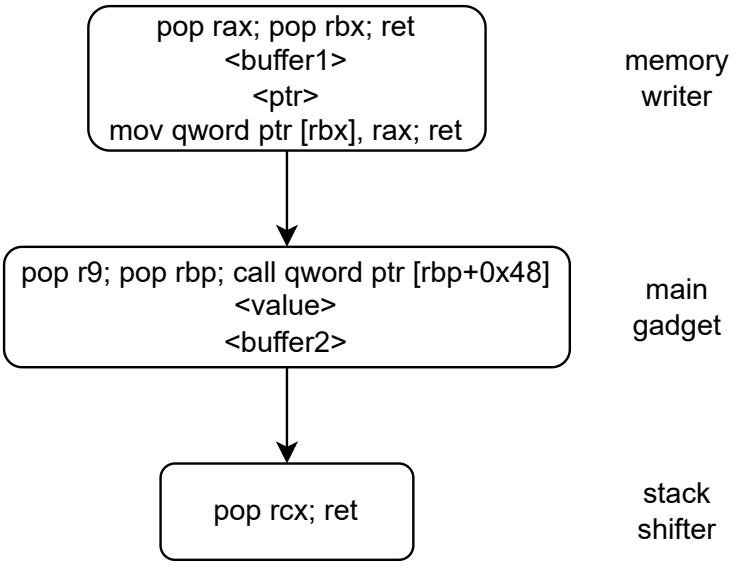

Figure 11. How ropbot normalizes a `jmp_mem` gadget.

ROPBlocks by definition). This allows us to perform simple tasks such as setting registers or writing to memory using the same algorithm described in Section 5.2.5.

5.2.4.1 Register Moving Graph

We then build a directed graph modelling the register moving effect. As shown in Figure 10, we consider each register a node in the graph. We add an edge (Reg A, Reg B) if and only if there are ROPBlocks that move register A to register B. By using this graph, we can decide whether there exists a chain that can move any register to another. For example, in Figure 10, the chain builder can tell there is a chain `push rax; pop rbx; ret | mov r12, rbx; ret` that moves `rax` into `r12`. Note that edges are associated with ROPBlock, so they can be easily chained together into a new ROPBlock (§5.2.5.1).

5.2.4.2 Gadget Normalization

Then, we check whether any non-self-contained gadget provides unique capabilities (*i.e.*, sets a register that all existing ROPBlocks cannot or performs a unique register move); if yes, we "normalize" them into ROPBlock for later use. Here, the goal is to group the non-self-contained gadget with other ROPBlocks into a new ROPBlock while maintaining its unique capability.

Here, the gadget normalization procedure is as follows:

- if a gadget is a `jmp_reg` or `jmp_mem` gadget, prepend it with a chain (in fact, ROPBlock) that can set the target register or write to the memory. (*e.g.*, prepend `pop rax; ret` to `pop rbx; jmp rax` to gain the capability of setting `rbx`)
- if a gadget has a non-positive stack change or takes `PC` outside its stack patch, chain it with a stack shifting gadget (*e.g.*, chain `pop rax; call rax` with `add rsp, 8; ret`).
- if a gadget contains conditional branches, prepend it with a ROPBlock that sets the correct constraints.

The result of the procedure described above is a list of gadgets and corresponding constraints (*e.g.*, where to write when normalizing `jmp_mem` gadgets). We turn the list of gadgets into a ROPBlock using symbolic execution. Specifically, we first create a fully symbolized state and symbolically execute the first gadget. When the execution finishes, the state will be at an "unconstrained" state because the PC comes from a user-controlled data (by the definition of "gadget"). Note that the gadget normalization process ensures that all the PC values can only come from the stack (if not, it will be prepended with a chain to set the register/memory, etc). Then we can constrain the

state PC to be the next gadget's address, which is essentially setting the correct stack data to the address. During symbolic execution, we will apply the chain constraints so that the execution will only follow the chainable state. For example, when `angr` sees a write to a symbolic address, we will concretize the variable using values from the provided constraints. By finishing symbolically executing the list of gadgets, we obtain a ROPBlock together with its stack patch. After this process, we also perform the effect analysis on the generated ROPBlock just like what we do with the gadget effect. As a result, if there are any conflict constraints in the ROPBlock, it will be captured during symbolic execution and reflected in the ROPBlock effect result.

Note that the aforementioned symbolic execution step essentially embeds the constraints into each ROPBlock so that it can be chained independently from other ROPBlocks, which makes sure ROPBlocks are unconditionally chainable. For example, we can normalize `cmp rax, 1; jne <label>; pop rbx; ret` into `pop rax; ret | 1 | cmp rax, 1;jne <label>; pop rbx; ret`, which explicitly satisfies the constraint of `rax == 1` so that the whole ROPBlock can execute correctly and deterministically without the help of other gadgets.

In Figure 11, we show how ropbot normalizes a `pop r9; pop rbp; call [rbp+0x48]` gadget to gain the unique capability of setting `r9` while maintaining the control-flow. We first prepend the main gadget with a memory writing chain and write the address of the stack shifter there. Now we symbolically step through the main gadget, which sets `r9` to `<value>`, sets `rbp` to `buffer2`, and then invokes `[rbp+0x48]`. Here, ropbot will notice the symbolic read of PC and redirect it to the previous symbolic write, adding the constraint of `buffer1==buffer2+0x48`. Finally, the chain will execute the stack shifter, clean up the return address pushed onto the stack, and move fresh unconstrained stack data into the PC, thus maintaining

the control-flow. After the normalization process, the whole sequence becomes a ROPBlock that has the ability to set `r9`.

By using the gadget normalization procedure, we turn gadgets that are hard to work with but provide unique capabilities into ROPBlock so that they are usable by our chain builder. However, not all gadgets can be normalized if they require gadget setting capabilities that our chain builder does not currently possess.

5.2.4.3 Graph Optimization

Then we utilize the Register Moving Graph to gain capabilities in setting new registers.

Basically, we use the graph to check whether there is any path from a register that we can set (*e.g.*, Reg A) to a register we cannot set (*e.g.*, Reg B). If such a path exists, by combining the ROPBlock that sets Reg A and the list of ROPBlocks that move Reg A to Reg B, we obtain a new ROPBlock that can set Reg B.

As such, new edges in the Register Moving Graph may lead to new register setting capability; new register setting capability may enable the normalization of gadgets that provide unique capabilities. So, we perform the optimization iteratively until we can no longer normalize any gadget that provides unique capabilities.

5.2.5 Chain Generation

We split the chain building task into sub-tasks: Register Setting, Register Moving, Function Invocation (Syscall Invocation), Memory Writing, and Stack Shifting.

Among them, Memory Writing and Stack Shifting are simple. Memory Writing

involves choosing a ROPBlock that performs a memory write at a symbolic location. By correctly setting the registers using Register Setting capability, we can ensure that when chained with the register setting chain, the ROPBlock can perform a controlled write at the desired address. Stack Shifting is simply shifting the SP without clobbering the existing state by choosing a ROPBlock that has a wanted stack change value. This module is needed for normalizing gadgets for stack cleaning as demonstrated in Figure 11.

Register Moving can be performed by using the Register Moving Graph built during graph optimization.

5.2.5.1 Payload Chaining

ropbot's Chain Builder only works with ROPBlock. By definition, a ROPBlock has positive stack changes, thus having a stack patch.

The abstraction of the stack patch makes payload chaining naturally an easy task. As shown in Figure 12, it consists of two steps: 1. concatenate the stack patches of two blocks, 2. fix up the `next_pc` value in stack patch to chain the two ROPBlock. This abstraction makes it easy to generate CR chains that are not linear.

One notable property of this payload chaining process is that the chaining result is also a ROPBlock, making it easy for further manipulation.

Note that in this model, Return-Oriented-Programming on x86 becomes a special case where all gadgets/ROPBlock have `next_pc` values in the last position of their stack patches (because `ret` moves the last value in the stack patch to PC).

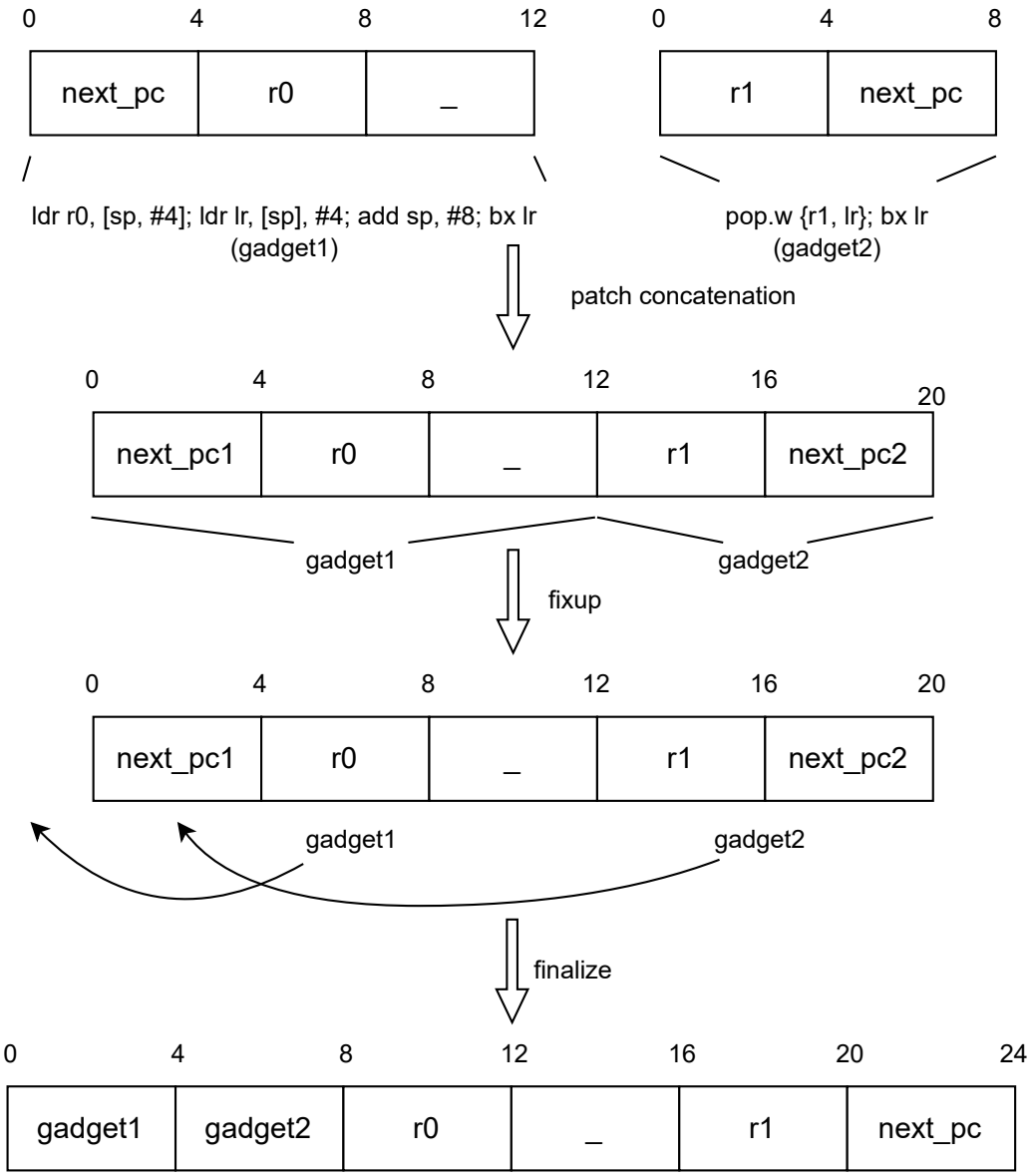

Figure 12. The Process of Chaining Two ROPBlocks.

5.2.5.2 Register Setting

Setting registers is the core challenge of CR chain generation.

In our work, we consider three ways of setting registers:

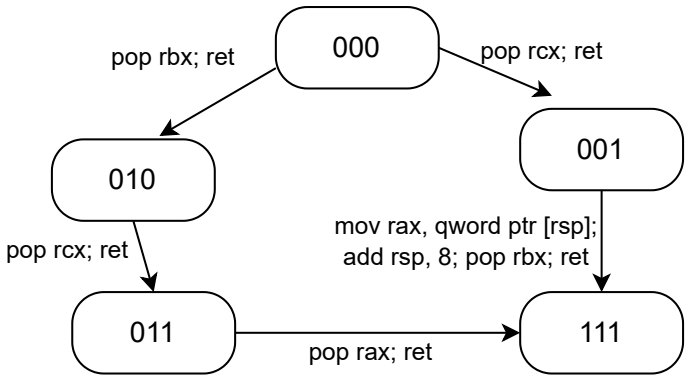

Figure 13. Register Setting Graph of Setting {rax,rbx,rcx}.

- register pop. This is the most common way of setting registers, *e.g.*, `pop rax; ret`. In our framework, we consider all memory loading from the stack "register pop", including `mov rbx, [rsp+0x10]`.
- register moves. It is common that not all registers can be set through register popping due to the lack of gadgets or constraints on inputs (*e.g.*, bad bytes). In those cases, we can set the value in one register using register pop and then move it to the target register. Gadgets with unique register moving capabilities will be turned into register pop ROPBlocks during graph optimization.
- concrete values. In cases where a value cannot be set through register popping at all, we may resort to using existing concrete values in gadgets (*e.g.*, use `xor rax, rax; ret` to set 0 when `pop rax; ret` does not exist).

The core idea of the solution is to build a directed graph to represent the program state transition and find a path that can set all the requested registers. During Iterative Graph Optimization (§5.2.4), we gain the ability to set each individual register (in the form of register pops); we now need to find the shortest chain for a given set of registers with no conflict.

With all the ROPBlock, we build a directed graph where each node represents

the state of register control. Specifically, when a user wants to set `rax, rbx, rcx`, each node is a 3-boolean tuple that represents whether each register is already in our control. For example, `(True, False, True)` (written as 101), in the order of rax, rbx, rcx, is one of the nodes that means rax and rcx are in our control (can be popped), while rbx is not. Each edge means there is at least one ROPBlock that transitions from one state to another. Register pop ROPBlocks can clearly lead to state transitions, thus added as edges in the graph. Besides them, we also check ROPBlock with concrete value effects. For example, if the request is to set `rdx` to 0, this step will add `xor rdx, rdx; ret` as an edge because this ROPBlock can lead to the correct register control. As a result, our algorithm can utilize all three ways of setting registers (register move gadget will be turned to register pop ROPBlock during graph optimization) in a uniform way.

During the graph-building process, we model all ROPBlock precisely. Besides their register pop effects and concrete values, we also check whether they clobber any wanted registers. For example, the `pop rcx; xchg rax, rsi; ret` ROPBlock sets `rcx` but clobbers `rax`. So it will add an edge 100-001 (in the order of rax, rbx, rcx) to the graph (besides other edges such as 000-001).

Now, the task of finding a chain that sets all the registers becomes finding a path from the Node representing no register control to the Node representing full register control. In the above example, it means finding a path from Node 000 to Node 111, which is visualized in Figure 13.

This graph takes $\mathcal{O}(n)$ to build, where n is the number of ROPBlocks, because we need to go through all ROPBlocks and add corresponding edges if needed. And it takes $\mathcal{O}(2^{2k})$ to solve, where k is the number of registers to set: let V be the number of nodes in the graph, then $V = 2^k$. To solve the shortest path problem in the graph

takes $\mathcal{O}(V^2)$, which is $\mathcal{O}(2^{2k})$. Since k is small and constant in practice, $\mathcal{O}(2^{2k})$ is negligible. As a result, the problem of register setting takes $\mathcal{O}(n)$ to solve using this graph search.

5.2.5.3 Function/Syscall Invocation

The goal of CR chains is to invoke functions or system calls such as `mprotect` (inject shellcode) or `execve` (execute "/bin/sh"). But this task is non-trivial across architectures due to the variety of calling conventions out there.

Here, we design a calling-convention-agnostic algorithm to handle function and syscall invocation by introducing Invocation gadget, a gadget that invokes a function or syscall. Essentially, the Invocation gadget is a pseudo-gadget that has different gadget effects under different architectures and calling convention configurations.

For function invocation, the Invocation gadget is a pseudo-gadget that points to the target function's address. Notice that, depending on the architecture and calling convention, an Invocation gadget may or may not be self-contained. For example, on x64, a function Invocation gadget is as simple as the address of the function itself with a stack change of 8, and it is self-contained (functions end with `ret`, which is equivalent to `pop pc`). But on AArch64, it is no longer self-contained because its stack change is zero and functions end with `ret`, which is equivalent to `jmp x30`. This means that the function Invocation pseudo-gadget is a "jmp_reg" gadget. For syscalls, the invocation gadget is just a normal gadgets that invoke syscalls (*e.g.*, `syscall; ret`). Note that we identify system call invocation by checking the basic block's jump kind (syscalls are specially marked as `Ijk_Sys` in Valgrind [85]'s vex

library, which is used internally by angr). So we do not rely on architecture-specific instruction identification.

With the Invocation gadget, we leverage the calling convention analysis in angr to map function arguments passed by users to registers and stack data. We can pass stack arguments as we control the stack. And we utilize the previously described Register Setting capability to set register arguments. Then, we append the Invocation gadget. Finally, we normalize the gadgets to make the whole chain a ROPBlock. For example, on AArch64, a function Invocation gadget is a `jmp_reg` gadget, so we normalize it by setting `x30` before setting arguments and invoking the function. We do this for all calling conventions that return to a register. On x86, function Invocation needs to clean up the arguments on the stack, which can be done by setting the `next_pc` to a stack-shifting gadget. We infer the need for it by checking the number of stack arguments.

Note that the effects of Invocation gadgets are inferred from angr's calling convention analysis (angr can analyze function and syscall calling conventions based on the architecture and operating systems), no manual effort is needed to craft the Invocation gadget on a per-architecture basis.

5.2.6 Scalability

Although ropbot uses symbolic execution in its design, we limit its use to simple tasks (*i.e.*, gadget effect analysis and turning lists of gadgets into ROPBlock). We do not utilize it for exploration, thus avoiding the notorious state explosion problem. This is evidenced by the fact that ropbot works for Chromium (one of our x64 binaries) and the Linux kernel (§5.3.5), two of the largest binaries.

Table 13. Evaluation results for all CR chain generation tools on x64 binaries.

Tool	Success	Generated	F.P.	G.T.	Opt-Time	Chain-Time	Timeout	Total Time
facefeed (total: 1022 binaries)								
ropbot	**635**	**635**	**0%**	11.5s	6.9s	**0.4s**	**0.0%**	**5.3h**
exrop	357	358	0.3%	**9.8s**	-	72.8s	4.1%	23.4h
SGC	616	633	3.8%	225.3s	-	312.6s	14.8%	130.9h
Crackers	-	353*	-	24.4s	-	237.7s	3.9%	72.4h
Arcanist	-	416	-	180.8s	-	457.2s	20.2%	167.4h
mmap (total: 64 binaries)								
ropbot	**21**	**21**	**0%**	**25.3s**	11.4s	**1.2s**	**0.0%**	**0.7h**
exrop	4	4	0%	56.7s	-	199.9s	10.9%	4.5h
SGC	0	0	-	932.1s	-	240.3s	28.1%	17.5h
Crackers	-	0	-	179.8s	-	1215.4s	62.5%	22.1h
Arcanist	-	0	-	903.0s	-	1453.8s	79.7%	28.1h
execve† (total: 272 binaries)								
ropbot	**244**	**244**	**0%**	**28.4s**	11.9s	**2.5s**	**0.0%**	**3.2h**
exrop	90	117	23.1%	33.6s	-	240.5s	13.6%	20.9h
SGC	27	27	1.6%	763.1s	-	1527.8s	80.9%	123.0h
Crackers	-	0	-	83.6s	-	695.9s	21.0%	55.2h
fullchain (total: 37 binaries)								
ropbot	**37**	**37**	**0%**	**81.8s**	12.9s	**3.7s**	**0.0%**	**1.0h**
exrop	21	22	4.5%	152.3s	-	92.7s	5.4%	2.5h

G.T.: Gadget finding time. F.P.: False positives.

* the number differs from Crackers's paper because it was not evaluated under this strict configuration. Their original configuration assumes attackers control all registers and memory, which is not realistic.

† Arcanist does not support memory writing, thus excluded from the `execve` evaluation, which requires writing "/bin/sh" into memory.

In this section, we evaluate ropbot extensively against all existing open-source CR chain generation tools to show its effectiveness. We assume attackers do not control the initial program state except for the stack. Although this configuration is strict, we think it is reasonable and realistic.

During our evaluation, we noticed that some of the chains generated by existing

tools are incorrect, which is also mentioned by previous works [104, 106]. They may lead to crashes by accessing memory at an unconstrained address. Or they may lead to nondeterministic execution results due to the use of gadgets with conditional jumps without satisfying the jump constraints. As such, we developed a symbolic verification framework that is directly built upon angr. Specifically, this framework simulates a control-flow hijacking scenario and loads the bytes produced by each tool into the symbolic state and starts execution. It considers a chain correct only if the execution is deterministic, all memory accesses are constrained to mapped regions, and the chain achieves the expected outcome (*e.g.*, execute an `execve` syscall).

Notice that Crackers and Arcanist cannot generate CR chains in byte form, so we are unable to verify their generated chains. SGC and Arcanist assume knowing the address of the payload on stack, and there is no option to turn it off. Since all other tools do not assume knowing the address of the payload, this may give SGC and Arcanist an unfair advantage.

All the evaluations were performed on a 40-core machine with `Intel(R) Xeon(R) CPU E5-2670 v2 @ 2.50GHz` CPU. All tools are configured to fully utilize the 40 cores. Following Crackers's setup, we also set a timeout of 30 minutes for each target.

5.3.1 x64 Evaluation

We first evaluate ropbot against all existing tools on x64, the most tested architecture. The result can be found in Table 13. As shown in the table, ropbot not only outperforms all tools in terms of the success rate in all tasks, but also the speed.

dataset. The dataset consists of 1016 random ALLSTAR [111] binaries used in

Crackers and 6 binaries used in SGC. In total, we obtain 1022 binaries of diverse sizes and functionalities.

task1: facefeed. The first task is to invoke a function with three arguments: `0xfacefeed(0xdeadbeef, 0x40, 0x7b)`, which is the evaluation task used by Crackers. We use this task as a baseline to test each CR chain generation tool's functionality. We perform this task on all 1022 binaries.

task2: mmap. The second task is to invoke the `mmap` function with six arguments: `mmap(0x41414000`, `0x1000`, `PROT_READ |PROT_WRITE |PROT_EXEC`, `MAP_ANON |MAP_PRIVATE|MAP_FIXED`, `-1`, `0)`, which is an evaluation task used by SGC. We use this task to stress each tool's ability to generate chains that set multiple registers. We think this chain is realistic because one can use this function call to inject shellcode (combined with memory write capability) and obtain arbitrary code execution. We perform this task on binaries containing `mmap` plt function, which leads to a total of 64 binaries.

task3: execve. The third task is to invoke the `execve("/bin/sh"`, `NULL`, `NULL)` system call. It checks each tool's ability to invoke system calls and memory writing capability (binaries usually do not contain the "/bin/sh" string). We perform this task on all binaries with at least one "syscall" instruction, which results in 272 binaries.

task4: fullchain. The fourth task is to invoke a full chain: `dup2(3,0)` + `dup2(3,1)` + `execve("/bin/sh", NULL, NULL)`. This CR chain is commonly used in exploiting servers, known as the socket reuse payload [128]. We want to use it to test each tool's ability to generate realistic CR chains. We run this test on binaries containing both `dup2` function call and "syscall" instructions, resulting in 37 binaries.

5.3.1.1 Result

As shown in the table, exrop and Crackers are fast in finding gadgets. This is because they both use static analysis to identify potential gadgets first, following the Galileo Algorithm (exrop uses ROPgadget internally) and use either symbolic execution or SMT solvers to analyze the gadgets. As mentioned in Section 5.1.1.2, this approach misses gadgets (*e.g.*, gadgets spanning multiple basic blocks) by design, which significantly affects their ability to set registers as shown in the `facefeed` task. In our Ablation Study (§5.4), we show that gadgets spanning multiple basic blocks are necessary for generating the `facefeed` chain in 51 binaries. ropbot is fast in gadget finding because of the position-dependent gadget caching mechanism (§5.2.3.1).

SGC performs exceptionally well in the `facefeed` task. This is because of two reasons. First, SGC generates multiple chains at once by design, while all other tools stop at the first chain. We consider one test a "Success" if any of the chains generated by SGC for the binary passes verification. Second, SGC has an unfair advantage of knowing where the payload is on stack so that it can utilize gadgets like `call [rbx]` by concretizing `rbx` to the payload address and place 0xfacefeed in the payload. ropbot can utilize these gadgets as well, but requires writing to the location first. In fact, 94.1% chains generated by SGC require knowing the address of the payload.

"False Positives" for SGC is calculated for all the chains it generates. "Success" means whether it generates at least one chain that passes verification, and "Generated" means whether it can generate chains for the target binary. So even though "Success == Generated" for execve in the table, its false positive rate is non-zero because it generates invalid chains besides the valid ones.

The false positives in SGC are caused by the incorrect concretization of pointers.

More specifically, when SGC puts a pointer on the stack, it may incorrectly concretize the upper bytes to `0xff`, making it an invalid pointer. exrop generates false chains due to its incorrect modelling of gadgets. Internally, exrop uses ROPgadget to find gadget candidates, which sometimes list a sequence of instructions that end with a constant jump (*e.g.*, `pop rax; jmp <label>`, which may not be a gadget in this paper's definition depending on where it jumps to). But exrop treats the constant jump the same way as a `ret`, failing to model the destination basic block, thus leading to crashes.

Table 14. Evaluation result of the `0xdeadbeef(0x40404040,0x41414141,0x42424242)` chain.

Tool	Success	Generated	F.P.	G.T.	Opt-Time	Chain-Time	Timeout	Total Time
AArch64 (total: 172 binaries)								
ropbot	**53**	**53**	**0%**	**237.4s**	89.9s	**0.5s**	**2.9%**	**13.9h**
RiscyROP	9	17	47.1%	833.1s	-	176.6s	28.5%	43.4h
RISC-V (total: 1000 binaries)								
ropbot	**362**	**362**	**0%**	**192.7s**	133.3	**0.5s**	**3.8%**	**77.1h**
RiscyROP	77	112	31.3%	933.3s	-	485.8s	46.6%	316.2h

G.T.: Gadget finding time. F.P.: False positives.

5.3.2 Extensibility Evaluation

To demonstrate the extensibility of ropbot and evaluate against RiscyROP, we added support for RISC-V. The whole development process only took us two hours. It involves 12 lines of changes to ropbot, 88 lines to `angr`, and 152 lines to `archinfo`. The changes were mostly to fix the wrong RISC-V specification used by `angr` and `archinfo`. The 12 lines of changes to ropbot are trivial changes, only for declaring RISC-V support. We attribute the ease of extensibility to ropbot's architecture-agnostic design.

Arch	Success	Total	Rate	F.P.	G.T.	Opt-Time	Chain-Time	Timeout	Total Time
x64	635	1022	62.1%	0%	11.5s	6.9s	0.4s	0.0%	5.3h
MIPS	285	1000	28.5%	0%	11.2s	24.3s	0.8s	0.1%	10.1h
AArch64	58	172	33.7%	0%	24.3s	59.6s	0.6s	1.2%	3.9h
ARM	660	1000	66.0%	0%	6.0s	8.1s	0.6s	0.0%	4.1h
RISC-V	358	1000	35.8%	0%	34.7s	60.5s	0.5s	1.1%	24.1h

Table 15. Evaluation result of the `0xfacefeed(0xdeadbeef,0x40,0x7b)` chain. We filter out PAC-enabled AArch64 binaries, leading to a smaller number compared to other architectures.

5.3.3 Non-x64 Evaluation

In this section, we evaluate ropbot's capability on non-x64 architectures. Although Arcanist supports ARM, it is only configured for a few specific CVE targets, not ready for generic evaluation on ARM. RiscyROP is the only tool that supports non-x64 architectures and is ready for generic evaluation, which works for AArch64 and RISC-V.

To fairly evaluate ropbot against RiscyROP, we perform the evaluation with the CR chain generation task hardcoded in RiscyROP, which is to invoke `0xdeadbeef(0x40404040, 0x41414141, 0x42424242)` on both architectures. And we enabled conditional-branch gadget support for ropbot, which is off by default due to the fact that they significantly slow down gadget finding and chain building (§5.4.2).

dataset. Since we are not aware of any large dataset of AArch64 and RISC-V binaries, we sampled 1000 binaries from Ubuntu 24.04 for both architectures. During our evaluation, we noticed that both ropbot and RiscyROP reported extremely low success rates for AArch64. After our investigation, we found out that most of the Ubuntu binaries on AArch64 have pointer authentication (PAC) enabled. Specifically, they

verify the return address at each function epilogue before issuing the `ret` instruction. As a result, there are not enough gadgets usable for both tools, leading to failures.

To correctly compare ropbot's and RiscyROP's capability, we filter out binaries with PAC enabled on AArch64, ending up with 172 AArch64 and 1000 RISC-V binaries.

5.3.3.1 result.

As shown in Table 14, ropbot achieves nearly 5X success compared to RiscyROP under the same configuration. The result also reveals RiscyROP's high false positive rates. It comes from the improper use of gadgets with conditional branches. The way RiscyROP works is by finding a sequence of gadgets that may potentially work, then symbolically executing through them for verification. It focuses on finding a state that satisfies the expected outcome and does not reject chains that lead to nondeterministic results.

5.3.4 Multi-Arch Support

Besides evaluating against all existing works, we also performed the `facefeed` test on all supported architectures to investigate the impact of architecture designs on CR chain generation.

The result can be found in Table 15. It supports the claim that ropbot can generate CR chains across architectures. But its performance is significantly better on x64 and ARM compared to other architectures. We investigated this issue and initially thought it was due to the diversity of gadgets. Indeed, our analysis on gadget

```
1 p = b""
2 p += p64(0xffffffff822a76a1)# xor edi, edi;
3                             # jmp 0xffffffff822a7e60;ret
4 p += p64(0xffffffff8114d660)# prepare_kernel_cred
5 p += p64(0xffffffff8196eeeb)# pop rdx; pop rdi;
6                             # jmp 0xffffffff822a7e60; ret
7 p += p64(0xffffffff82504104)# lea rsp,[rsp+0x8]; ret
8 p += p64(0xffffffff83600198)# <buffer>
9 p += p64(0xffffffff82211ea2)# mov qword ptr [rdi], rdx;
10                            # xor edx, edx; xor edi, edi;
11                            # jmp 0xffffffff822a7e60; ret
12 p += p64(0xffffffff8240191a)# pop rbp;
13                            # jmp 0xffffffff822a7e60; ret
14 p += p64(0xffffffff83600150)# <buffer> - 0x48
15 p += p64(0xffffffff81aca5d2)# push rax; pop rdi;
16                            # call qword ptr [rbp + 0x48]
17 p += p64(0xffffffff8114d3a0)# commit_creds
```

Listing 5. A `commit_creds(prepare_kernel_cred( NULL))` chain generated by ropbot.

diversity showed that x64 binaries have 711 self-contained gadgets on average, and more than 90% of the binaries include gadgets that can pop `rdi`, `rsi`, 32.8% can pop `rdx`. On other architectures, they have around 200 self-contained gadgets on average, and 2%-20% binaries include gadgets that can pop the target registers. This result explains why ropbot performs worse on MIPS/AArch64/RISC-V, but fails to explain why it does well on ARM. We inspected the chain ropbot generated for ARM and immediately noticed a pattern of using two gadgets: `pop {r3,r4,r5,r6,r7,r8,r9,pc}` and `mov r0,r7;mov r1,r8;mov r2,r9;blx r3`. These two gadgets alone are enough to generate the `facefeed` chain used in our evaluation, and they are pervasive in the target binaries. It turns out they are the equivalent of the ret2csu gadgets on ARM. Due to the architecture features, they are compiled into `jmp_reg` gadgets, which makes them even more powerful for code reuse attacks.

5.3.5 Case Study - Linux Kernel

We evaluate ropbot's capability of chain building on large binaries by trying to generate the `commit_creds(prepare_kernel_cred(NULL))` chain for the Linux kernel. While it is commonly seen in Linux kernel exploitation, it is not an easy task due to the lack of `mov rdi, rax; ret` gadget. Security researchers have resorted to finding sophisticated chaining methods to accomplish the same effect such as using `push rax; jmp [rsi] | pop rdi;| ret` [37] and `pop rdi; ret | or rdi, rax; ret` [38]. These chains are time-consuming to craft due to the need to explore all possible ways of moving `rdi` to `rax`. In our experiment, ropbot can craft a similar chain in one second (with cached gadget analysis results, which takes 7 minutes to build), shown in Listing 5. In the chain, ropbot writes a `lea rsp, [rsp+8]; ret` gadget into a buffer and then invokes it using `push rax; pop rdi; call [rbp+0x48]` to achieve `mov rdi, rax` while maintaining the control flow. This is similar to what human researchers did in CVE-2023-3390 [37].

Due to the Retpoline protection [123] adopted in the Linux kernel, all functions end with `jmp indirect_thunk`, which in turn executes `ret`. In Listing 5, this is `jmp 0xffffffff822a7e60; ret`, where `0xffffffff822a7e60` is the `indirect_thunk`. As a result, many existing gadget finders [101, 103] will misclassify these gadgets as gadgets ending with `jmp`, while in reality, they are equivalent to ending with `ret`. This demonstrates the importance of gadget effect analysis used in ropbot instead of instruction-based analysis and the need to analyze gadgets spanning multiple basic blocks.

5.4 Ablation Study

In this section, we design a series of experiments to investigate what leads to ropbot's unparalleled performance in all the evaluations. The result can be found in Table 16.

5.4.1 Graph Search vs Bounded Generate-and-Test

In our evaluation, we notice that even the base version of ropbot outperforms all other tools in the `execve` task. The base version of ropbot only uses single-block self-contained gadgets and does not perform graph optimization. It is supposed to perform worse than other tools because other tools do use `jmp_reg` gadgets that are not self-contained. This expectation matches the result of the `facefeed` evaluation but not the `execve` one.

We investigated the `execve` evaluation and concluded that this is because `execve` chains are usually long (require performing a memory write, setting four registers, and invoking the syscall gadget), which stresses the limit of bounded generate-and-test (BGAT). On one hand, they are at the risk of encountering state explosion when they are sampling longer lists of gadgets, leading to timeouts. As evidenced by their much higher timeout rate compared to the `facefeed` experiment. On the other hand, they are not able to find chains longer than their bound by design, leading to lower success rates. For example, SGC allows at most 16 controlled words on stack, while 53.5% `execve` chains generated by ropbot are longer than 16 words.

As shown in Table 16, even only working with single-block self-contained gadgets, graph optimization alone can boost the success rate by around 8%, surpassing the

Config	Task	Success	Total	Rate
Base	execve	193	272	71.0%
Base+Graph_Opt	execve	215	272	79.0%
exrop	execve	90	272	33.1%
Crackers	execve	0	272	0%
SGC	execve	27	272	9.9%
Base	facefeed	319	1022	31.2%
Base+Graph_Opt	facefeed	391	1022	38.3%
Crackers	facefeed	353	1022	34.5%
exrop	facefeed	358	1022	35.0%
Full-got_gadgets	facefeed	574	1022	56.2%
Full-ret2csu_gadgets	facefeed	570	1022	55.8%
Full-multi_bb_gadgets	facefeed	570	1022	55.8%
Full-jmp_mem_gadgets	facefeed	457	1022	44.7%
Full	facefeed	635	1022	62.1%
Full	facefeed	55	172	32.0%
Full+cond_br_gadgets	facefeed	56	172	32.6%

Table 16. Evaluation on ropbot with different configurations. Base is ropbot with only single-block self-contained gadgets and does not perform graph optimization.

success rate of BGAT tools except for SGC (it uses an extra condition of knowing where the payload is). This is because graph optimization allows ropbot to find a complicated chain that provides unique effects and use it as one ROPBlock. For example, ropbot can chain `pop rax;ret | mov rbx,rax;ret | mov rdx,rbx;ret` to set `rdx`. This chain is already hard for BGAT tools, let alone if the ability to set `rdx` is needed twice in the chain generation task.

5.4.2 New Gadget Categories

We design a series of experiments to measure the impact of each newly supported gadget category on the evaluation results. Each experiment is performed by removing

a specific gadget category and checking whether ropbot can still generate a CR chain. As shown in Table 16, each gadget type contributes uniquely to ropbot's success rate. Note that GOT gadgets contribute 5.9% and ret2csu gadgets contribute 6.3% to the success rate. But when they are combined, `jmp_mem` gadgets contribute 17.4% to the success rate. This suggests that there are 5.2% cases where CR chains can be generated with either GOT gadgets or ret2csu gadgets.

To evaluate the impact of gadgets with conditional branches, we design an additional evaluation on the 172 AArch64 binaries with a 30-minute chain generation timeout, with and without conditional branches. We did not perform the evaluation on the x64 binaries due to the fact that they have abundant self-contained gadgets, so ropbot prefers not to use gadgets with conditional branches because normalizing them takes up a considerable amount of payload space, which defeats the purpose of the evaluation. Our result shows that utilizing gadgets with conditional branches hurts the overall performance. This is because handling gadgets with conditional branches is slow compared to other gadgets due to state forking, leading to more timeouts, as evidenced by the elevated timeout rate (from 1.2% to 4.1%) and gadget finding time (from 32.3s to 237.6s). We also manually checked the one failed case and confirmed that it failed because of timeouts. By giving ropbot an infinite timeout, it can generate five more (9% improvement) CR chains with the help of gadgets with conditional jumps. Overall speaking, although gadgets with conditional branches contribute unique capabilities to the chain generation engine, the timeout outweighs their contribution.

5.5 Discussion

Although ropbot can tackle many of the CR chain generation tasks, it does not completely solve them. In this section, we discuss future works that can improve ropbot towards solving the code reuse attack synthesis problem in general.

More Gadget Types. ropbot already supports a wide range of gadget types, such as gadgets with conditional branches, GOT gadgets, and more. But there are other gadget types that can assist CR chain generation. For example, some of the failed chains in our evaluation can be made possible by using gadgets that can load registers from a symbolic address (*e.g.*, `mov rax, [rbx]; ret`). These gadgets can be put into use if we prepend them with a chain that writes the wanted value to memory and sets `rbx` to the correct address. Currently, ropbot does not support it.

5.6 Limitation

Although ROPBlock improves the state of the art of CR chain generation, it has its limitations.

Miss Optimization Opportunities. Due to the use of ROPBlock, ropbot may overlook optimization opportunities offered by directly using gadgets and thus generating longer chains. For example, to utilize `cmp rax,1; jne <label>; pop rbx; ret`, one needs to set `rax` to 1. It is clear that one can set `rax` once and then utilize the gadgets multiple times. But turning the gadget into ROPBlock will forcefully prepend the gadget with a chain that sets `rax`. As a result, every time we utilize the gadget in question, we will need to set `rax` again, which is unnecessary. But this issue can

be fixed by adding an optimization pass after a correct chain is generated to remove redundant gadgets.

Payload Generation on Discrete Controlled Regions. In real-world scenarios, there are often cases that we control multiple regions that are isolated from each other. For example, in Linux kernel exploitation, it is common that we control a small amount of data on stack and a larger region on the heap. In that case, it is preferrable to generate a chain that pivots onto the heap and start the code execution there. However, currently, ropbot does not have a way to reason and concludes the need of pivoting to the heap, let alone chaining data on multiple different regions to serve one payload generation request. This limitation may be resolved by generalizing the controlled-region-based graph search algorithm described in RetSpill [142].

5.7 Related Works

We propose a new technique to rethink code reuse attacks. Orthogonal research has focused on mitigations to prevent such attacks and data-only attacks.

Mitigations. Over the years, countless mitigations have been proposed to prevent, weaken, or contain memory corruption-based attacks. Among the traditional methods that made it into mainstream adoption are *Data Execution Prevention (DEP)* or *W^X*, both designed to thwart code injection attacks by making data pages non-executable. These defenses have led to the advent of code *reuse* attacks. Another widely popular technique is *Address Space Layout Randomization (ASLR)*, where the memory layout of a program is randomized upon startup to avoid predictable addresses. This raises the bar and requires attackers to find an information leak.

To prevent attackers from overwriting return addresses in the first place, stack

canaries [124], shadow stacks [11], or Control-Flow Integrity (CFI) [1] can be used. Stack canaries place guard values on the stack that abort execution when overwritten [25]. Similarly, shadow stacks place a copy of the return address on a second stack, such that simple stack-based buffer overflows can be detected. Recently, Intel's Control-flow Enforcement provides hardware support for shadow stacks and combines it with indirect branch tracking to identify control-flow hijacking via indirect control flow transfers. CFI (called Branch Target Indicators on ARM) validates that the destination of a control flow transfer is valid within the context of the program, limiting the attacker's control to destinations used by regular control flow. On ARM, Pointer Authentication Codes (PACs) allow signing and verifying pointers or return addresses, effectively thwarting control flow hijacking. All these defenses have seen varying degrees of success, with the attackers moving to new strategies to circumvent them. In this paper, we assume attackers have already bypassed these mitigations using other vulnerabilities or primitives (*e.g.*, use info leak to bypass stack canary) and obtained the control-flow hijacking primitive.

Data-Only Attacks. Beyond code reuse, another class of attacks has recently gained popularity: data-only attacks, or data-oriented programming. Other than code injection or code reuse attacks, where the attacker's first goal is to hijack the control flow, data-only attacks modify solely data without touching control flow-relevant information, such as return addresses. This way, they bypass any security policy monitoring the control flow, which is only indirectly affected through the modified data. Following an initial study [55], Hu *et al.* [56] proposed *data-oriented programming* and proved it to be Turing-complete for x86 programs in 2016. Subsequent work has proposed to automate the task of finding and stitching data-oriented gadgets [59, 90], with BOPC [59] being the first publicly available tool. Lately, Ling *et al.* [77]

use programming language synthesis to craft data-only exploits more efficiently and effectively. Despite promising to overcome mitigations such as CFI or shadow stacks, DOP is more limited than code reuse in terms of the capabilities it provides an attacker with. It has yet to see widespread adoption in practice, and the current state of the art requires simplifying targets manually before attacks can be synthesized [77]. ropbot is orthogonal to DOP and considers only the generation of gadget chains under the traditional code reuse attack model.

5.8 Conclusion

In this work, we apply exploit generalization in the field of automated CR payload generation. We collect and analyze existing CR chain generation techniques and identify gadget chainability as the dominating factor. By proposing a new architecture-agnostic abstraction called ROPBlock that guarantees chainability, we can now use a simple graph-search algorithm to drastically reduce the time complexity of the code reuse attack synthesis problem. Based on ROPBlock, we designed a research prototype, ropbot. We perform extensive evaluation on ropbot, against all existing works. And ropbot achieves unparalleled results: ropbot outperforms all existing tools significantly on all CR chain generation tasks. We also performed an ablation study to investigate where the improvement comes from. We believe this work is a solid step towards solving the code reuse attack synthesis problem in general, which again showcases the feasibility and potential of exploit generalization.

Chapter 6

CONCLUSION

Software exploits are a critical component of cyber attack campaigns. They consist of exploit techniques, revealing weaknesses in the target systems. However, lacking an approach to systematically study known exploits, although there are abundant sources of exploits, they are understudied. How to extract knowledge from and what can be learnt from known exploits remains unknown.

This dissertation aims to fill the gap. We propose the concept of exploit generalization, a three-step procedure to understand and generalize known software exploits, leading to a better understanding of the target program, identifying new attack surfaces, and the discovery of new exploitation techniques. Following this line of work, we identified four dominating factors in kernel heap-based exploit techniques and derived the Context Conservation technique. We pointed out that the user data access is the dominating factor for kernel CFHP exploits, identified the kernel stack as a new attack surface, and found the new Retspill exploitation technique. We found out that chainability is the dominating factor in code reuse payload generation, designed a new algorithm to guarantee chainability, and proposed the new ROPBlock-based payload generation technique.

REFERENCES

[1] Martín Abadi et al. "Control-Flow Integrity: Principles, Implementations, and Applications". In: *ACM Transactions on Information and System Security (TISSEC)* 13.1 (2009), pp. 1–40.

[2] Apple. *Towards the next generation of XNU memory safety: kalloc_type*. https://security.apple.com/blog/towards-the-next-generation-of-xnu-memory-safety/. 2022.

[3] Jeff Arnold et al. "Security impact ratings considered harmful". In: *arXiv preprint arXiv:0904.4058* (2009).

[4] Thanassis Avgerinos et al. "AEG: Automatic Exploit Generation". In: *Proceedings of the 2016 Network and Distributed System Security Symposium (NDSS)*. 2011.

[5] Brandon Azad. *A survey of recent iOS kernel exploits*. https://googleprojectzero.blogspot.com/2020/06/a-survey-of-recent-ios-kernel-exploits.html. 2020.

[6] Brandon Azad. *Examining Pointer Authentication on the iPhone XS*. https://googleprojectzero.blogspot.com/2019/02/examining-pointer-authentication-on.html. 2019.

[7] Nicolas Bailluet et al. "Nothing is Unreachable: Automated Synthesis of Robust {Code-Reuse} Gadget Chains for Arbitrary Exploitation Primitives". In: *34th USENIX Security Symposium (USENIX Security 25)*. 2025, pp. 625–643.

[8] Tiffany Bao et al. "Your Exploit is Mine: Automatic Shellcode Transplant for Remote Exploits". In: *Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P)*. 2017.

[9] Tyler Bletsch et al. "Jump-Oriented Programming: A New Class of Code-Reuse Attack". In: *ACM Symposium on Information, Computer and Communications Security (ASIACCS)*. 2011.

[10] David Brumley et al. "Automatic Patch-Based Exploit Generation is Possible: Techniques and Implications". In: *Proceedings of the 29th IEEE Symposium on Security and Privacy (S&P)*. 2008.

[11] Nathan Burow, Xinping Zhang, and Mathias Payer. "SoK: Shining Light on Shadow Stacks". In: *IEEE Symposium on Security and Privacy (S&P)*. 2019.

[12] *CFS Scheduler — The Linux Kernel documentation*. https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html.

[13] Stephen Checkoway et al. "Return-Oriented Programming without Returns". In: *ACM Conference on Computer and Communications Security (CCS)*. 2010.

[14] Hao Chen, David Wagner, and Drew Dean. "Setuid demystified". In: *11th USENIX Security Symposium (USENIX Security 02)*. 2002.

[15] Weiteng Chen et al. "KOOBE: Towards Facilitating Exploit Generation of Kernel Out-Of-Bounds Write Vulnerabilities". In: *Proceedings of the 29th USENIX Security Symposium (USENIX Security)*. 2020.

[16] Yueqi Chen, Zhenpeng Lin, and Xinyu Xing. "A Systematic Study of Elastic Objects in Kernel Exploitation". In: *Proceedings of the 27th ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2020.

[17] Yueqi Chen and Xinyu Xing. "SLAKE: Facilitating Slab Manipulation for Exploiting Vulnerabilities in the Linux Kernel". In: *Proceedings of the 26th ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2019.

[18] Haehyun Cho et al. "Exploiting Uses of Uninitialized Stack Variables in Linux Kernels to Leak Kernel Pointers". In: *14th USENIX Workshop on Offensive Technologies (WOOT)*. 2020.

[19] Tobias Cloosters et al. "RiscyROP: Automated Return-Oriented Programming Attacks on RISC-V and ARM64". In: *Symposium on Recent Advances in Intrusion Detection (RAID)*. 2022.

[20] Kees Cook. *Hardened usercopy*. https://lwn.net/Articles/693745/. 2016.

[21] Kees Cook. *Kernel address space layout randomization*. https://lwn.net/Articles/546035/. 2013.

[22] Jonathan Corbet. *Supervisor mode access prevention*. https://lwn.net/Articles/517475/. 2012.

[23] d4em0n. *exrop*. 2020. URL: https://github.com/d4em0n/exrop.

[24] Dino Dai Zovi. "Practical return-oriented programming". In: *Source boston* (2010).

[25] Thurston HY Dang, Petros Maniatis, and David Wagner. "The Performance Cost of Shadow Stacks and Stack Canaries". In: *ACM Symposium on Information, Computer and Communications Security (ASIACCS)*. 2015.

[26] Lucas Davi et al. "PT-Rand: Practical Mitigation of Data-only Attacks against Page Tables." In: *NDSS*. 2017.

[27] Vincent Dehors. *Exploitation of a double free vulnerability in Ubuntu shiftfs driver*. https://www.synacktiv.com/en/publications/exploitation-of-a-double-free-vulnerability-in-ubuntu-shiftfs-driver-cve-2021-3492.html. 2021.

[28] Mark DenHoed and Tom Melham. "Synthesis of Code-Reuse Attacks from p-code Programs". In: *USENIX Security Symposium*. 2025.

[29] Rémi Denis-Courmont et al. "Camouflage: Hardware-assisted CFI for the ARM Linux kernel". In: *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE. 2020, pp. 1–6.

[30] Marco Elver. *stack: Introduce CONFIG_RANDOMIZE_KSTACK_OFFSET*. https://lore.kernel.org/lkml/YfQ54x8zglPT%2FYnL@dev-arch.archlinux-ax161/t/#u. 2022.

[31] *Exploit Database*. URL: https://www.exploit-db.com/.

[32] Nicolas FABRETTI. *CVE-2017-11176: A step-by-step Linux Kernel exploitation*. https://blog.lexfo.fr/cve-2017-11176-linux-kernel-exploitation-part4.html.

[33] Jorge Fandinno and Lute Lillo. "Solving Epistemic Logic Programs using Generate-and-Test with Propagation". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 14. 2025, pp. 14902–14910.

[34] FizzBuzz101. *Will's Root: CVE-2022-0185*. https://www.willsroot.io/2022/01/cve-2022-0185.html. 2022.

[35] Xinyang Ge et al. "Fine-grained control-flow integrity for kernel software". In: *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE. 2016, pp. 179–194.

[36] Enes Göktas et al. "Speculative probing: Hacking blind in the Spectre era". In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 2020, pp. 1871–1885.

[37] Google. *CVE-2023-3390*. 2023. URL: https://github.com/google/security-research/blob/master/pocs/linux/kernelctf/CVE-2023-3390_lts_cos_mitigation/exploit/lts-6.1.31/exploit.c.

[38] Google. *CVE-2023-4004*. 2023. URL: https://github.com/google/security-research/blob/master/pocs/linux/kernelctf/CVE-2023-4004_lts_cos_mitigation/exploit/mitigation-6.1/exploit.c.

[39] Google. *Kernel Control Flow Integrity*. https://source.android.com/docs/security/test/kcfi.

[40] Google. *Kernel Exploit Recipes Notebook - Google Docs*. https://docs.google.com/document/d/1a9uUAISBzw3ur1aLQqKc5JOQLaJYiOP5pe_B4xCT1KA/edit.

[41] Google. *kernelCTF*. 2023. URL: https://google.github.io/security-research/kernelctf/rules.html.

[42] Google. *syzbot*. https://syzkaller.appspot.com/. 2017.

[43] Google. *syzbot - Found Bugs*. https://syzkaller.appspot.com/upstream/graph/found-bugs. 2017.

[44] Google. *Vulnerability Reward Program: 2020 in Review*. https://security.googleblog.com/2021/02/vulnerability-reward-program-2020-year.html. 2021.

[45] Google. *Vulnerability Reward Program: 2021 in Review*. https://security.googleblog.com/2022/02/vulnerability-reward-program-2021-year.html. 2022.

[46] Google. *Vulnerability Reward Program: 2022 in Review*. https://security.googleblog.com/2023/02/vulnerability-reward-program-2022-year.html. 2023.

[47] Google. *Vulnerability Reward Program: 2023 in Review*. https://security.googleblog.com/2024/03/vulnerability-reward-program-2023-year.html. 2024.

[48] Google. *Vulnerability Reward Program: 2024 in Review*. https://security.googleblog.com/2025/03/vulnerability-reward-program-2024-in.html. 2025.

[49] Grimm. *NotQuite0DayFriday/2021.03.12-linux-iscsi at trunk · grimm-co/NotQuite0DayFriday*. https://github.com/grimm-co/NotQuite0DayFriday/tree/trunk/2021.03.12-linux-iscsi.

[50] Daniel Gruss et al. "Kaslr is dead: long live kaslr". In: *International Symposium on Engineering Secure Software and Systems*. Springer. 2017, pp. 161–176.

[51] Daniel Gruss et al. "Prefetch side-channel attacks: Bypassing SMAP and kernel ASLR". In: *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 2016, pp. 368–379.

[52] Garrett Gu and Hovav Shacham. "Return-oriented programming in risc-v". In: *arXiv preprint arXiv:2007.14995* (2020).

[53] Sean Heelan, Tom Melham, and Daniel Kroening. "Automatic Heap Layout Manipulation for Exploitation". In: *Proceedings of the 27th USENIX Security Symposium (USENIX Security)*. 2018.

[54] Sean Heelan, Tom Melham, and Daniel Kroening. "Gollum: Modular and Greybox Exploit Generation for Heap Overflows in Interpreters". In: *Proceedings of the 26th ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2019.

[55] Hong Hu et al. "Automatic Generation of Data-Oriented Exploits". In: *USENIX Security Symposium*. 2015.

[56] Hong Hu et al. "Data-Oriented Programming: On the Expressiveness of Non-control Data Attacks". In: *IEEE Symposium on Security and Privacy (S&P)*. 2016.

[57] Ralf Hund, Carsten Willems, and Thorsten Holz. "Practical Timing Side Channel Attacks Against Kernel Space ASLR". In: *Proceedings of the 34th IEEE Symposium on Security and Privacy (S&P)*. 2013.

[58] Isovalent. *Detecting a Container Escape with Cilium and eBPF - Isovalent*. https://isovalent.com/blog/post/2021-11-container-escape/. 2021.

[59] Kyriakos K Ispoglou et al. "Block-Oriented Programming: Automating Data-only Attacks". In: *ACM Conference on Computer and Communications Security (CCS)*. 2018.

[60] Yeongjin Jang, Sangho Lee, and Taesoo Kim. "Breaking Kernel Address Space Layout Randomization with Intel TSX". In: *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2016.

[61] David Bouman Jayden Rivers. *CVE-2022-29582: An io_uring vulnerability*. https://ruia-ruia.github.io/2022/08/05/CVE-2022-29582-io-uring/. 2022.

[62] Seth Jenkins. *Exploiting CVE-2022-42703 - Bringing back the stack attack*. https://googleprojectzero.blogspot.com/2022/12/exploiting-CVE-2022-42703-bringing-back-the-stack-attack.html. 2022.

[63] Xingyu Jin et al. *Monitoring Surveillance Vendors: A Deep Dive into In-the-Wild Android Full Chains in 2021*. https://i.blackhat.com/USA-22/Wednesday/US-22-Jin-Monitoring-Surveillance-Vendors.pdf. 2022.

[64] Vasileios P Kemerlis, Michalis Polychronakis, and Angelos D Keromytis. "ret2dir: Rethinking kernel isolation". In: *23rd USENIX Security Symposium (USENIX Security 14)*. 2014, pp. 957–972.

[65] Vasileios P Kemerlis, Georgios Portokalidis, and Angelos D Keromytis. "{kGuard}: Lightweight Kernel Protection against {Return-to-User} Attacks". In: *21st USENIX Security Symposium (USENIX Security 12)*. 2012, pp. 459–474.

[66] Andrey Konovalov. *Linux Kernel Exploitation*. https://github.com/xairy/linux-kernel-exploitation. 2020.

[67] Andrey Konovalov. *Project Zero: Exploiting the Linux kernel via packet sockets*. https://googleprojectzero.blogspot.com/2017/05/exploiting-linux-kernel-via-packet.html.

[68] Alexander Lobakin Kristen Carlson Accardi. *Function Granular KASLR [LWN.net]*. https://lwn.net/Articles/832434/. 2020.

[69] Greg Kroah-Hartman. *Introduce STATIC_USERMODEHELPER to mediate call_usermodehelper() - Patchwork*. https://lore.kernel.org/all/20170116165044.GC29693@kroah.com/. 2017.

[70] Donghyun Kwon et al. "VM-CFI: Control-flow integrity for virtual machine kernel using Intel PT". In: *International Conference on Computational Science and Its Applications*. Springer. 2018, pp. 127–137.

[71] Dang Le. *Learning Linux Kernel Exploitation - Part 1*. https://lkmidas.github.io/posts/20210123-linux-kernel-pwn-part-1/#the-simplest-exploit---ret2usr.

[72] Dang Le. *Learning Linux Kernel Exploitation - Part 3 - Midas Blog*. https://lkmidas.github.io/posts/20210205-linux-kernel-pwn-part-3/. 2021.

[73] Yoochan Lee, Changwoo Min, and Byoungyoung Lee. "{ExpRace}: Exploiting kernel races through raising interrupts". In: *30th USENIX Security Symposium (USENIX Security 21)*. 2021, pp. 2363–2380.

[74] Zhenpeng Lin. *How AUTOSLAB Changes the Memory Unsafety Game*. https://grsecurity.net/how_autoslab_changes_the_memory_unsafety_game. 2021.

[75] Zhenpeng Lin, Yuhang Wu, and Xinyu Xing. "DirtyCred: Escalating Privilege in Linux Kernel". In: *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2022.

[76] Zhenpeng Lin et al. "GREBE: Unveiling Exploitation Potential for Linux Kernel Bugs". In: *2022 IEEE Symposium on Security and Privacy (S&P)*. IEEE. 2022, pp. 2078–2095.

[77] Yuxi Ling et al. "Sound and Efficient Generation of Data-Oriented Exploits via Programming Language Synthesis". In: *USENIX Security Symposium*. 2025.

[78] Linxz. *PaX - structleak*. https://linxz.tech/post/compilers/2021-10-10-structleak/. 2021.

[79] Moritz Lipp et al. "Meltdown: Reading Kernel Memory from User Space". In: *Proceedings of the 27th USENIX Security Symposium (USENIX Security)*. 2018.

[80] William Liu, Joseph Ravichandran, and Mengjia Yan. "Entrybleed: A universal kaslr bypass against kpti on linux". In: *Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy*. 2023, pp. 10–18.

[81] Kangjie Lu et al. "Unisan: Proactive kernel memory initialization to eliminate data leakages". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 920–932.

[82] Kangjie Lu et al. "Unleashing Use-Before-Initialization Vulnerabilities in the Linux Kernel Using Targeted Stack Spraying". In: *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*. 2017.

[83] Contributors to Malware Database Wikia. *WannaCry Ransomware | Malware Database Wikia | Fandom*. URL: https://malware-history.fandom.com/wiki/WannaCry_Ransomware.

[84] Hector Marco-Gisbert and Ismael Ripoll. "Return-to-csu: A New Method to Bypass 64-bit Linux ASLR". In: *Black Hat Asia*. 2018.

[85] Nicholas Nethercote and Julian Seward. "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation". In: *ACM Sigplan Notices* 42.6 (2007), pp. 89–100.

[86] Andy Nguyen. *CVE-2021-22555: Turning \x00\x00 into 10000$ | security-research*. https://google.github.io/security-research/pocs/linux/cve-2021-22555/writeup.html. 2021.

[87] Gene Novark and Emery D Berger. "Dieharder: securing the heap". In: *Proceedings of the 17th ACM conference on Computer and communications security*. 2010, pp. 573–584.

[88] Masao Ohira et al. "A Dataset of High Impact Bugs: Manually-Classified Issue Reports". In: *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. 2015.

[89] OSDev.org. *Supervisor Memory Protection - OSDev Wiki*. https://wiki.osdev.org/Supervisor_Memory_Protection.

[90] Jannik Pewny, Philipp Koppe, and Thorsten Holz. "STEROIDS for DOPed Applications: A Compiler for Automated Data-Oriented Programming". In: *IEEE European Symposium on Security and Privacy (EuroS&P)*. 2019.

[91] *Phoronix Test Suite*. http://www.phoronix-test-suite.com/. 2015.

[92] Marios Pomonis et al. "kR^X: Comprehensive kernel protection against just-in-time code reuse". In: *Proceedings of the Twelfth European Conference on Computer Systems*. 2017, pp. 420–436.

[93] Alexander Popov. *CVE-2017-2636: Exploit the race condition in the n_hdlc Linux kernel driver*. https://a13xp0p0v.github.io/2017/03/24/CVE-2017-2636.html. 2017.

[94] Alexander Popov. *STACKLEAK: A Long Way to the Linux Kernel Mainline*. https://events19.linuxfoundation.org/wp-content/uploads/2017/11/STACKLEAK-A-Long-Way-to-the-Linux-Kernel-Mainline-Alexander-Popov-Positive-Technologies.pdf. 2018.

[95] Alexander Potapenko. *security: allow using Clang's zero initialization for stack variables*. https://lwn.net/Articles/823152/. 2020.

[96] Sergej Proskurin et al. "xmp: Selective memory protection for kernel and user space". In: *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2020, pp. 563–577.

[97] Elena Reshetova. *randomize kernel stack offset upon syscall*. https://lwn.net/Articles/785484/. 2019.

[98] Frank E. Ritter, Gordon D. Baxter, and Elizabeth F. Churchill. *Foundations for Designing User-Centered Systems. What System Designers Need to Know about People*. Springer, 2014.

[99] Matteo Rizzo. *Kernote Writeup*. https://org.anize.rs/0CTF-2021-finals/pwn/kernote. 2021.

[100] AliAkbar Sadeghi, Salman Niksefat, and Maryam Rostamipour. "Pure-Call Oriented Programming (PCOP): Chaining the Gadgets using call Instructions". In: *Journal of Computer Virology and Hacking Techniques* 14 (2018), pp. 139–156.

[101] Jonathan Salwan. *ROPgadget*. 2011. URL: https://github.com/JonathanSalwan/ROPgadget.

[102] Samsung. *Real-time Kernel Protection (RKP)*. https://www.samsungknox.com/en/blog/real-time-kernel-protection-rkp. 2016.

[103] Sascha Schirra. *Ropper*. 2015. URL: https://github.com/sashs/Ropper.

[104] Moritz Schloegel et al. "Towards automating code-reuse attacks using synthesized gadget chains". In: *Computer Security–ESORICS 2021: 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part I 26*. Springer. 2021, pp. 218–239.

[105] Edward J Schwartz, Thanassis Avgerinos, and David Brumley. "Q: Exploit hardening made easy". In: *20th USENIX Security Symposium (USENIX Security 11)*. 2011.

[106] Edward J Schwartz et al. "A Generic Technique for Automatically Finding Defense-aware Code Reuse Attacks". In: *ACM Conference on Computer and Communications Security (CCS)*. 2020.

[107] Hovav Shacham. "The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86)". In: *Proceedings of the 14th ACM conference on Computer and communications security*. 2007, pp. 552–561.

[108] Yan Shoshitaishvili et al. "Firmalice - Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware". In: *Proceedings of the 2015 Network and Distributed System Security Symposium (NDSS)*. 2015.

[109] Yan Shoshitaishvili et al. "Sok: (State of) the Art of War: Offensive Techniques in Binary Analysis". In: *IEEE Symposium on Security and Privacy (S&P)*. 2016.

[110] Kevin Z Snow et al. "Just-In-Time Code Reuse: On the Effectiveness of Fine-grained Address Space Layout Randomization". In: *IEEE Symposium on Security and Privacy (S&P)*. 2013.

[111] J Staff. *Assembled labeled library for static analysis research (allstar) dataset*. 2019.

[112] statcounter. *Browser market share*. https://gs.statcounter.com/browser-market-share. 2025.

[113] Nick Stephens et al. "Driller: Augmenting Fuzzing Through Selective Symbolic Execution". In: *Proceedings of the 2016 Network and Distributed System Security Symposium (NDSS)*. 2016.

[114] angr team. *angr/angrop*. https://github.com/angr/angrop.

[115] Sami Tolvanen. *KCFI support [LWN.net]*. https://lwn.net/Articles/893164/. 2022.

[116] Linus Torvalds. *Kernel Stack documentation*. https://docs.kernel.org/x86/kernel-stacks.html.

[117] Linus Torvalds. *Linux kernel calling convention source code*. https://elixir.bootlin.com/linux/v5.17/source/arch/x86/entry/calling.h#L18.

[118] Linus Torvalds. *native_write_cr4*. https://elixir.bootlin.com/linux/v6.0/source/arch/x86/kernel/cpu/common.c#L447.

[119] Linus Torvalds. *Page Table Isolation (PTI) documentation*. https://www.kernel.org/doc/html/latest/x86/pti.html.

[120] Linux Torvalds. *Linux kernel source code: fs/select.c*. https://elixir.bootlin.com/linux/v5.17/source/fs/select.c#L982.

[121] Linux Torvalds. *Linux kernel source: arch/x86/include/asm/ptrace.h*. https://elixir.bootlin.com/linux/v5.17/source/arch/x86/include/asm/ptrace.h#L59.

[122] Linux Torvalds. *Linux kernel source: fs/read_write.c*. https://elixir.bootlin.com/linux/v5.17/source/fs/read_write.c#L899.

[123] Paul Turner. *Retpoline: A Software Construct for Preventing Branch-target-injection*. 2018. URL: https://support.google.com/faqs/answer/7625886.

[124] Perry Wagle, Crispin Cowan, et al. "Stackguard: Simple Stack Smash Protection for GCC". In: *GCC Developers Summit*. 2003.

[125] Lun Wang et al. "{PrivGuard}: Privacy Regulation Compliance Made Easier". In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 3753–3770.

[126] Yan Wang et al. "Revery: From Proof-of-Concept to Exploitable". In: *Proceedings of the 25nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2018.

[127] Yu Wang. "A New CVE-2015-0057 Exploit Technology". In: *BlackHat Asia*. 2015.

[128] WangYihang. *Socket Reuse Shellcode*. 2014. URL: https://www.exploit-db.com/exploits/34060.

[129] Yuan Wei et al. "Arg: Automatic rop chains generation". In: *IEEE Access* 7 (2019), pp. 120152–120163.

[130] *What is LockBit Ransomware | Akamai*. URL: https://www.akamai.com/glossary/what-is-lockbit-ransomware.

[131] *What is RCU, Fundamentally? [LWN.net]*. https://lwn.net/Articles/262464/.

[132] willsroot. *EntryBleed*. https://www.willsroot.io/2022/12/entrybleed.html. 2022.

[133] Wei Wu et al. "{KEPLER}: Facilitating control-flow hijacking primitive evaluation for Linux kernel vulnerabilities". In: *28th USENIX security symposium (USENIX security 19)*. 2019, pp. 1187–1204.

[134] Wei Wu et al. "FUZE: Towards Facilitating Exploit Generation for Kernel Use-After-Free Vulnerabilities". In: *Proceedings of the 27th USENIX Security Symposium (USENIX Security)*. 2018.

[135] Wen Xu et al. “From collision to exploitation: Unleashing use-after-free vulnerabilities in linux kernel”. In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 2015, pp. 414–425.

[136] Xi Yang et al. “Why nothing matters: The impact of zeroing”. In: *Acm Sigplan Notices* 46.10 (2011), pp. 307–324.

[137] Sungbae Yoo et al. “{In-Kernel}{Control-Flow} Integrity on Commodity {OSes} using {ARM} Pointer Authentication”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 89–106.

[138] Insu Yun, Dhaval Kapil, and Taesoo Kim. “Automatic Techniques to Systematically Discover New Heap Exploitation Primitives”. In: *Proceedings of the 29th USENIX Security Symposium (USENIX Security)*. 2020.

[139] Kyle Zeng. *[CVE-2022-1786] A Journey To The Dawn*. https://blog.kylebot.net/2022/10/16/CVE-2022-1786/.

[140] Kyle Zeng. *KHeaps*. https://github.com/sefcom/KHeaps/blob/master/exploit_env/CVEs/CVE-2017-7533/poc/poc_cfh_combo.c.

[141] Kyle Zeng et al. “Playing for {K (H) eaps}: Understanding and Improving Linux Kernel Exploit Reliability”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 71–88.

[142] Kyle Zeng et al. “Retspill: Igniting user-controlled data to burn away linux kernel protections”. In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, pp. 3093–3107.

[143] Peter Zijlstra. *x86: Kernel IBT beginnings*. https://lwn.net/ml/linux-kernel/20211122170301.764232470@infradead.org/. 2021.

[144] Xiaochen Zou et al. “SyzScope: Revealing High-Risk Security Impacts of Fuzzer-Exposed Bugs in Linux kernel”. In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 3201–3217.

APPENDIX A

PERMISSION FROM CO-AUTHORS

Permission has been obtained from all co-authors for use of the following published works in this dissertation:

- Kyle Zeng et al. "Playing for {K (H) eaps}: Understanding and Improving Linux Kernel Exploit Reliability". In: *31st USENIX Security Symposium (USENIX Security 22)*. 2022, pp. 71–88.
- Kyle Zeng et al. "Retspill: Igniting user-controlled data to burn away linux kernel protections". In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2023, pp. 3093–3107.